ABAP INTERVIEW PREPARATION

ABAP interview handbook

Senior ABAP Cloud, RAP and Clean Core first; supporting ABAP and integration knowledge

329 questions across 24 study segments. Reviewed 11 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Read online in GitHub through README.md, or download the repository and open START_HERE.html for offline recall cards. The PDF is a portable reading and printing edition.

Study segments

Senior ABAP Cloud architecture decisions / 18 questions

RAP foundations and transactions / 15 questions

ABAP Cloud: senior RAP transactions and behavior design / 24 questions

CDS data modeling / 12 questions

Senior CDS, query contracts and security / 18 questions

RAP service design and advanced behavior / 13 questions

Senior RAP testing, performance and production / 18 questions

Clean Core levels and extensibility / 18 questions

Senior Clean Core governance and tradeoffs / 18 questions

ABAP foundations / 14 questions

Practical ABAP: values, expressions, and failure paths / 16 questions

OO design and Clean ABAP / 15 questions

Practical OO: contracts, state, and substitutability / 12 questions

Practical ABAP design patterns and trade-offs / 12 questions

ABAP SQL, HANA, and performance / 14 questions

OData Gateway and Fiori integration / 12 questions

BTP and cloud integration for ABAP developers / 10 questions

RICEFW, enhancements, and workflow / 15 questions

Testing security and delivery / 10 questions

Interview answers and project discussion / 6 questions

HTTP, REST, JSON and XML / 12 questions

Function modules, RFC, BAPI, parallel processing, and jobs / 12 questions

IDoc, ALE, AIF, and interface operations / 12 questions

Supporting skills: HANA decisions and source-to-release delivery / 3 questions

Answer structure

Start with a definition and one useful example. Add the main trade-off or failure mode. If syntax or support depends on the SAP release, name that dependency and explain how you would verify it.

Use the practice workbook for scenarios and mock interviews. Use the quick revision PDF for the final review. References at the end of each segment link to official primary documentation. They support the concepts; examples and rehearsal advice are original instructional material.

Senior ABAP Cloud architecture decisions

Treat each question as a design review: state the business invariant, choose a boundary, explain the rejected alternative, and name the evidence that would validate the decision. ABAP Cloud is the main development model here; deployment, API scope, and operational constraints still differ across SAP products. These scenarios demand decisions and verification rather than memorized framework definitions.

Choose an extension approach

Start with the business requirement, supported contracts and transaction boundary

First	**Can standard functionality or configuration meet the need?**	Use the standard capability and avoid unnecessary custom code.
Key user	**Does a delivered business context cover the adaptation?**	Use supported custom fields, logic or UI adaptation tools.
On-stack	**Does the logic need local transactional consistency?**	Use released local contracts and suitable ABAP Cloud extensibility.
Side-by-side	**Does the process need an independent or cross-system app?**	Use supported remote APIs and events; design retries and consistency.

Placement does not determine the Clean Core level. Check dependencies separately.

Start from the business need and transaction boundary before choosing an extension location.

See the visual atlas for the recall prompt, limits and related questions.

CLOUD-01 A new approval extension needs immediate document validation and an external supplier portal. Would you put everything on-stack or on BTP?

I would split only where the business and transaction boundaries justify it. Validation that must reject the core document belongs at a supported on-stack extension point when available. The supplier portal may fit side-by-side because its users, scaling, and release lifecycle differ. I would define released remote contracts and tolerate explicit asynchronous states between the two. The decision depends on available APIs, latency, identity, operational ownership, and failure recovery, not a blanket preference for BTP.

Example and follow-up If the portal calls a remote service during the core save, a supplier-network outage could block all internal posting. Separating a committed submission from later supplier interaction avoids that coupling, but requires visible pending and failed states, retry correlation, and a business rule for withdrawal after submission.

Avoid this mistake Deploying side-by-side does not make synchronous cross-system validation atomic or automatically clean-core compliant.

Interviewer probes Which invariant must be enforced before the core commit? / How does the user recover when the portal accepts a request but the core is unavailable?

Strong answer criteria Separates business and runtime boundaries; Names latency and lifecycle tradeoffs; Defines partial-failure recovery

Verification evidence An architecture decision record, target API availability checks, and a failure-sequence test showing pending, retry, and withdrawal outcomes.

Read and verify this answer

SAP extension architecture guidance

CLOUD-02 A stakeholder says ABAP Cloud means moving the existing application to SAP BTP ABAP environment. How would you correct the proposal?

I would separate the development model from the runtime. ABAP Cloud combines restricted language versions, released dependencies, modern tooling, and programming models such as RAP; it is available in supported S/4HANA environments as well as SAP BTP ABAP environment. Moving code to BTP changes data locality, connectivity, operations, and available business APIs. I would first identify the required product capabilities and choose deployment deliberately, then assess the application's compatibility with that environment.

Example and follow-up A class that reads local sales data in an S/4HANA system cannot assume the same application tables or local APIs exist in a BTP ABAP system. A remote service, data replication, or a different ownership model may be necessary even if the ABAP language syntax is valid.

Avoid this mistake Cloud-compatible syntax does not establish identical application content, API availability, or deployment portability.

Interviewer probes Which dependencies become remote after the move? / What target-system evidence would you require before estimating migration?

Strong answer criteria Distinguishes model from runtime; Identifies lost data locality; Checks product-specific capabilities

Verification evidence A target capability matrix mapping each dependency to its supported local or remote equivalent and a tested integration spike.

Read and verify this answer

SAP ABAP Cloud development model

CLOUD-03 A developer proposes dynamic calls to an unreleased SAP class because static ABAP Cloud checks reject it. What do you do?

I would reject bypassing the development contract and investigate the actual capability gap. First confirm the object's release status, visibility, and a suitable released alternative on the target version. Then reconsider standard functionality, another extension point, or a supported remote API. If a private-edition classic boundary is genuinely needed, isolate and govern it explicitly. Dynamic invocation must not disguise a prohibited dependency or be presented as equivalent to a released interface.

Example and follow-up A dynamic call can fail after an upgrade even when a superficial static scan no longer identifies the dependency. It may also be rejected by the language or runtime. The useful prototype is a supported replacement path with representative inputs, not a demonstration that one forbidden call happened to work once.

Avoid this mistake An ATC suppression or dynamic name does not create a supported API contract.

Interviewer probes What options remain in Public Edition when no released extension exists? / How would you track an API gap without blocking unrelated delivery?

Strong answer criteria Rejects hidden dependency bypass; Verifies target-specific alternatives; Separates private-edition exceptions from public-cloud rules

Verification evidence ADT API-state evidence, a documented capability gap, and a decision record showing supported alternatives and any approved bounded exception.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-04 Two teams want independent releases but share pricing helpers and persistence types. How would you structure packages and software components?

I would identify which shared capabilities can carry a stable contract and which require coordinated releases. Packages organize implementation and access boundaries; software components also define lifecycle and delivery dependencies. A stable pricing interface can belong to a reuse component, while private persistence types should remain with their owner. Where supported, component relations permit targeted internal access, but they are not a substitute for a compatibility commitment. I would avoid cycles and make import order explicit.

Example and follow-up If both teams depend on each other's database row types, an allegedly independent release may require simultaneous activation everywhere. Replacing those types with a narrow request/result contract reduces coupling. If the contract is not stable yet, a coordinated release may be more honest than promising independence through extra components.

Avoid this mistake More software components do not create independent lifecycles when their dependencies still require lockstep delivery.

Interviewer probes When would you choose a component relation instead of a C1 API? / How would an incompatible dependency removal be detected?

Strong answer criteria Models dependency direction; Distinguishes access permission from stability; Aligns boundaries with delivery reality

Verification evidence A dependency graph without cycles, explicit component relations, a transport/import sequence, and compatibility checks against the prior delivered API snapshot.

Read and verify this answer

SAP software component dependencies and API snapshots

SAP software component organization and lifecycle boundaries

CLOUD-05 The same business capability is exposed through a local released API and a remote service. Which should an on-stack application consume?

I would compare the actual contracts rather than select by familiarity. A suitable local API can avoid network overhead and may participate in the supported local transaction model. A remote service provides a separate integration boundary, identity, and lifecycle, but introduces latency and independent failure semantics. Local and remote variants may expose different operations or consistency guarantees. I would choose the one whose authorization, transaction, availability, and release contract fits the caller.

Example and follow-up Calling the system's own HTTP endpoint from inside a save routine can create another session, lock contention, and confusing commit behavior. Conversely, a local read API does not replace a remote business command simply because both mention the same business object. Compare operation semantics and test concurrent access.

Avoid this mistake C1 local consumption and C2 remote consumption are different purposes; one release contract does not authorize the other.

Interviewer probes Can the chosen local command participate in this RAP save phase? / What changes if the consumer moves to BTP next year?

Strong answer criteria Compares full contracts; Recognizes self-call transaction risks; Plans a replaceable consumption boundary

Verification evidence Documented local and remote API contracts plus a transaction and authorization test for the chosen consumption path.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-06 A CDS object compiles in the development system but fails after delivery to an older supported release. How would you prevent recurrence?

I would establish a supported product and release matrix before choosing provider contracts, CDS features, binding types, and APIs. Compilation on one system proves only that configuration. I would separate broadly supported domain contracts from newer delivery features where feasible and test the lowest supported target. API release visibility and language restrictions must be checked alongside syntax. Unsupported features require a compatible design, an explicit support-floor change, or a separately maintained variant.

Example and follow-up A projection provider contract or service binding available in one release can be absent in another. Copying an example from current documentation may therefore fail during activation rather than at runtime. A compatibility spike should include dependent CDS, behavior, service artifacts, and a representative request, not just one class.

Avoid this mistake An object being active in development does not prove compatibility with every product edition or supported transport target.

Interviewer probes Where would you document the minimum feature level? / Which differences justify a versioned implementation rather than a shared code path?

Strong answer criteria Sets a concrete support matrix; Tests the lowest supported release; Includes dependent repository artifacts

Verification evidence A versioned compatibility matrix, target activation evidence, and service tests tied to the exact supported release and delivery revision.

Read and verify this answer

SAP ABAP Cloud development model

SAP released APIs, contracts, visibility, and states

CLOUD-07 Should a request, its approval history, and an external purchase order be one RAP business object?

I would choose the business-object boundary around invariants, ownership, lifecycle, and transactional consistency, not screen layout. Request items that cannot exist independently may belong in its composition tree. Approval history may have separate retention and immutability requirements. An external purchase order has another owner and transaction boundary, so I would normally reference it through a supported contract. Cross-boundary consistency becomes an explicit workflow or reconciliation concern rather than a fictional shared database transaction.

Example and follow-up Deleting a request should not accidentally delete legally relevant approval evidence or attempt to cascade into another system. Conversely, splitting tightly coupled request lines into unrelated objects can make total validation and locking unnecessarily difficult. Model allowed transitions and failure states before drawing the CDS composition hierarchy.

Avoid this mistake A composition relationship is not a convenient visual grouping and cannot provide atomicity across an external system.

Interviewer probes Which invariant requires one commit? / How do retention and cancellation rules change your composition tree?

Strong answer criteria Starts from business invariants; Distinguishes ownership and retention; Designs cross-boundary consistency

Verification evidence An invariant table, lifecycle/state model, composition rationale, and tests for deletion, cancellation, and remote-posting failure.

Read and verify this answer

SAP extension architecture guidance

CLOUD-08 A partner wants one ABAP solution to serve several customers. What isolation questions must be answered before calling it multitenant?

I would verify the target runtime's supported multitenancy model and treat tenant identity as a security boundary throughout persistence, configuration, caching, jobs, logs, and integration destinations. Ordinary client handling is only part of that analysis and does not prove SaaS isolation. Provider operations and customer access need separate responsibilities. I would design onboarding, upgrades, offboarding, and data deletion explicitly, then require negative tests showing that one tenant cannot observe or influence another.

Example and follow-up A static cache keyed only by document number can leak a result if different tenants use the same number and the runtime scope permits reuse. An outbound destination selected from the wrong tenant context can expose payloads even when database queries are correctly scoped. Isolation testing must cover both paths.

Avoid this mistake Adding a tenant field or relying on automatic client filtering alone does not establish complete multitenancy isolation.

Interviewer probes Which caches or jobs outlive one request? / How do you prove offboarding removes access without damaging another tenant?

Strong answer criteria Covers non-database isolation paths; Uses supported tenant context; Includes tenant lifecycle operations

Verification evidence A tenant-boundary threat model, two-tenant negative integration tests, and an onboarding/offboarding runbook validated in the target runtime.

Read and verify this answer

SAP ABAP development and multitenancy guidance

CLOUD-09 A business team requests a new field, a rule, and a custom app. How do you decide between configuration, key-user, and developer extensibility?

I would decompose the request by purpose and first check standard configuration and delivered extensibility. A supported custom-field scenario may already propagate the field through relevant services and UIs. A small rule can fit key-user logic when its execution context and limits match. A custom transactional application may need developer extensibility. I would keep ownership, transport, validation, and field semantics consistent across these mechanisms rather than rebuild everything in code for convenience.

Example and follow-up A developer-added field in a custom table may not appear in standard forms or analytics, while a supported key-user field can provide those publication paths. Conversely, putting complex orchestration into a limited key-user hook can create unsupported coupling. Verify the exact business context and downstream propagation requirements.

Avoid this mistake Key-user extensibility is not inherently less serious engineering, and developer extensibility is not a reason to duplicate standard configuration.

Interviewer probes Who owns a rule that business users can change? / How will the field reach the required integration and output artifacts?

Strong answer criteria Performs fit-to-standard decomposition; Checks propagation and hook limits; Assigns lifecycle ownership

Verification evidence A requirement-to-extension matrix and a target-system demonstration of field propagation, rule authorization, and transport behavior.

Read and verify this answer

SAP extensibility approaches and target environments

CLOUD-10 Can you expose the same RAP projection to a Fiori UI and long-lived external consumers?

I would reuse the business model where appropriate but avoid coupling an external contract to every UI change. UI consumers may need draft, feature control, presentation annotations, and navigation optimized for interaction. External consumers need stable operation semantics, versioning, paging, idempotency, and communication setup. Separate projections or service contracts can protect those needs. I would confirm which protocol and binding capabilities are supported, then test both consumers against the shared business invariants.

Example and follow-up A UI team renaming a property or narrowing visible fields could break a nightly integration if both share one uncontrolled service contract. A draft-oriented workflow may also be unsuitable for an external command that expects immediate final posting. Share policy implementation while making each consumer's lifecycle explicit.

Avoid this mistake An OData endpoint generated for a UI is not automatically a released, durable integration contract for external systems.

Interviewer probes Which behavior should be shared and which should be projected differently? / How would you introduce a breaking external-contract change?

Strong answer criteria Separates consumer contracts; Preserves shared business rules; Addresses external API lifecycle

Verification evidence Service metadata baselines, UI and integration contract tests, and documented compatibility and deprecation rules.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-11 A released CDS view contains all required fields. Why might it still be unsuitable for implementing a business command?

A read model exposes data, not necessarily the business operation that may change it. I would find the released transactional contract and its authorization, locking, validation, and save semantics. Query reuse can support decisions or presentation, but direct writes to underlying persistence bypass ownership. If no command API exists, I would revisit the process or extension design and raise a capability gap rather than infer write permission from a useful read view.

Example and follow-up Reading a purchase order status does not authorize updating its database field to simulate release. That shortcut can omit change documents, outputs, follow-on checks, and dependent state. A valid prototype must execute the supported command and observe its complete outcome, including rejection paths and concurrency behavior.

Avoid this mistake A released read API does not release its underlying tables for mutation or guarantee that its fields form a write contract.

Interviewer probes How would you handle a query result that becomes stale before the command? / What evidence distinguishes a supported command from a database shortcut?

Strong answer criteria Separates query and command semantics; Preserves provider ownership; Handles stale reads and concurrency

Verification evidence The command API's release and behavior documentation, plus concurrent-change and validation-failure tests.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-12 A large classic application cannot be replaced in one release. How would you introduce ABAP Cloud without creating two inconsistent applications?

I would identify a bounded capability that can move behind a supported contract while retaining one authoritative owner for each business state. A phased transition needs routing, compatibility, shared identifiers, and reconciliation between old and new paths. In a suitable private-edition environment, a deliberately governed classic boundary may support coexistence. I would avoid uncontrolled dual writes and define measurable retirement criteria so the temporary bridge does not become an unowned permanent platform.

Example and follow-up If the old report and new RAP app both update the same business object through different rules, users can create inconsistent states. A safer first slice may reuse the existing approved posting owner while replacing a query or interaction path. The cutover needs a reversible routing decision and verified reconciliation.

Avoid this mistake Using RAP for the new UI does not by itself modernize or validate the legacy posting transaction behind it.

Interviewer probes Which system owns writes during each transition phase? / What objective condition lets you remove the bridge?

Strong answer criteria Defines one authoritative writer; Plans coexistence and cutover; Sets retirement criteria

Verification evidence A phased ownership map, routing and rollback plan, reconciliation results, and an explicit bridge-removal milestone.

Read and verify this answer

SAP extension architecture guidance

CLOUD-13 A shared custom API is widely used but has no named owner. What would you change before accepting another dependency on it?

I would establish ownership of its semantics, compatibility, security, operations, and support window. A technically accessible object without those responsibilities is not a reliable platform contract. I would identify existing consumers, capture current behavior, and agree the release or component-access mechanism that fits its maturity. New consumers should receive documented limits and a change-notification path. If nobody can sustain the contract, I would avoid presenting it as independently reusable infrastructure.

Example and follow-up A helper that silently changes rounding can break several applications without changing its ABAP signature. Ownership must therefore cover business behavior as well as activation compatibility. Existing consumers need regression examples for amounts, units, exceptions, and transaction assumptions before the API is advertised more broadly.

Avoid this mistake Public visibility or a C1 release flag does not replace accountable ownership and semantic compatibility testing.

Interviewer probes Who approves a behavioral change without a signature change? / How do you discover consumers outside the local where-used list?

Strong answer criteria Assigns contractual ownership; Includes semantic compatibility; Inventories actual consumers

Verification evidence An API owner record, consumer inventory, support policy, and executable consumer-contract examples tied to a released baseline.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-14 A released BAdI exists, but it runs after the point where the requirement must reject a transaction. Is it still the right extension?

Not necessarily. Release status establishes a supported extension contract, but functional suitability depends on execution timing, available data, allowed changes, and error handling. I would map the requirement to the actual transaction phases and verify whether that hook can enforce the invariant. If it cannot, I would find another supported extension or revise the process. I would not hide an early validation requirement inside a late callback that can only report or trigger follow-up work.

Example and follow-up A post-save notification hook cannot reliably prevent a committed document merely by raising a message. A late reversal is a different business process with its own audit and downstream effects. Test the hook's behavior under invalid input, rollback, and update failure rather than infer it from the method name.

Avoid this mistake A released extension point can still be the wrong place for a particular business rule.

Interviewer probes What data and modifications does the BAdI contract actually permit? / Would compensating after save meet the same business requirement?

Strong answer criteria Checks hook timing and permissions; Distinguishes prevention from compensation; Validates behavior with failures

Verification evidence A transaction-phase map, the extension-point contract, and a target-system test proving when rejection or follow-up occurs.

Read and verify this answer

SAP extensibility approaches and target environments

CLOUD-15 Your domain classes depend on Gateway messages, SQL row types, and HTTP clients. How would you improve the architecture without a wholesale rewrite?

I would isolate one meaningful business decision behind a typed application contract and move transport-specific mapping to adapters. Domain logic should receive the information it needs and return business outcomes, while persistence and communication remain replaceable collaborators. I would preserve transaction ownership and existing callers during extraction. The goal is not a class per framework artifact; it is a boundary that allows policy tests and infrastructure changes without rediscovering the same business rules.

Example and follow-up An eligibility calculation can accept a stable request and policy data rather than an HTTP request object. Its result can then be translated to RAP messages or an integration response at the boundary. Characterization tests protect existing rounding and failure behavior while database and protocol dependencies are incrementally removed.

Avoid this mistake Wrapping every framework call in another class adds layers without isolating business policy if framework types still dominate the contract.

Interviewer probes Which extraction has the lowest transaction risk? / What should remain in the RAP behavior implementation?

Strong answer criteria Extracts a coherent business responsibility; Preserves external and transaction contracts; Uses observable regression tests

Verification evidence A bounded before/after dependency map and passing business-policy tests plus unchanged integration acceptance results.

Read and verify this answer

SAP extension architecture guidance

CLOUD-16 A side-by-side service needs to act for users and also run scheduled reconciliation. Would you use one technical identity for both?

I would distinguish interactive delegation from machine-to-machine processing and define the business authorization required for each. The supported communication scenario determines available services and authentication methods; arrangements and identities must be configured accordingly. A broad technical identity can hide the initiating user and over-authorize actions. I would make audit attribution, credential rotation, tenant selection, and job permissions explicit, using separate identities or scopes when the contracts and operational model require them.

Example and follow-up A nightly reconciliation may legitimately read many documents, while an interactive user should see only their authorized scope. Reusing the batch identity for UI requests can bypass that distinction. Conversely, propagating an end user's short-lived credentials into an unattended job creates a different reliability and security problem.

Avoid this mistake Authentication success proves identity, not permission for the requested business operation or tenant.

Interviewer probes Where is the initiating user recorded when a technical identity performs the command? / What happens during credential rotation or tenant offboarding?

Strong answer criteria Separates user and machine access; Preserves business authorization; Plans identity lifecycle and audit

Verification evidence A communication and authorization matrix with negative access tests, audit examples, and a credential-rotation test.

Read and verify this answer

SAP communication arrangements and scenarios

CLOUD-17 A released dependency is deprecated but still works. How do you decide when and how to replace it?

I would record the announced successor, support constraints, affected consumers, and target-release availability, then plan migration according to exposure and business criticality. Deprecated does not automatically mean already removed, but new reliance increases future work. I would compare semantics and failure behavior rather than mechanically rename calls. Compatibility tests, rollout sequencing, and monitoring should cover both the old and new paths until the transition is complete and the old dependency can be retired.

Example and follow-up A successor API may use a different key, paging model, authorization scope, or asynchronous outcome even if it serves the same business purpose. Migrating without checking those differences can create missing records or duplicate submissions. A small production-safe rollout needs explicit success measures and a supported reversal strategy.

Avoid this mistake A release contract does not mean an API never evolves, and deprecation is different from decommissioning.

Interviewer probes What if the successor is unavailable on your lowest supported release? / Which semantic differences would block a direct replacement?

Strong answer criteria Distinguishes lifecycle states; Checks successor semantics and availability; Plans measured migration

Verification evidence A dependency lifecycle register, old/new contract comparison, and regression results across the supported release matrix.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-18 What would you require before approving a senior team's ABAP Cloud architecture for production?

I would require evidence for the main business invariants and nonfunctional risks, not only a component diagram. That includes transaction ownership, authorization, release-compatible dependencies, realistic workload behavior, observability, recovery, data retention, and deployment sequencing. Each consequential assumption needs an owner and a validation result. I would permit proportionate residual risk with explicit operational controls, while distinguishing a prototype, a locally tested implementation, and a solution proven on the intended runtime.

Example and follow-up A successful Fiori demo with mocked data does not establish real RAP authorization, lock handling, or remote error recovery. A useful readiness review walks through one concurrent update, one dependency outage, one failed deployment, and one recovery drill, with evidence tied to the actual release candidate.

Avoid this mistake A clean syntax check, green ATC run, or attractive diagram is not an end-to-end production-readiness certificate.

Interviewer probes Which risk would you prove first with a technical spike? / What evidence would make you delay the release despite a successful demo?

Strong answer criteria Prioritizes consequential risks; Requires runtime-linked evidence; Defines operational ownership and recovery

Verification evidence A release-readiness dossier with targeted load, security, transaction, deployment, and recovery results for the exact candidate revision.

Read and verify this answer

SAP extension architecture guidance

Official references

SAP ABAP Cloud development model

SAP extensibility approaches and target environments

SAP extension architecture guidance

SAP released APIs, contracts, visibility, and states

SAP software component dependencies and API snapshots

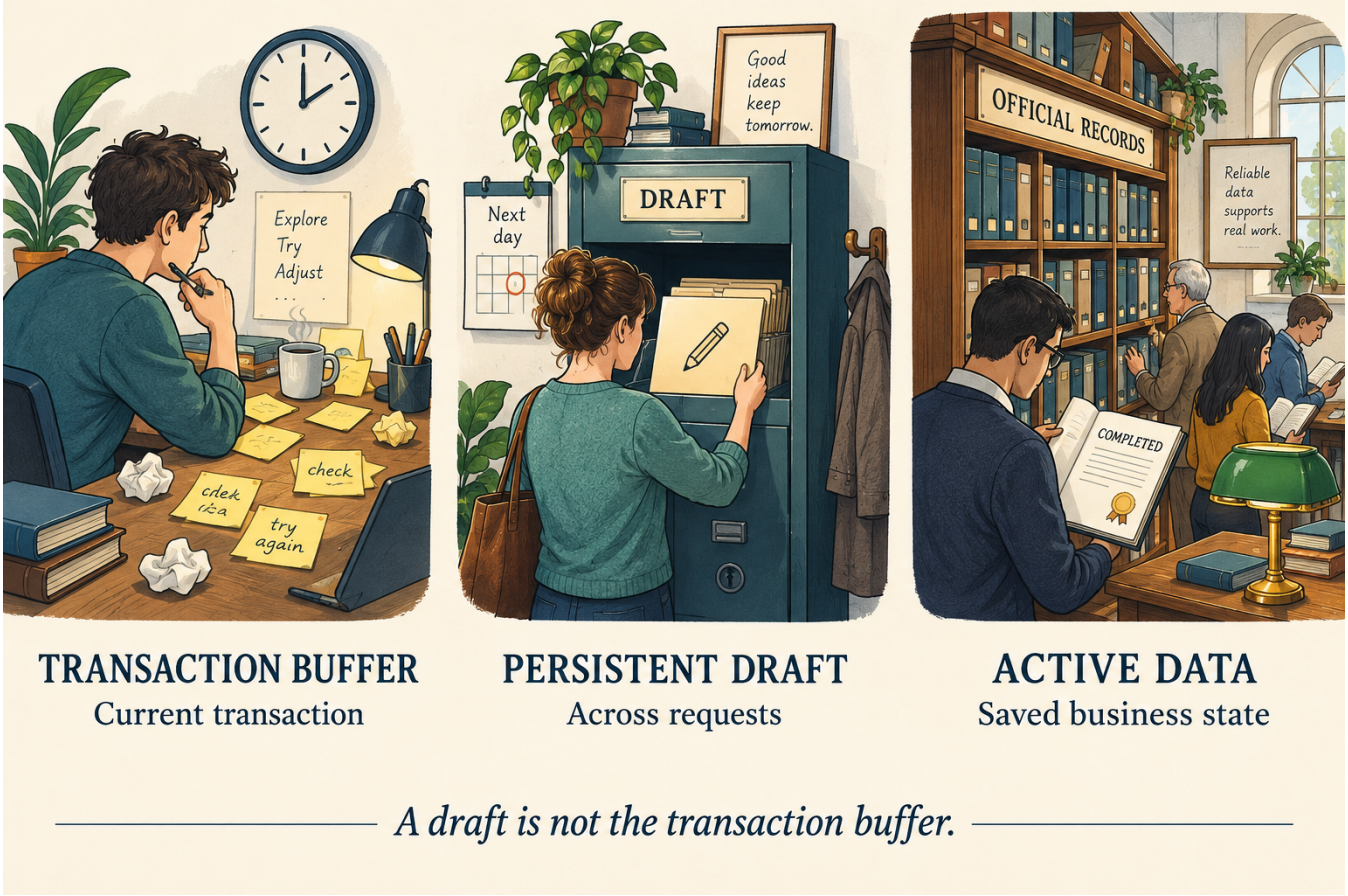
SAP software component organization and lifecycle boundaries

SAP communication arrangements and scenarios

SAP ABAP development and multitenancy guidance

RAP foundations and transactions

Start with the business-object boundary and transaction owner. Explain how the consumer changes the transactional buffer, how business rules run, and when the framework persists the result. These distinctions prevent the most serious mistakes in the original notes.



The desk represents the current transactional buffer; the draft cabinet represents persistent work across requests; the shared archive represents saved active data.

See the visual atlas for the recall prompt, limits and related questions.

RAP-01 What are the building blocks of a RAP application?

Persistence and CDS describe the data and BO composition. The behavior definition declares operations and rules; behavior implementation supplies application logic. Projections tailor the model and behavior. A service definition exposes entities, and a service binding chooses the protocol and service type. DCL and UI metadata complete relevant concerns.

Example and follow-up Trace one Submit request from Fiori through the service and projection to the BO action, transactional buffer and save sequence. Distinguish generated framework operations from business logic you implement.

Avoid this mistake Behavior is not implemented by the service definition, and a binding alone does not make the endpoint production-ready.

Read and verify this answer

SAP Learning on behavior implementation options

RAP-02 How do you choose managed versus unmanaged RAP?

Managed RAP supplies the transactional buffer and standard persistence handling for a suitable data model. Unmanaged RAP lets existing business logic supply interaction and save implementations while still obeying RAP's transaction contract. Managed with unmanaged save keeps managed interaction but delegates persistence. Choose by ownership of behavior and persistence, not application age alone.

Example and follow-up A new request app with simple owned tables is a managed candidate. A proven transaction-consistent legacy API may justify unmanaged save. Inspect the API's commit behavior before integrating it.

Avoid this mistake Unmanaged does not permit arbitrary COMMIT WORK or database changes anywhere in a handler.

Read and verify this answer

SAP Learning on behavior implementation options

RAP-03 What is the transactional buffer?

It holds the current BO state and uncommitted changes for the RAP transaction. EML operates through business behavior and can see buffered state. Direct SQL reads persistence and can therefore miss unsaved changes. Managed and unmanaged implementations differ in who implements buffer management.

Example and follow-up After changing an item quantity, a determination should calculate totals from the transaction's current state, not just a database snapshot that excludes the change.

Avoid this mistake The buffer is not a durable draft store and does not survive arbitrary stateless HTTP requests.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-04 Where may COMMIT ENTITIES be called?

An external ABAP EML consumer can use COMMIT ENTITIES to trigger the save sequence. Inside a RAP behavior implementation, the framework owns commit and rollback. Do not call COMMIT ENTITIES or COMMIT WORK in actions, validations, determinations or saver methods; violating the contract can cause a runtime error.

Example and follow-up A standalone consumer collects modification responses, checks failures and commits at the transaction boundary. An OData consumer lets the RAP request framework manage that boundary.

Avoid this mistake Moving the commit from validation into SAVE or CANCEL does not make it valid.

Read and verify this answer

SAP Learning on EML and transaction ownership

SAP RAP BO contract

RAP-05 How do you explain the RAP save sequence?

The interaction phase processes operations against the transactional buffer. The save sequence finalizes and checks the transaction, performs configured numbering, and persists changes. Detect predictable business failures during early save checks. Ordinary late-save processing assumes a consistent transaction; documented failure-capable saver variants for specific BAPI integration scenarios are an exception whose availability and contract must be verified for the target release.

Example and follow-up For a legacy API, separate validation from actual persistence and verify that the API fits RAP's save contract. Additional save is a scenario-specific hook, not a universal stage every BO implements.

Avoid this mistake Do not present a memorized list of saver methods as universally identical for managed and unmanaged BOs.

Read and verify this answer

SAP RAP BO contract

SAP BAPI consumption in RAP: late numbering and save sequence

RAP-06 Determination validation action or precheck?

A determination derives data automatically from declared triggers. A validation checks consistency and can reject saving. An action is an explicitly invoked business operation. A precheck rejects unsuitable incoming modifications before they enter the transactional buffer. Choose based on timing and responsibility.

Example and follow-up Default currency in a determination; reject an invalid date range in a validation; submit a request through an action; reject an unauthorized proposed field value in a suitable precheck. In unmanaged scenarios, BDEF determinations and validations apply to draft instances; active-instance consistency is implemented in the appropriate unmanaged handlers and save checks.

Avoid this mistake An action is not simply another name for a CRUD event, and validations should not modify business data.

Read and verify this answer

SAP Learning on numbering and determinations

SAP RAP determinations and unmanaged scope

RAP-07 How do FAILED REPORTED and MAPPED differ?

FAILED identifies instances or operations that failed. REPORTED carries messages and their context, including severity and field references. MAPPED returns identity mappings where the operation contract supports them, for example creation. A message alone does not necessarily reject the operation.

Example and follow-up For a rejected validation, populate the failed instance and a useful reported message. For a successful create, preserve the correlation between the consumer's content ID and the created identity.

Avoid this mistake Error severity in REPORTED is not a substitute for FAILED; determinations do not expose FAILED.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-08 What is EML and when do you use it instead of SQL?

Entity Manipulation Language is ABAP's typed interface to RAP business objects. READ and MODIFY ENTITIES access BO behavior and transactional state, including supported actions and associations. SQL accesses a database result and does not execute the BO's transactional contract. Use EML when consuming business operations.

Example and follow-up Updating a table directly can skip validations, authorization and lock handling. A report that invokes Submit should call the exposed BO action and process both interaction and save responses.

Avoid this mistake EML does not mean that every read is automatically the best performance choice; use the correct semantic interface.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-09 What does IN LOCAL MODE mean?

It is intended for internal consumption of a BO within its own behavior implementation. Depending on the EML operation it bypasses controls such as authorization, feature control and prechecks so internal logic can maintain its own state. It remains part of the RAP transaction and does not bypass the BO contract.

Example and follow-up A determination may update a calculated readonly field through internal EML. Keep that logic behind the public operation's checks and review cross-BO calls separately.

Avoid this mistake It is not a general-purpose option for external callers to disable security on any BO.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-10 What do %cid %cid_ref %tky and %pid mean?

%cid correlates a created instance within a request. %cid_ref refers to a related creation by that content ID. %tky is the transactional key, including the relevant identity context such as draft state. %pid is a preliminary identifier used in late-numbering scenarios. They have different lifetimes and purposes.

Example and follow-up When creating a header and items, correlate the parent creation for create-by-association. In handlers, return the supplied transactional identity rather than reconstructing only a subset of key fields. Late numbering in ADJUST_NUMBERS maps preliminary identification in %pre to the final %key.

Avoid this mistake %tky means transactional key, not temporary key; %cid is not a persisted business document number.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-11 Early numbering versus late numbering?

Early numbering supplies the final key during interaction, either by the consumer or implementation as supported. Late numbering assigns final identifiers late in save, after preliminary identity has been used. Managed UUID numbering is a common technical-key option. Business numbering and technical identity may be separate requirements.

Example and follow-up Late numbering can reduce numbers consumed by rejected transactions. A legally gap-free document sequence still needs a suitable number-range design, transaction guarantees and handling of failures and cancellations.

Avoid this mistake Late numbering alone does not guarantee a gap-free sequence, and SELECT MAX plus one is unsafe under concurrency.

Read and verify this answer

SAP Learning on numbering and determinations

SAP RAP late-numbering mapping

RAP-12 What problem does draft solve?

Draft persists an incomplete editing state across stateless requests and interruptions, separately from active business data. The framework provides a draft lifecycle with operations such as Edit, Activate, Discard and Resume; Prepare can run designated checks and determinations. Draft and active identities must be distinguished.

Example and follow-up A user can return to a partly completed request. Activation transfers valid draft changes into active processing, while the request's save sequence still completes the transaction.

Avoid this mistake Every BO is not automatically draft-enabled, and a custom Submit action is not automatically draft activation.

Read and verify this answer

SAP Learning on draft

RAP-13 Can you rely on determination and validation order?

Do not create hidden dependencies on an arbitrary order among independent determinations or validations. Model trigger conditions and explicit business dependencies; make derivations consistent and safe to repeat. Framework phase ordering exists, but that is different from a promise about a specific handler order.

Example and follow-up If total calculation depends on currency conversion, put the dependent calculation in a coherent operation or derive both from stable current data. Test repeated updates and cycles.

Avoid this mistake Calling the order random is imprecise; the important point is that an unsupported ordering assumption is unsafe.

Read and verify this answer

SAP RAP determinations and unmanaged scope

RAP-14 How does persistent field mapping work?

Behavior mapping associates CDS element names with persistence fields when names differ. It is a structural correspondence used by the selected persistence implementation. It does not transform data arbitrarily or replace semantic modeling. Projections and operation-derived types also have their own identity and naming contracts.

Example and follow-up A CDS element RequestUUID may map to request_uuid in the table. Currency conversion belongs in supported business or query logic, not in a field-name mapping declaration.

Avoid this mistake Mapping is not a database join or a universal value-conversion mechanism.

Read and verify this answer

SAP Learning on behavior implementation options

RAP-15 What should be checked before wrapping a BAPI in RAP?

Check API availability and release classification, transaction consistency, locks, validation and save separation, update-task behavior, authorization checks and any implicit or explicit commit. Then choose a compatible RAP implementation scenario and map messages and failures. A callable function module is not automatically a valid RAP save implementation.

Example and follow-up Prototype failure and rollback behavior with the real API. Verify that a later failure cannot leave the external document committed while the RAP transaction reports failure. For BAPI authorization checks that conflict with phase restrictions, use the supported authorization-context pattern together with RAP authorization handlers. Perform the actual authorization decision at the allowed operation boundary; disabling a repeated check in validation or late save must not remove the underlying protection.

Avoid this mistake A wrapper changes the interface; it does not erase the underlying API's transaction behavior or Clean Core classification.

Read and verify this answer

SAP BAPI consumption in RAP: late numbering and save sequence

SAP BAPI consumption in RAP: Authorizations

Official references

SAP Learning on behavior implementation options

SAP Learning on EML and transaction ownership

SAP Learning on numbering and determinations

SAP Learning on draft

SAP RAP BO contract

SAP RAP determinations and unmanaged scope

SAP RAP late-numbering mapping

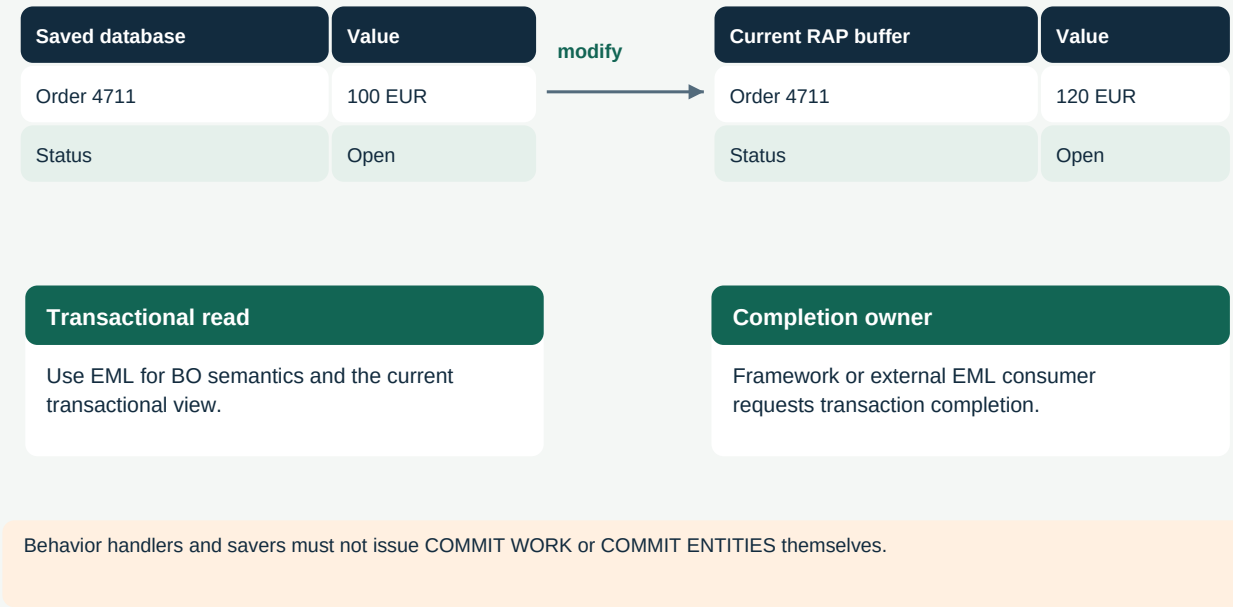
SAP BAPI consumption in RAP: late numbering and save sequence

ABAP Cloud: senior RAP transactions and behavior design

The main interview track: explain a decision, trace its failure paths, and name the evidence that would prove it. These original scenarios extend the supplied RAP notes beyond terminology. They are practice scenarios, not a measured ranking of interview frequency. The target product, ABAP release, language version, BO implementation type and consumer are intentionally unspecified: establish them before promising syntax or runtime support. Read each short answer aloud, then defend it against both follow-up probes.

A buffer is a working set, not a second commit owner

The same business object can have different in-memory and saved states



During modification, the transactional view can show 120 EUR while saved data still shows 100 EUR.

See the visual atlas for the recall prompt, limits and related questions.

SRAP-01 A new approval app has custom persistence, while its posting step must reuse an existing transactional API. Which RAP implementation type would you choose?

I would separate approval persistence from the posting boundary before selecting an implementation type. Managed RAP is my starting point for new custom tables. Managed with unmanaged save fits when RAP can own interaction buffering but an eligible API must own persistence. Fully unmanaged is justified when interaction behavior itself must adapt an existing transactional model. I would inspect the API's buffering, numbering, locks, authorizations and commit behavior; the label BAPI alone does not establish compatibility.

Example and follow-up The decision record would compare one normal request and one failing request through each boundary. If posting is remote, a local managed BO plus asynchronous orchestration may be more honest than claiming one atomic save. Additional save extends managed persistence; it is not a replacement for the persistence owner.

Avoid this mistake Choosing unmanaged merely because the application is complex, or assuming managed with unmanaged save removes RAP transaction rules.

Interviewer probes What concrete API property would rule out a synchronous RAP adapter? / Which implementation responsibilities would your choice leave with the framework?

Strong answer criteria Chooses from persistence and interaction ownership; Inspects the actual API rather than its name; Separates remote orchestration from local atomicity

Verification evidence Prepare a decision table and trace a create, failed update and rollback against the selected API on the target release.

Read and verify this answer

SAP: Save options

SRAP-02 An audit record must be saved with every managed order change. Should you use a determination, additional save, or unmanaged save?

I would clarify whether this is a transactional business audit or operational logging. For a business audit that must accompany managed persistence, additional save is a candidate: managed save remains responsible for the order while the supported save extension records the audit without committing independently. I would not replace persistence with unmanaged save just for logging. A determination derives BO state; it should not emit a durable saved-change record while the transaction can still be rejected.

Example and follow-up Audit design needs actor, operation, relevant values and a correlation key. Changed-field input may differ from a full before-and-after image; verify the save contract and any supported full-data option rather than assuming every field is supplied. Decide separately how technical failure diagnostics survive a business rollback.

Avoid this mistake Writing the audit in another LUW and claiming it is guaranteed to match the order, or updating managed order tables directly from additional save.

Interviewer probes How do you obtain a trustworthy before-image? / Should an audit failure reject the transaction, and in which phase can you detect predictable errors?

Strong answer criteria Distinguishes business audit from diagnostics; Preserves managed persistence and commit ownership; Checks available change-image semantics

Verification evidence Test successful create/update/delete and rejected save; compare order and audit persistence, including explicitly cleared fields.

Read and verify this answer

SAP: Save options

SRAP-03 A developer puts ordinary business validation in SAVE because the final data is available there. What would you change?

I would move predictable business rejection into the supported early checks, after required derivations and before the point of no return. The late save phase is designed to persist an already consistent transaction, not to provide routine interactive validation. A reported error alone is not a general rollback mechanism. I would also avoid the opposite absolute: SAP documents failure-capable saver support for specific BAPI integration scenarios, which must be verified against the actual release and saver contract.

Example and follow-up For an invalid approval limit, reject before persistence and return a keyed business message. An unexpected database or infrastructure failure is a separate technical recovery case. A supported late-failure adapter must not be used to defer checks that can reasonably run earlier or to justify independently committed side effects.

Avoid this mistake Saying either 'all errors can be returned from SAVE' or 'no supported late-save failure mechanism exists anywhere'.

Interviewer probes Where do final derivations belong relative to final consistency checks? / What evidence would justify using a failure-capable saver here?

Strong answer criteria Places predictable rejection before late save; Separates business and technical failures; Qualifies documented BAPI exceptions precisely

Verification evidence Trace the save phases for invalid input and inspect the generated saver signature and SAP guidance for the installed release.

Read and verify this answer

SAP: General RAP BO implementation contract

SAP: BAPI consumption, late numbering and save sequence

SRAP-04 One action updates two local RAP BOs and then calls a remote payment service. Can the whole action be atomic?

Two compatible local RAP BOs can participate in the same RAP-controlled SAP LUW, with the transaction owner controlling the final save. An ordinary remote HTTP call is outside that local atomic boundary. If payment succeeds and local save fails, rollback cannot undo the remote effect. I would model a durable pending state and reliable asynchronous delivery, or an explicit compensating process, and define retry, reconciliation and customer-visible status rather than promise distributed atomicity.

Example and follow-up For two local BOs, verify cross-BO phase restrictions and consume supported behavior interfaces; do not commit each BO separately. For payment, store a stable business correlation key and record the remote outcome. A timeout is an unknown result until reconciliation establishes whether the remote operation actually completed.

Avoid this mistake Assuming that calling HTTP inside an action enrolls the remote system in the SAP database transaction.

Interviewer probes What happens after a payment timeout followed by a retry? / How would you recover after local commit succeeds but event delivery is delayed?

Strong answer criteria Draws the local transaction boundary; Models uncertain remote outcomes; Designs idempotency and reconciliation

Verification evidence Fault-inject before and after each local save and remote response boundary; verify that every persisted state has a documented recovery path.

Read and verify this answer

SAP: General RAP BO implementation contract

SRAP-05 A bulk EML update contains 200 instances, including missing keys and unauthorized changes. How do you report and process the result?

I would process the full input set using the operation's derived types and preserve instance correlation. FAILED identifies rejected operations or instances; REPORTED carries associated messages and can also contain non-error information. MAPPED serves creation or key-correlation contracts, not a universal success list. I would collect responses from nested calls without losing earlier failures, avoid returning success for unprocessed rows, and distinguish interaction results from the subsequent save outcome and the consumer's atomicity policy.

Example and follow-up I would bulk-read required transaction state and validate the applicable set instead of executing a database query per row. Mixed failures require a deliberate consumer decision: reject the logical request or proceed with eligible work where the contract permits. Count reconciliation must include missing, rejected and successfully handled inputs.

Avoid this mistake Checking only the first input row, treating every REPORTED entry as failure, or declaring the operation committed after MODIFY ENTITIES.

Interviewer probes How would you associate a child error with its root request? / What do you check again after the external consumer issues COMMIT ENTITIES?

Strong answer criteria Handles all inputs and aggregates responses; Separates response roles and transaction phases; Avoids row-by-row dependency access

Verification evidence Run a mixed-input EML test with multiple distinct failures; assert key/message correlation, processed counts and final database state.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-06 An API consumer sends an empty comment to clear it, but the previous text remains. What do you investigate?

I would distinguish an explicitly supplied initial value from an omitted field. EML update intent is conveyed by the field list or the corresponding %control component; an initial value without update intent may correctly leave existing data unchanged. I would trace the incoming payload, protocol-to-EML conversion, adapter mapping and any determination that restores a default. Fixing this by marking every field as supplied risks clearing unrelated fields that the caller intentionally omitted.

Example and follow-up A partial update with only Comment should preserve Amount and Currency. A legacy adapter may additionally require its own change indicators, mapped from RAP control information. The same distinction matters for zero, false and blank dates according to the field's domain and business rules, not just character strings.

Avoid this mistake Using IS INITIAL to mean 'not provided' for every update field, or blindly setting all control flags on.

Interviewer probes How would you represent 'leave unchanged' versus 'set to zero'? / Where can a defaulting determination accidentally undo the requested clear?

Strong answer criteria Separates presence from value; Preserves partial-update semantics; Traces control mapping through adapters

Verification evidence Test omitted Comment, explicit empty Comment and nonempty Comment while asserting that all untouched fields remain unchanged.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-07 You must create three new orders with items in one EML request before final order numbers exist. How do you correlate the tree?

I would give creates unique request content IDs and connect create-by-association input to its newly created parent using the supported %cid_ref correlation. Each child create also needs its own appropriate identity and response mapping. I would use the BDEF-derived types, retain draft and preliminary-key context, and read MAPPED by identifiers rather than by table position. Content IDs are request correlation values; they are not persisted business keys or reusable retry tokens.

Example and follow-up Consider one invalid item under the second order while the first and third are valid. The error must identify the right child and parent regardless of input ordering. Later key assignment must preserve the original relationship, and the external consumer must decide the desired all-or-nothing behavior at its transaction boundary.

Avoid this mistake Assigning a fake database key to a new parent, correlating by row index, or confusing %cid with %pid.

Interviewer probes What changes if the BO uses draft and late numbering? / How do you correlate an error when the final parent key is still unavailable?

Strong answer criteria Uses content-ID correlation for deep create; Keeps transactional identity intact; Tests mixed parent/child failures

Verification evidence Execute a reordered multi-parent deep-create case with one failed child and verify associations, mappings and error paths before and after save.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-08 A posting API returns the final document number only when called. How would you integrate numbering without publishing incorrect keys?

I would assess the documented late-numbering integration pattern and maintain preliminary identities until the save sequence assigns final keys. The adjustment response must correlate the preliminary identity in %pre with the final %key; consumers must not assume an earlier key is final. SAP's BAPI guidance places a numbering create call in ADJUST_NUMBERS and describes a failure-capable saver for that scenario. I would verify availability, API release status and failure semantics before choosing that design.

Example and follow-up Number allocation and external publication are different responsibilities. Do not send an email or remote event containing a supposedly committed number during an earlier phase. Late numbering does not by itself guarantee gap-free legal numbering; number-range configuration, rollback behavior and the posting API's own contract determine that requirement. For the documented failure-capable late-save scenario, if late save fails, COMMIT ENTITIES returns sy-subrc 8; an external EML consumer must issue ROLLBACK ENTITIES before continuing. Verify target-release support; this does not authorize a commit or rollback inside the behavior implementation.

Avoid this mistake Using SELECT MAX plus one, dropping preliminary correlation, or assuming ordinary ADJUST_NUMBERS accepts recoverable business failures without a supported special contract.

Interviewer probes How does the external EML consumer obtain the final key? / What must be demonstrated if the business insists on gap-free numbering?

Strong answer criteria Separates preliminary and final identities; Qualifies the BAPI saver exception; Does not promise gap-free numbers from RAP alone

Verification evidence Create several documents concurrently, include a failing API call, and verify final key conversion, associations and number-allocation behavior.

Read and verify this answer

SAP: BAPI consumption, late numbering and save sequence

SRAP-09 A user saves a draft, closes the browser, returns, and presses Activate. What state transitions and failures would you explain?

A persisted draft survives the individual request but is not yet the active business object. For a new draft, activation leads to active creation; for an edit draft, it applies changes to the active instance through the applicable runtime contract. Prepare runs the assigned consistency work, while activation and final save still need to succeed. Discard abandons the draft; discarding an edit draft must not delete its existing active counterpart. Business Submit is a separate modeled action.

Example and follow-up I would clarify which UI operation the interviewer calls 'save': a draft PATCH, draft activation and business submission can have different meanings. A failed validation should retain a correctable editing experience. Draft persistence and message behavior must be tested through the intended consumer, including a fresh browser session.

Avoid this mistake Treating draft persistence as a final business commit or implementing Submit by directly updating draft tables.

Interviewer probes Which checks must still protect an API that writes active instances directly? / What remains after discarding an edit draft versus a new draft?

Strong answer criteria Distinguishes draft, activation and business status; Explains both new and edit drafts; Preserves a recoverable validation experience

Verification evidence Test new/edit draft creation, browser reopen, failed Prepare, successful activation and both discard variants; inspect active and draft results.

Read and verify this answer

SAP: Draft runtime

SRAP-10 A draft lock expires, another process changes an active child, and the first user resumes editing. How do you prevent silent loss of the child change?

I would check the composition's locking model and total-ETag maintenance across all relevant active changes, including children. After the exclusive draft lock expires, resume must detect whether active BO state changed using the total ETag before editing continues under renewed protection. An entity ETag used for a normal request is a related but different guard. If a child-changing path does not advance the required aggregate version, the draft conflict check can miss the change.

Example and follow-up In an unmanaged implementation, version and lock responsibilities require explicit implementation evidence. Direct writes to persistence can bypass the intended maintenance entirely. The desired outcome for stale resume is an understandable conflict and recovery path, not silently copying the old draft over newly changed active data.

Avoid this mistake Checking only the root's own business fields or assuming an expired lock means the stored draft is automatically current.

Interviewer probes Which timestamp should change after a child-only modification? / How do you distinguish stale If-Match handling from draft resume protection?

Strong answer criteria Separates ordinary and total ETags; Includes child and alternative update paths; Defines a stale-draft recovery experience

Verification evidence Use two sessions, expire the draft lock, change only an active child, then resume and verify the conflict and unchanged active data.

Read and verify this answer

SAP RAP total ETag

SRAP-11 Two determinations calculate discount and total. Production results depend on which one executes first. How would you redesign them?

I would remove the dependency on unspecified execution order and repeated invocation. Derive the result from authoritative transaction state through a coherent calculation routine, with triggers covering every relevant input. Repeating the same calculation under unchanged conditions must not accumulate discounts or create extra children. If determinations update fields that trigger other rules, I would make those dependencies convergent and avoid cycles. For fully unmanaged active behavior, I would first verify that the proposed determination mechanism is supported.

Example and follow-up Changing Quantity and Discount together should produce the same total as changing them in separate requests. A rule that subtracts a discount from the already discounted amount fails this test. I would calculate from base values, update only required derived values and keep external communication out of the derivation.

Avoid this mistake Depending on declaration order, using a session flag to run once, or sending a notification each time the determination runs.

Interviewer probes What happens when activation reuses already executed consistency work? / How would you detect and prevent a determination-trigger cycle?

Strong answer criteria Requires idempotent calculations; Avoids order assumptions; Covers trigger dependencies and implementation restrictions

Verification evidence Test combined and separate field changes, repeated evaluation, draft activation and create-then-delete in one transaction; compare final values.

Read and verify this answer

SAP: Determinations

SRAP-12 A request changes CompanyCode to an unauthorized value and also makes its dates inconsistent. Would you use prechecks or validations?

I would separate the incoming-change decision from final-state consistency. A supported precheck can inspect requested values before they enter the transactional buffer, for example to reject a disallowed target context. A validation protects the resulting business state at its defined trigger time, such as the valid date interval. Neither replaces the complete RAP authorization model. I would merge partial input with relevant existing state when necessary and avoid assuming that every field is present in the request.

Example and follow-up The original and target company may require different authorization considerations. The date rule should still hold for another consumer that reaches the same supported operation path. For unmanaged active instances, use the appropriate handler or save consistency mechanism instead of assuming BDEF draft validations protect active persistence automatically.

Avoid this mistake Treating prechecks as a global security bypass or believing a UI mandatory marker enforces all backend invariants.

Interviewer probes Which state does the precheck see before the update is applied? / How would the rule behave for a partial update that omits both dates?

Strong answer criteria Separates incoming intent from final state; Keeps authorization explicit; Accounts for partial updates and unmanaged restrictions

Verification evidence Verify that a denied target context is rejected by the precheck before entering the transactional buffer. Separately verify that inconsistent dates are rejected by validation and cannot be persisted; test correction and successful retry through EML and the service.

Read and verify this answer

SAP RAP operation prechecks

SRAP-13 A CopyOrder action creates an order, but the UI stays on the source and the consumer cannot identify the copy. How do you review the action contract?

I would inspect whether the operation should be an instance factory action and whether its creation mapping preserves the caller's content ID. A normal action returning $self describes a different result contract. For an ordinary action with a declared result, the implementation must populate the correct result structure and cardinality. For a factory action, creation identification uses MAPPED rather than an ordinary result parameter. I would check supported cardinalities and actual UI navigation behavior on the target release.

Example and follow-up Copying also needs a field policy: generate new identity, reset approval state, omit audit ownership and decide which children are copied. Do not infer business copy semantics from a whole-structure assignment. A correctly returned key cannot compensate for copied status or authorization data that should never have been reused.

Avoid this mistake Assuming all factory-action cardinalities are identical on every release or solving wrong result mapping with a hard-coded UI redirect.

Interviewer probes What changes if one source must create several target objects? / Which source fields and children must never be copied automatically?

Strong answer criteria Chooses factory versus ordinary action deliberately; Preserves request-to-created-instance mapping; Defines copy semantics and release-dependent consumer behavior

Verification evidence Call the action through EML and the UI with multiple inputs; verify mappings, new identities, reset fields and navigation to the correct instance.

Read and verify this answer

SAP: Action definition

SRAP-14 A projection accepts a convenience field that must enrich an underlying create. Why might augmentation fit, and what are its boundaries?

Augmentation can translate supported projection-specific modify input into compatible underlying BO operations. I would preserve the original request's supplied values and relate any generated input to its originating request using the supported correlation mechanism. It is not an unrestricted hook for overwriting caller intent or an additional-save implementation. I would inspect permitted operations, control information and response behavior in the target release, because an augmenting EML call does not behave like an arbitrary synchronous modify call.

Example and follow-up For a requested display-language text, the projection may add a corresponding text creation while the base BO remains reusable. The main risks are conflicting supplied fields, duplicate content IDs and an error that loses its original request context. Persistent invariants must remain protected in the underlying BO for other consumers.

Avoid this mistake Using augmentation to bypass base validations or expecting an unrestricted local result from an augmenting operation.

Interviewer probes What if the caller already supplied the field your augmentation wants to set? / How is a generated child operation related back to the original request?

Strong answer criteria Keeps augmentation at the projection input boundary; Preserves supplied-field and correlation semantics; Retains reusable base invariants

Verification evidence Test absent/present convenience fields, conflicting underlying input, multi-instance requests and generated-operation failures with the supported augmentation API.

Read and verify this answer

SAP: Operation augmentation

SRAP-15 A legacy function module issues COMMIT WORK internally. Can you make it safe by wrapping it in a RAP action?

A wrapper does not remove the commit or transfer transaction ownership. I would inspect the full call chain and seek a released API with compatible buffering and save behavior, or separate the operation into an explicitly asynchronous process with its own transaction. Moving the same committing call to a saver is not sufficient. If a supported classic API integration is possible, its specific release, language-version allowance, authorization and save-phase rules must be established before implementation.

Example and follow-up The damaging case is a local order change followed by an internal legacy commit and then a later validation failure. The business sees a rejected request, yet some effects may already be durable. RFC indirection does not restore atomicity; it creates another boundary that needs state, retry and reconciliation design.

Avoid this mistake Hiding the commit in a wrapper, disabling runtime checks, or calling a separate task while continuing to promise one LUW.

Interviewer probes How would you discover an implicit commit several calls below the adapter? / What business states would an asynchronous replacement expose?

Strong answer criteria Preserves RAP transaction ownership; Audits nested and implicit transaction effects; Offers a viable alternative integration boundary

Verification evidence Review the dependency call graph and trace successful and rejected requests; document the first durable effect and the actual transaction owner.

Read and verify this answer

SAP: General RAP BO implementation contract

SAP: BAPI consumption, late numbering and save sequence

SRAP-16 An older RAP BO works without strict mode. The team wants to release it for cloud consumption by adding strict(2). Is that sufficient?

No. Strict mode introduces modeling and implementation requirements; enabling it can expose previously implicit behavior and incompatible derived-type usage. I would assess base and projection definitions together, make supported behavior explicit and review locks, authorizations, ETags and composition associations. API release also needs an intentional stable consumption contract and eligible dependencies. A successful syntax check is necessary evidence, but it does not demonstrate consumer compatibility, secure operation or a released API lifecycle.

Example and follow-up I would capture existing service metadata and consumer tests before changing the model. The target product may support a different feature set from current cross-product documentation. A release plan should identify allowed future changes and the separate consumption or extension contracts, with regression cases for existing EML and OData clients.

Avoid this mistake Treating strict(2) as a clean-core grade or API-release switch, or enabling it only on one layer and ignoring resulting contract changes.

Interviewer probes Which implicit behavior could disappear when strict mode is enabled? / What would you freeze in a released BO interface rather than in the base implementation?

Strong answer criteria Separates strict checks from API release; Reviews all affected layers; Plans compatibility and consumer regression evidence

Verification evidence Record before/after metadata, activate the full dependency set, run ATC and execute representative old consumer calls on the target release.

Read and verify this answer

SAP: Strict mode implementation requirements

SRAP-17 An unmanaged BO passes isolated tests but a second request reads leftovers after an earlier save rejection. Where do you look?

I would inspect the lifetime of its custom transactional buffer and cleanup paths, including rejected early save and explicit rollback by an external transaction owner. Reads must combine persistent data with the current transaction's changes, not stale static state. CLEANUP and CLEANUP_FINALIZE have different lifecycle purposes; early-save cleanup may need to undo finalize-derived work while preserving the correct state for continued interaction. I would define these semantics explicitly instead of clearing every structure indiscriminately.

Example and follow-up A static internal table is not automatically a valid cache just because RAP calls it from one behavior pool. Reuse of an internal session can reveal stale entries or counters. Test a failed transaction followed by a clean one and verify creates, updates, deletes, numbering maps and temporary dependency data independently.

Avoid this mistake Implementing cleanup only after success or deleting all user changes after an early-save rejection without understanding the retry contract.

Interviewer probes Which data was created during FINALIZE rather than during user interaction? / How would an in-transaction READ differ from a direct SELECT after an unsaved update?

Strong answer criteria Scopes state to a transaction; Distinguishes lifecycle cleanup responsibilities; Preserves correct read-your-writes and retry behavior

Verification evidence Run failure-then-success, rollback-then-read and repeated-save sequences within a reused session; assert no stale buffer entries or leaked temporary identifiers.

Read and verify this answer

SAP: Cleanup after early save failure

SRAP-18 An Approved event should trigger an external workflow. Why is calling its HTTP endpoint from an on-save determination unreliable?

The determination can run before the business transaction is durable and may be repeated. A remote HTTP effect cannot be rolled back with the local RAP buffer. I would use the supported RAP business-event and publication infrastructure, or another explicitly designed transactional delivery mechanism, and define the event as a fact about a committed transition. Broker delivery, consumer processing and business completion remain separate stages; consumers need idempotency and operational recovery rather than an assumption of exactly-once execution.

Example and follow-up The event payload should contain a stable identity and enough version or transition context to reject stale work. A retry of the same event must not start a second approval workflow. Monitoring should connect producer transaction, event publication, delivery attempts and the consumer's recorded processing outcome without exposing unnecessary business data.

Avoid this mistake Equating a successful HTTP response or a raised in-process event with durable, exactly-once end-to-end workflow execution.

Interviewer probes How would you recover an event that is delivered after the order is cancelled? / What identifier prevents duplicate workflow creation?

Strong answer criteria Separates transaction commit from remote effects; Designs idempotent and version-aware consumers; Names publication and processing observability

Verification evidence Test rejected save, publisher outage, duplicate delivery and stale delivery; verify no workflow for rejected data and one effective workflow per transition.

Read and verify this answer

SAP: Business events

SRAP-19 A credit-limit check runs successfully on a draft, but active data is later saved through another consumer with an invalid amount. What is missing?

The invariant must be enforced at every supported write path, not only in a convenient draft interaction. I would inspect validation triggers, Prepare assignment, active-instance processing and the implementation type. A managed validation can protect the final state through its supported save behavior; fully unmanaged active persistence needs its appropriate consistency implementation. I would also determine whether the check depends on concurrently changing credit usage and therefore requires a concurrency strategy beyond a one-time remote or database read.

Example and follow-up A successful earlier check is evidence about that earlier state, not a reservation of available credit. Two orders can each pass against the same balance and exceed it together. The design may need an authoritative reservation operation, suitable locking or a business process accepting deferred rejection, depending on the actual credit service.

Avoid this mistake Duplicating the check only in the UI or assuming validation alone eliminates a cross-transaction race.

Interviewer probes What prevents two parallel orders from consuming the same available credit? / Which direct active-instance consumer tests would you add?

Strong answer criteria Protects the invariant across consumers; Accounts for unmanaged active behavior; Recognizes time-of-check versus time-of-use risk

Verification evidence Exercise draft activation and direct active writes, then run two competing transactions whose combined amount exceeds the limit.

Read and verify this answer

SAP: Validations

SRAP-20 In a mass approval request, one instance fails. Must all other approvals be rolled back?

I would first identify the business requirement and exact transaction boundary: an EML interaction, the surrounding SAP LUW, an OData changeset or atomicity group, and an entire batch are different scopes. Per-instance failures during interaction do not by themselves define the final policy for all other inputs. The transaction owner must inspect responses and apply the required commit or rollback behavior. Backend action design must preserve invariants and report every affected instance consistently.

Example and follow-up If the business requires one approval package to succeed together, model and test that unit explicitly. If independent approvals may succeed separately, partition requests intentionally and expose per-item outcomes. Do not infer persistence from the HTTP envelope or from a successful response for an operation before the relevant save boundary.

Avoid this mistake Claiming that any FAILED entry universally rolls back every request, or that every OData batch is one atomic transaction.

Interviewer probes How does your answer change for separate atomicity groups? / What must an external EML consumer do after a partial interaction failure?

Strong answer criteria Names the actual atomic unit; Separates interaction responses from save outcome; Aligns transaction policy with business requirements

Verification evidence Test a mixed-success action via EML and the actual OData request shape; inspect persisted state after both intended commit and rollback paths.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-21 After a quantity change, the backend total is correct but the Fiori page displays the old value. Is this a transaction bug?

I would separate calculation correctness from client refresh. First read the relevant transactional state through the correct draft or active context and verify the determination. Then inspect action results, side-effect declarations, service metadata and actual client requests. A supported side effect tells the consumer which dependent values or permissions to refresh; it does not calculate the total or create a commit boundary. The answer may be a UI refresh contract defect rather than incorrect persistence.

Example and follow-up I would reproduce without the UI through EML or the service, then compare the browser's network trace. Refreshing the entire page can conceal a missing dependency but adds cost and may lose editing context. Child changes, totals and action availability often need a deliberately scoped refresh rather than a global reread.

Avoid this mistake Adding COMMIT WORK to make the UI update or treating a side-effect declaration as a substitute for a backend determination.

Interviewer probes What if the total is correct only after activation? / Which refresh targets and requests would you expect after an item quantity change?

Strong answer criteria Diagnoses backend state and client state separately; Preserves transaction ownership; Validates metadata against real consumer behavior

Verification evidence Compare EML/service values with browser requests before and after the change; verify draft context, refreshed totals and request volume.

Read and verify this answer

SAP RAP side effects

SRAP-22 Your team finds a useful SAP RAP base BO in ADT, but only a narrower BO interface is released. Which layer should your application consume?

I would consume the released interface with the required visibility and contract, and design within the operations it exposes. Source visibility or a callable-looking base object is not a supported consumption promise. The BO interface provides a stable boundary while the provider retains freedom to change internals. If an operation is missing, I would look for another released API, request the capability or redesign the process rather than bypass the interface with table updates or unreleased behavior access.

Example and follow-up A BO interface is not an arbitrary OO interface with a custom handler: it exposes a supported subset of the underlying RAP BO. I would check whether draft operations, fields, associations and actions needed by this consumer are included, as well as authorization and version-dependent restrictions on cross-component use.

Avoid this mistake Calling the base BO because it activates today, or equating C1 consumption permission with permission to extend every underlying artifact.

Interviewer probes What does the interface protect when the provider changes its persistence model? / How would you handle a required operation absent from the released interface?

Strong answer criteria Uses the released consumption boundary; Separates consumption and extension contracts; Checks capability gaps before implementation

Verification evidence Record API state and visibility in the target system; compile and test the consumer using only exposed fields, associations and operations.

Read and verify this answer

SAP: Business object interface

SRAP-23 Two requests execute Submit for the same order, and one client retries after a timeout. How do you prevent duplicate business effects?

I would define Submit as an explicit state transition with a concurrency policy and a durable notion of the business operation. Locking and version checks prevent conflicting updates, but a later retry still needs a defined outcome when the order is already submitted. I would either return the original effective result or reject the repeated transition consistently, and use stable idempotency correlation for downstream work. A disabled UI button is neither concurrency control nor duplicate-effect protection.

Example and follow-up Checking status before acquiring suitable protection leaves a race: both requests can observe Open. The transition and its durable follow-up intent must be coordinated. If a caller supplies an idempotency key, scope it by operation and relevant business context, detect incompatible payload reuse, and define retention and lookup behavior.

Avoid this mistake Relying on a singleton flag, browser debounce, or request-local %cid as a durable cross-request idempotency key.

Interviewer probes What response should the retry receive if the first call committed but its response was lost? / How do you handle the same idempotency key with different parameters?

Strong answer criteria Coordinates state transition and concurrency; Defines retry outcomes explicitly; Prevents duplicate downstream effects durably

Verification evidence Run simultaneous Submit calls and replay after a simulated lost response; verify one effective transition and one intended downstream operation.

Read and verify this answer

SAP: General RAP BO implementation contract

SAP: Business events

SRAP-24 A validation reports the right error once, but the message remains after correction or appears on the wrong child. How would you debug it?

I would inspect instance identity, field markers, parent paths and state-area handling in REPORTED, plus the corresponding FAILED entry where rejection is required. State messages represent current invalid state and need correct invalidation and revalidation; transition messages describe a particular operation. I would trace which validation triggers fire after correction and whether the UI is reading the right draft instance. An accurate message text cannot compensate for a wrong transactional key or stale state area.

Example and follow-up For an item delivery-date error, revalidate the affected item and clear the prior state appropriately before reporting the current result. Include a case where the item is deleted or moved out of the failing condition. Avoid blanket deletion of all messages, which can remove unrelated errors owned by other checks.

Avoid this mistake Appending error text on every call without resetting its state area, or assuming REPORTED alone rejects a save.

Interviewer probes Which message should survive a round trip and which should not? / How would you preserve an unrelated validation's error while clearing this one?

Strong answer criteria Correlates messages to transactional identity and fields; Manages state areas without erasing unrelated state; Separates messaging from operation rejection

Verification evidence Test invalid-to-valid-to-invalid edits on two children, child deletion and activation; assert message paths, stale-message removal and correct save rejection.

Read and verify this answer

SAP RAP state messages

Official references

SAP: General RAP BO implementation contract

SAP: Save options

SAP: BAPI consumption, late numbering and save sequence

SAP: Entity Manipulation Language

SAP: Draft runtime

SAP: Determinations

SAP: Action definition

SAP: Operation augmentation

SAP: Strict mode implementation requirements

SAP: Business object interface

SAP: Business events

SAP: Cleanup after early save failure

SAP RAP total ETag

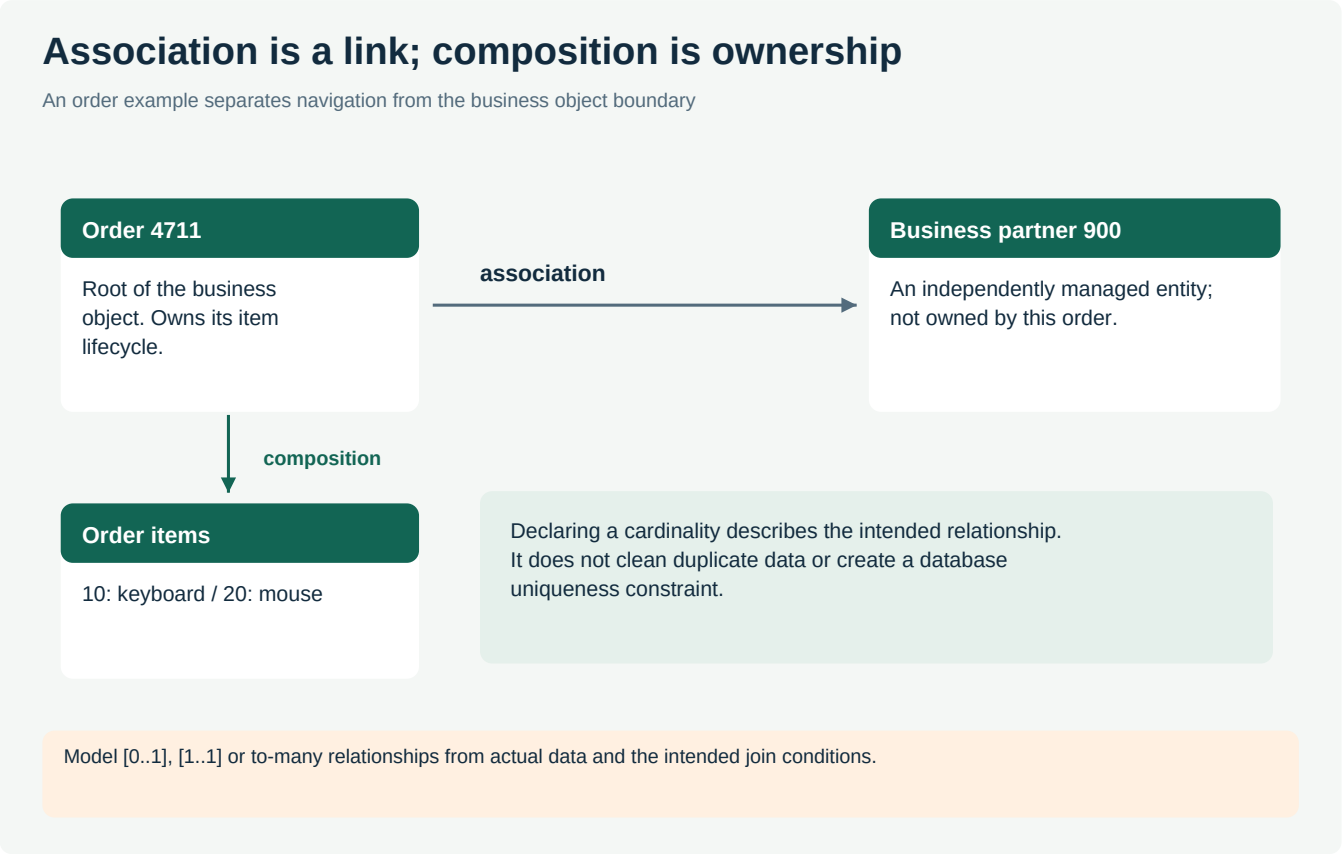
SAP RAP operation prechecks

SAP RAP side effects

SAP RAP state messages

CDS data modeling

Explain the data model before discussing annotations. A strong answer connects entity identity, association cardinality, authorization and the intended consumer. Distinguish a reusable CDS model from a transactional RAP projection and from an analytical query.



Items belong to the order BO; a partner reference crosses to an independently managed object.

See the visual atlas for the recall prompt, limits and related questions.

CDS-01 What does ABAP CDS add to a SQL query?

CDS defines reusable, semantically enriched data models in the ABAP repository. A model can describe elements, associations, annotations and access control, and can serve different consumers. SQL computes the result; CDS also supplies a stable model contract. Transactional behavior is added separately through RAP.

Example and follow-up For a sales-order list, expose keys, customer associations, currency semantics and authorized rows once. Several consumers can use that model without each recreating the joins.

Avoid this mistake A CDS definition does not automatically implement create, update and delete operations.

Read and verify this answer

SAP Learning on CDS concepts

CDS-02 What is the difference between a classic CDS view and a CDS view entity?

A classic define view creates a CDS entity plus a separately named DDIC SQL view using AbapCatalog.sqlViewName. A define view entity avoids that separate DDIC SQL-view artifact and is the preferred choice for new supported scenarios. Choose according to the target release and consumer compatibility.

Example and follow-up During migration, inventory consumers that refer directly to the generated SQL-view name. Changing the source definition is not sufficient if those consumers still depend on the old object.

Avoid this mistake Do not copy AbapCatalog.sqlViewName into a view entity or assume all older systems support view entities.

Read and verify this answer

SAP Learning on CDS concepts

CDS-03 Association or join and how do you select cardinality?

An association models a reusable relationship; a path expression can cause a join when the relationship is consumed. A join explicitly combines sources in the current query. Cardinality must describe the actual data relationship and ON condition. Wrong to-one assumptions can produce incorrect or optimizer-dependent results.

Example and follow-up For a flight-to-carrier association, an ON condition matching the carrier key can justify a to-one target cardinality if the data satisfies it. The flight's own identity is a separate question: it may require carrier, connection and flight date. Adding a date to the flight key does not establish the carrier relationship's cardinality. Test duplicate and missing target rows.

Avoid this mistake Associations are not automatically faster than joins and do not enforce referential integrity.

Read and verify this answer

SAP Learning on CDS concepts

CDS-04 Association versus composition in RAP?

An association represents a relationship. A composition describes ownership within a RAP business-object tree, such as order and items. The root defines the aggregate boundary, with dependent locking and authorization where modeled. Exposed projections redirect associations and compositions to the corresponding projected entities.

Example and follow-up An item cannot be treated as an unrelated aggregate just because it has its own table. Explain create-by-association and what happens when the parent is deleted.

Avoid this mistake Not every foreign key should become a composition; lifecycle ownership is the deciding factor.

Read and verify this answer

SAP Learning on compositions

CDS-05 What do interface composite consumption and projection mean?

Interface and consumption names describe modeling roles, while VDM basic, composite and consumption annotations describe a virtual data model convention. A RAP projection adapts one underlying entity for a use case under its provider contract. These labels are not interchangeable and do not themselves publish a service.

Example and follow-up Keep reusable business semantics below the consumer projection. Expose only needed fields and operations, and redirect associations consistently. Avoid adding layers whose only purpose is a naming pattern.

Avoid this mistake A VDM composite annotation is neither a prerequisite for service binding nor permission to perform arbitrary joins in a RAP projection.

Read and verify this answer

SAP Learning on CDS concepts

CDS-06 What are CDS abstract and custom entities?

An abstract entity describes a structured type without a database result, often for action parameters, results or events. A custom entity represents a query model whose data is supplied by an ABAP implementation instead of a generated SQL query. Neither is an object-oriented abstract superclass.

Example and follow-up A Submit action may accept an abstract entity containing a comment. A custom query provider may assemble data from a remote service and must handle requested filters, paging and count.

Avoid this mistake Do not describe an abstract entity as a BO that other CDS entities inherit and persist.

Read and verify this answer

SAP CDS abstract entities

CDS-07 Does DCL on a base view automatically protect every higher view?

No. Read authorization must be designed for each externally accessed CDS entity. A higher view can use its own access control with inherited conditions where supported. Merely adding AccessControl.authorizationCheck: #CHECK does not invent an access rule. RAP operation authorization is a separate concern.

Example and follow-up Test the projection used by the actual service with users from different company codes. Inspect the effective DCL and inheritance mapping, not just a protected lower-layer view.

Avoid this mistake DCL on one layer does not automatically secure an arbitrary view-on-view stack.

Read and verify this answer

SAP Learning on DCL inheritance

CDS-08 Where do UI annotations and value helps belong?

Use metadata extensions for supported UI and consumption annotations when the model permits extensions. Keep core data semantics, such as amount and currency relationships, in the data model. Check annotation eligibility, layer precedence and release support rather than moving every annotation mechanically.

Example and follow-up A Fiori elements list can use line-item annotations and a currency-aware amount. A value help supports selection, but the backend must still validate submitted values and authorizations.

Avoid this mistake UI.hidden and a value help are not security or data-integrity controls.

Read and verify this answer

SAP Learning on CDS concepts

CDS-09 What is a virtual element and what are its limits?

A virtual element is calculated in the application layer for a supported query scenario, commonly through a SADL calculation exit. It does not become a database column. Filtering, sorting and requesting the element need explicit supported handling and may differ by service and release.

Example and follow-up A display-only status text derived from an external source may fit. A field needed for efficient database filtering usually belongs in SQL/CDS or persistence instead. Fetch only needed calculation inputs.

Avoid this mistake requestAtLeast is a UI request hint, not a universal guarantee that every consumer calculates a virtual element.

Read and verify this answer

SAP RAP: implementing sorting of virtual elements

CDS-10 How do analytical CDS models differ from transactional models?

Analytical models organize measures and dimensions, often around a cube and an analytical query or analytical projection. Transactional RAP models focus on identifiable instances and business operations. Aggregations must respect units, currencies and the query engine. One model is not automatically suitable for both purposes.

Example and follow-up Never sum EUR and USD amounts without the intended conversion or grouping. Confirm which analytical features the chosen protocol and consuming application actually execute.

Avoid this mistake A VDM annotation or an OData $apply request alone does not guarantee correct analytical semantics.

Read and verify this answer

SAP analytical projection views

CDS-11 How do CDS keys and database keys relate?

A CDS key communicates entity identity to consumers but does not automatically enforce uniqueness in the query result. Database keys enforce uniqueness in persistence. Joins, missing key elements and aggregations can change result uniqueness. RAP and OData need stable, truly unique entity identities.

Example and follow-up If SFLIGHT rows differ by CARRID, CONNID and FLDATE, exposing only the first two as identity can merge or confuse rows in a client. Include the full business-relevant identity.

Avoid this mistake Marking an element key does not remove duplicate rows.

Read and verify this answer

SAP Learning on CDS concepts

CDS-12 Is a maintenance view a RAP view and how are defaults supplied?

A classic DDIC maintenance view belongs to table/view maintenance, often with generated maintenance dialogs. A RAP application uses a CDS model and behavior instead. RAP defaults may be implemented with determinations or supported defaulting functions; field visibility depends on the chosen UI and metadata.

Example and follow-up First clarify whether the interviewer means SM30 maintenance or a Fiori RAP maintenance app. Then explain the appropriate maintenance events or RAP behavior mechanism.

Avoid this mistake Metadata.ignore and Metadata.defaultValue are not generic annotations that implement maintenance behavior.

Read and verify this answer

SAP Learning on CDS concepts

SAP Learning on DCL inheritance

Official references

SAP Learning on CDS concepts

SAP Learning on DCL inheritance

SAP Learning on compositions

SAP analytical projection views

SAP CDS abstract entities

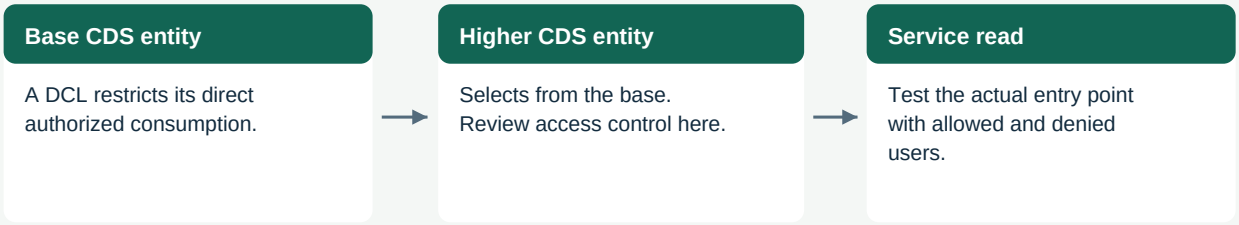
SAP RAP: implementing sorting of virtual elements

Senior CDS, query contracts and security

Diagnose data leaks, identity defects and query errors through the actual consumer boundary. Preserve business semantics, authorization and supported contracts when redesigning the model.

The exposed entity needs its own access decision

A base-layer DCL is not automatically inherited as an access filter



Mechanism	Purpose
DCL	Read access for the consumed CDS entity
RAP authorization	Permission to execute operations
Feature control	Availability in the current context

@AccessControl.authorizationCheck: #CHECK is not an authorization rule by itself.

Review security where consumers enter the model, rather than assuming that lower layers protect every upper view.

See the visual atlas for the recall prompt, limits and related questions.

MODEL-01 A new consumption entity exposes orders outside the user's sales organization. The base CDS has DCL. What do you investigate?

Trace the actual read entry point and its access control, including associations and alternative services. A protected base entity does not automatically confer equivalent protection on every higher entity. Define or inherit the intended restrictions through supported DCL constructs and mappings for the consumed entity. Keep operation authorization separate. The fix must preserve both authorized records and deliberate exceptions, rather than hiding the problem with a UI filter.

Example and follow-up Create two organizations with otherwise identical orders and use a restricted identity. Compare direct base consumption, the exposed projection and association navigation. Include count and value help: leaking a key or aggregate is still a data disclosure even when the main list looks correct.

Avoid this mistake Do not add #CHECK and assume it creates a missing rule.

Interviewer probes Which entry point actually evaluated DCL? / How would a field rename affect inherited restrictions?

Strong answer criteria Traces the exposed read path; Defines restrictions at the correct boundary; Tests negative and positive identities

Verification evidence Capture the real service responses for two organizational scopes, including navigation and counts.

Read and verify this answer

SAP Learning: authority checks and DCL

MODEL-02 An approver may update organization A. Can an update move an order from A to B using only existing-instance authorization?

Authorization on the current instance can establish permission for its existing state, but that does not automatically approve the proposed target values. Treat the incoming organization change as a separate policy decision. Use supported prechecks and business validation/operation checks at appropriate points, with explicit handling of create and update. Derivations must not silently introduce a target scope that the caller was never allowed to choose.

Example and follow-up An order starts in A, so an existing-instance check succeeds. The payload proposes B. Test permitted A-to-A changes, forbidden A-to-B changes, initial values and batch requests. Ensure internal local-mode updates occur behind a verified public operation and still preserve business consistency.

Avoid this mistake Do not equate authorization for the current object with authorization for any future object state.

Interviewer probes What is checked for creation? / What happens if a determination changes the organization?

Strong answer criteria Separates current and proposed state; Checks the public mutation boundary; Considers derived and bulk changes

Verification evidence Demonstrate a rejected cross-scope update through the actual service with no unauthorized persisted change.

Read and verify this answer

SAP RAP authorization control

MODEL-03 A custom query returns only authorized rows but an unrestricted total count. Is it secure?

No. Authorization must constrain the logical result before both counting and paging. A custom query provider owns the supported request semantics and cannot assume database-managed protections automatically cover remote or manually assembled data. Build one consistent authorized predicate and use it for data and count. Do not load all remote records into memory merely to hide unauthorized rows after an unrestricted count or aggregation.

Example and follow-up The caller may see five permitted orders while count reveals 900 hidden orders. Test zero-access users, count-only requests, filters, ordering and navigation. If the provider cannot securely support a requested aggregate or filter, reject it under a documented contract instead of returning a misleading result.

Avoid this mistake Filtering only the displayed page leaves count, aggregates and existence checks exposed.

Interviewer probes How do you handle count-only requests? / What if the remote API cannot apply your authorization predicate?

Strong answer criteria Uses one authorized result definition; Protects count and aggregates; Rejects unsupported semantics explicitly

Verification evidence Use restricted identities and compare authorized data, count and aggregate responses over the same fixture.

Read and verify this answer

SAP: Requesting and Setting Data or Count

MODEL-04 A query-provider filter cannot be converted to simple ranges. A developer catches the exception and returns everything. What is the correction?

Preserve the requested Boolean semantics or reject the unsupported filter clearly. Range conversion is not a universal representation of arbitrary nested AND/OR expressions and functions. Use a supported expression representation or another safe translation strategy appropriate to the provider. Combine user filters with mandatory authorization predicates independently, so a parsing failure cannot remove either restriction. Validate dynamic identifiers and values through supported APIs.

Example and follow-up Consider (region=A AND amount>100) OR (region=B AND urgent=true). Flattening this into independent region and amount ranges changes the result. Build fixtures that distinguish the intended expression from the flattened one, and include an unsupported-function request.

Avoid this mistake A swallowed filter exception must not become an unrestricted selection.

Interviewer probes Can the backend express the same predicate? / How do mandatory authorization filters survive translation failure?

Strong answer criteria Preserves AND/OR grouping; Separates mandatory restrictions; Returns an explicit unsupported-request outcome

Verification evidence Compare expected row sets for nested filters and prove failure never broadens authorized access.

Read and verify this answer

SAP: Implementing Filtering in an Unmanaged Query

MODEL-05 A paged custom entity repeats some rows after an insertion. How would you make the contract predictable?

Start with deterministic ordering that includes a unique tie-breaker; sorting only by a nonunique date is insufficient. Then distinguish deterministic order from snapshot consistency: concurrent inserts can still shift offset pages. Choose the supported continuation or consistency mechanism, or document the weaker browsing behavior if a snapshot is unnecessary. For an export requiring completeness, design a stable extraction boundary rather than assuming UI paging is a reliable bulk-transfer protocol.

Example and follow-up Twenty orders share a creation date. A new order is inserted between page requests. Test tied values, reverse sorting, deleted rows, empty pages and a count change. State whether the product requires an interactive current list or a reproducible export.

Avoid this mistake Adding ORDER BY alone does not guarantee a frozen dataset across requests.

Interviewer probes What is the tie-breaker? / Does the use case require a snapshot or just stable ordering?

Strong answer criteria Separates ordering from consistency; Chooses a contract for concurrent changes; Tests page boundaries with duplicate sort values

Verification evidence Run successive requests while inserting and deleting records and verify the documented paging behavior.

Read and verify this answer

SAP: Interface IF_RAP_QUERY_PAGING

MODEL-06 Adding a language-dependent text association doubles order totals. Would DISTINCT be a good fix?

First identify the grain of the result and the actual join multiplicity. Check language, validity and other conditions that make the text association to-one or to-many. Declared cardinality is not data cleanup and can mislead optimization when false. Repair the relationship or aggregate at the correct grain before exposing the measure. DISTINCT can mask the symptom while retaining incorrect totals or discarding legitimate detail.

Example and follow-up One order worth 100 EUR joins two text rows and contributes 200 EUR to a sum. Add overlapping validity periods to the fixture as well as two languages. Verify both the detailed entity and the consuming analytical or list result.

Avoid this mistake Do not fix an incorrect join by simply declaring a to-one cardinality.

Interviewer probes What is one row supposed to represent? / Which data condition establishes uniqueness?

Strong answer criteria Defines result grain; Proves association multiplicity; Checks the downstream measure

Verification evidence Show expected rows and totals for multiple texts, missing texts and overlapping validity intervals.

Read and verify this answer

SAP CDS modeling course

MODEL-07 A CDS report sums 100 EUR and 100 USD into 200. What should the model and service do?

Preserve amount/currency and quantity/unit semantics throughout the model. A sum is meaningful only within compatible units or after an explicitly defined conversion. Decide the target currency, rate type, date and rounding policy with the business owner. An annotation communicates semantics but does not, by itself, perform a correct conversion or make arbitrary aggregation valid. Verify how the actual analytical or transactional consumer interprets the exposed fields.

Example and follow-up Provide mixed currencies, a missing exchange rate and a value requiring rounding. A summary can group by currency or apply approved conversion, but must not silently combine incomparable values. Keep original values available where reconciliation requires them.

Avoid this mistake Do not confuse semantic annotations with conversion logic.

Interviewer probes Which date determines the rate? / At which stage is rounding applied?

Strong answer criteria Maintains unit semantics; Names the business conversion policy; Reconciles rounded and original values

Verification evidence Compare expected totals for same-currency, mixed-currency and missing-rate fixtures.

Read and verify this answer

SAP CDS modeling course

MODEL-08 A transactional projection needs a calculated field that its provider contract does not permit. How do you redesign it?

Check the exact projection/provider-contract restrictions in the target release. Place permitted calculations in an appropriate underlying data model, expose a suitable read model, or use a supported virtual-element mechanism when its limitations fit. Do not turn a transactional projection into an arbitrary join solely to satisfy the UI. Preserve stable identity, behavior mapping, authorization and update semantics across the chosen design.

Example and follow-up The UI requests a derived risk band and wants to filter by it. A transient calculation may display correctly but fail server-side filtering. Decide whether the risk belongs in persistent business state, a database expression or a read-only enrichment, and test that consumer requirement.

Avoid this mistake A field that displays in preview is not proof of a supported filterable transactional contract.

Interviewer probes Does the field participate in filtering or ordering? / Who owns recalculation when source data changes?

Strong answer criteria Checks the provider contract; Preserves transactional identity; Designs for consumer query semantics

Verification evidence Exercise read, filter, order and update against the intended binding on the target release.

Read and verify this answer

SAP CDS modeling course

MODEL-09 The business wants to sort a million-row list by a virtual element computed through a remote call. What would you propose?

Challenge the requested access pattern before implementing row-by-row enrichment. A virtual element does not automatically support efficient database sorting, filtering or aggregation. Consider a permitted persistent read model with refresh rules, a supported pushdown calculation, or a bounded enrichment where sorting is not required. State freshness, failure and reconciliation requirements. Avoid moving an unbounded database workload into ABAP or remote calls.

Example and follow-up A risk score depends on a remote provider. Define how stale it may be and what the UI shows when the provider is unavailable. Compare query latency and result correctness with representative volumes; a fast demo using ten rows does not establish scalability.

Avoid this mistake Do not claim that adding a virtual element automatically pushes its logic to HANA.

Interviewer probes What freshness is acceptable? / How many remote calls occur for one page and its count?

Strong answer criteria Identifies unsupported query operations; Controls remote-call volume; Makes freshness and failure explicit

Verification evidence Measure request count, rows transferred and latency with a realistic dataset and a failing remote dependency.

Read and verify this answer

SAP RAP: implementing sorting of virtual elements

MODEL-10 Two rows in a service share the declared CDS key after a join. Why is this more than a display issue?

Consumer identity must uniquely and stably distinguish each exposed instance. Declaring an element as key does not make a nonunique SQL result unique. Duplicate identities can corrupt navigation, client caching, selection and mutation targeting. Correct the data grain, join or composite key while assessing compatibility for existing consumers. If the current public identity is wrong, plan an explicit API transition rather than silently changing key meaning.

Example and follow-up An order-item service exposes only OrderId as key. Two items appear with the same identity and the UI targets the wrong item. Test read-by-key and navigation, not merely row count. Include repeated reads in different orders to reveal unstable client behavior.

Avoid this mistake Do not hide duplicate-key rows with client-side deduplication.

Interviewer probes Can the public key change compatibly? / Is the entity a business object instance or an aggregate?

Strong answer criteria Proves key uniqueness at exposed grain; Explains consumer consequences; Plans compatible identity evolution

Verification evidence Validate uniqueness, stable canonical URLs and correct read/update targeting for multiple child records.

Read and verify this answer

SAP CDS modeling course

MODEL-11 A value help reveals customer names that the order list hides. Where should the fix live?

Treat the value-help entity and its text/navigation paths as separately consumed data contracts. Apply the required access restrictions and validate actual user identity and context. UI annotations can connect a value help but do not establish its authorization. Ensure the selected value is still validated at the write boundary because clients can submit a value without using the UI chooser.

Example and follow-up A restricted user opens customer search and learns names outside their organization. Test direct access to the value-help service, search/count behavior and a forged customer value in an order update. The read restriction and business consistency check solve related but distinct problems.

Avoid this mistake Hiding the value-help control is not a backend security fix.

Interviewer probes Can the entity be consumed directly? / What checks a value submitted outside Fiori?

Strong answer criteria Protects auxiliary data exposure; Checks direct service access; Validates proposed business references

Verification evidence Execute denied-user searches and direct payload changes using allowed and forbidden customer keys.

Read and verify this answer

SAP Learning: authority checks and DCL

MODEL-12 A batch job asks to use privileged EML because normal authorization rejects it. How do you evaluate the request?

First establish the job identity, its legitimate business scope and whether normal authorization can represent that scope. Privileged access is a provider-enabled contract with specific bypass behavior, not a generic workaround. Review the defined authorization contexts, exposed entry points and caller trust boundary. If privilege is justified, narrowly encapsulate it, preserve remaining business checks and produce auditable evidence of who requested which effect.

Example and follow-up A repair job should reconcile one approved organizational scope, not become an unrestricted posting API. Test both the approved job path and attempts to reach the helper from an ordinary consumer. Confirm which checks are bypassed in the actual provider contract.

Avoid this mistake Do not assume PRIVILEGED, IN LOCAL MODE and a technical user are equivalent.

Interviewer probes Which authorization context is disabled? / Why cannot ordinary scoped authorization satisfy the requirement?

Strong answer criteria Requires a provider-supported privilege contract; Limits the trusted entry point; Retains business checks and auditability

Verification evidence Review privilege configuration and demonstrate allowed repair plus rejected unauthorized invocation.

Read and verify this answer

SAP EML local and privileged modes

MODEL-13 A developer uses IN LOCAL MODE in a helper to avoid feature checks. What makes the helper safe or unsafe?

Locate the helper within its own BO implementation and trace every public operation that can reach it. Local mode supports internal state maintenance and bypasses defined checks depending on the operation; it does not authorize arbitrary external business requests or erase transactional constraints. Ensure public inputs have already been checked and that internal changes preserve invariants. Review cross-BO calls under their own contracts.

Example and follow-up A determination legitimately updates a derived readonly amount. The same helper becomes unsafe if a public action forwards arbitrary caller values to it. Test that the derived field cannot be controlled indirectly through a newly added action parameter.

Avoid this mistake Do not fix missing authorization by routing all work through local mode.

Interviewer probes Which controls does this specific operation bypass? / Can a caller influence the supposedly derived values?

Strong answer criteria Traces trust from public input; Confines internal state changes; Distinguishes own-BO and cross-BO access

Verification evidence Use adversarial action inputs and verify that only approved derivations reach readonly or protected fields.

Read and verify this answer

SAP EML local and privileged modes

MODEL-14 A CDS unit test passes, but the same entity leaks data for a restricted user. What may the test be missing?

Inspect the test environment and access-control configuration. A test focused on relational logic with CDS doubles may intentionally isolate or disable access control; it is not automatically an authorization test. Enable the supported access-control testing mechanism with meaningful role data where available, and add real service tests under restricted identities. Keep query-correctness evidence separate from security evidence.

Example and follow-up One test proves the join returns three rows. Add fixtures for no authorization and two distinct scopes, then ensure the test actually evaluates DCL. A final service test also verifies binding, identity propagation and the real consumed entity.

Avoid this mistake A green database-double test does not prove deployed authorization.

Interviewer probes Is DCL enabled in the test environment? / Which real identity is used by the service?

Strong answer criteria Inspects double behavior; Adds denied and scoped authorization cases; Separates unit and deployed evidence

Verification evidence Show test configuration plus restricted-user service results; do not infer one from the other.

Read and verify this answer

SAP CDS access-control tests

MODEL-15 An integration user can access all tenants or clients through a custom read provider. What boundaries must be reviewed?

Separate technical client handling, tenant isolation, business authorization and remote credentials. Standard data-access behavior does not automatically protect arbitrary remote calls or custom assembled datasets. Avoid accepting unrestricted client/tenant selectors from untrusted request input. Use supported tenant-aware access and configuration, enforce business scope and test identity propagation. The exact tenant model differs by environment and must be established explicitly.

Example and follow-up A provider forwards an incoming company or tenant parameter to a privileged remote destination. Even if the local CDS looks restricted, the remote response may cross the intended boundary. Test two scopes with overlapping business IDs and inspect both request construction and returned data.

Avoid this mistake Do not treat a client field, a destination name or authentication alone as complete tenant isolation.

Interviewer probes Who controls the remote tenant selector? / Which identity does the remote system evaluate?

Strong answer criteria Separates technical and business scope; Constrains untrusted selectors; Checks end-to-end identity propagation

Verification evidence Run cross-scope negative tests and review the permitted communication configuration without exposing secrets.

Read and verify this answer

SAP RAP authorization control

MODEL-16 A projection hides an operation, but another consumer can still call the base BO. Is hiding sufficient?

Projection restrictions define a consumer-facing operation surface, not a universal revocation of the underlying capability. Review who can consume the base BO or another released interface, and enforce security at the appropriate behavior boundary. Keep public API release, package visibility and operation authorization aligned. Feature control can improve availability feedback but cannot replace authorization for alternative entry points.

Example and follow-up A UI projection excludes Delete while a background consumer uses a broader local BO interface. Decide whether that capability is intentional for the background role. Test direct access to every supported mutation path using an unauthorized identity.

Avoid this mistake Do not use the UI service shape as the only security policy.

Interviewer probes Which consumers are permitted to use the base contract? / Where does the decisive permission check execute?

Strong answer criteria Inventories alternate entry points; Separates surface restriction from permission; Tests local and remote consumers

Verification evidence Provide a consumer/operation/identity matrix and evidence of denied mutation through each exposed path.

Read and verify this answer

SAP BO interfaces

MODEL-17 A released entity gains a new field that is expensive and sensitive. Is an additive change automatically safe?

Not necessarily. Check default projection, serialization, authorization, query cost and consumers that interpret metadata or request broad selections. Preserve compatibility while deciding whether the field belongs in the same public contract or a more restricted projection. Avoid unexpected remote enrichment or large payload growth on existing list requests. Verify the relevant release contract and supported evolution rules before shipping.

Example and follow-up A new personal-data field is included by broad selects and exports. Another computed field adds one remote call per row. Existing callers did not change, but privacy and latency did. Test default reads, explicit selection, metadata-driven UIs and restricted identities.

Avoid this mistake Do not assess compatibility using only whether old source code still compiles.

Interviewer probes Does an existing broad select expose it? / How does it affect authorization and request cost?

Strong answer criteria Assesses semantic and operational compatibility; Controls default exposure; Tests unchanged consumers

Verification evidence Compare old/new payloads, access results and latency for representative existing requests.

Read and verify this answer

SAP BO interfaces

MODEL-18 An analytical requirement is added to a transactional BO. Would you put every aggregate into its projection?

Choose a read model appropriate to the analytical grain and consumer rather than distorting transactional identity and behavior. Reuse trustworthy underlying semantics where suitable, then define aggregation, currency, authorization and freshness deliberately. Keep transactional mutations on the BO contract. A separate analytical model may be justified, but avoid duplicating business rules without an ownership and consistency plan.

Example and follow-up An approval object needs both item editing and monthly spend by organization. The monthly aggregate is not the same entity identity as an editable item. Compare whether live calculation meets latency or whether a reconciled read model is necessary.

Avoid this mistake Do not make an aggregate writable merely because its source is a transactional BO.

Interviewer probes What is the aggregate grain? / How are authorization and currency preserved across the models?

Strong answer criteria Separates transaction and analytical identity; Reuses semantics intentionally; Defines freshness and reconciliation

Verification evidence Reconcile grouped totals to authorized transactional fixtures and validate the intended analytical consumer.

Read and verify this answer

SAP analytical projection modeling

Official references

SAP RAP authorization control

SAP Learning: authority checks and DCL

SAP EML local and privileged modes

SAP CDS access-control tests

SAP CDS modeling course

SAP BO interfaces

SAP custom-entity application example

SAP analytical projection modeling

SAP RAP: implementing sorting of virtual elements

RAP service design and advanced behavior

Use this segment for deeper follow-up questions. Connect each feature to a concrete requirement and explain its limits. Release-specific BDL syntax and UI behavior must be checked against the employer's ABAP and SAPUI5 versions.



Alice and Ben read version 7. Alice updates successfully, producing version 8. Ben must not overwrite that newer state using the stale version 7 precondition.

See the visual atlas for the recall prompt, limits and related questions.

ADV-01 How do pessimistic locks and ETags complement each other?

A pessimistic lock prevents conflicting processing for a time. An ETag detects whether the version a client read is still current when it writes. RAP defines lock ownership and optimistic concurrency through behavior. Draft scenarios also need total-ETag handling for relevant active-data changes.

Example and follow-up Two users read a request. After one saves, the other's stale If-Match must not silently overwrite it. A draft lock and an active-data version check solve different parts of this problem.

Avoid this mistake Locking and logging are different concepts; an ETag is not a log and does not reserve the object.

Read and verify this answer

SAP draft concept

ADV-02 Global authorization instance authorization or feature control?

Global authorization checks permissions independent of a specific instance. Instance authorization considers an instance's context. Feature control enables operations or fields according to business state. UI availability and feature control help consumers, but authorization must enforce who may perform the operation.

Example and follow-up A submitted request may be readonly for everyone as a feature rule; approval may additionally require the assigned approver's authority. Test a direct API request as well as the disabled UI button.

Avoid this mistake Hiding or disabling a button is not an authorization check.

Read and verify this answer

SAP Learning on authorization

ADV-03 What does authorization none on an action mean?

Where supported, authorization: none exempts that action from the RAP authorization checks specified for it. It does not remove authentication at the service boundary or every authorization check in downstream code. Use it only with a deliberate security model and the exact supported behavior syntax.

Example and follow-up An action that needs no object-specific authorization may still operate through an authenticated service and call another API that performs its own checks. Document why the exemption is justified.

Avoid this mistake Do not propose it as a universal fix for authorization errors or expose a sensitive action by accident.

Read and verify this answer

SAP Learning on authorization

ADV-04 Instance static and factory actions?

An instance action operates on identified instances. A static action belongs to the entity without requiring an existing instance. A factory action creates instances and returns the appropriate creation mapping. Their inputs, results and identity handling differ, so choose according to the business operation.

Example and follow-up Approve acts on a request, CalculatePreview may be static, and CopyRequest can be a factory action. A normal instance action returning $self is not automatically a factory action.

Avoid this mistake Static does not mean a static ABAP method that can ignore RAP identity and response contracts.

Read and verify this answer

SAP action definitions

ADV-05 How do action parameters and results work?

Define a supported parameter and result shape in the behavior contract, often using a CDS abstract entity for structured input. Implement the generated handler signature and populate the requested result and response structures correctly. Cardinality and binding to instances are part of the contract.

Example and follow-up Submit may accept a comment; CalculatePreview may return totals without persisting an object. The UI may generate a dialog from the parameter metadata, subject to consumer support.

Avoid this mistake Parameters are not defined by putting an action inside an abstract entity.

Read and verify this answer

SAP action definitions

ADV-06 What is augmentation and how does it differ from additional save?

Augmentation adapts supported modifying operations at a RAP projection boundary, allowing projection-specific input to be enriched or translated. Additional save adds persistence-related work alongside a managed save scenario. They run for different purposes and are not interchangeable hooks.

Example and follow-up A projection may transform consumer-specific input into changes understood by the underlying BO. Audit records associated with managed persistence may instead need an additional-save design compatible with the transaction.

Avoid this mistake Augmentation is not a generic before-save hook for copying every virtual element into a table.

Read and verify this answer

SAP operation augmentation

ADV-07 What are side effects in a RAP UI?

Side-effect metadata tells a capable consumer which values, messages or permissions to refresh after a relevant change or action. The backend must still compute correct results; the metadata coordinates refresh behavior. Backend release, service metadata and SAPUI5 support determine which declarations are available.

Example and follow-up After changing quantity, refresh total amount and action availability. Verify network requests and the displayed values, especially after an action result or draft update.

Avoid this mistake A side-effect declaration does not implement the calculation or force all non-UI consumers to issue extra reads.

Read and verify this answer

SAP RAP side effects

ADV-08 State messages versus transition messages?

A state message describes an invalid state of an instance and may persist in a supported draft scenario until revalidated. A transition message relates to the current operation or request. Use the appropriate state area and instance or field references and clear obsolete state correctly.

Example and follow-up A missing mandatory date can remain highlighted while editing a draft. A successful one-time submission notification should not reappear forever as a stale state error.

Avoid this mistake Every error should not be stored as a persistent state message, and REPORTED alone does not block saving.

Read and verify this answer

SAP RAP state messages

ADV-09 Service definition versus binding and UI versus Web API?

A service definition selects exposed entities and names. A binding attaches a protocol and service type, such as OData V4 UI or Web API. UI services support relevant UI semantics; Web APIs target programmatic consumers. Activation, local publication, authorization and deployment are separate concerns.

Example and follow-up Choose the consumer first, then inspect generated metadata and test the actual endpoint. A local ADT preview is useful, but transport and production communication setup still need validation.

Avoid this mistake VDM.viewType: #COMPOSITE neither creates nor activates a service binding.

Read and verify this answer

SAP service binding documentation

ADV-10 What must a custom query provider implement?

A provider supplies data according to the requested query contract, including applicable filtering, ordering, paging, selected elements and total count. It should process only requested information and preserve authorization. Unsupported query options must be handled deliberately. Remote sources add latency, consistency and error-mapping concerns.

Example and follow-up Handle data and total-count requests independently: a count-only request must not depend on a data-request branch. Compute count over the same authorized, filtered result before paging, and use deterministic ordering for data. A zero page size requests no rows; distinguish it from IF_RAP_QUERY_PAGING=>PAGE_SIZE_UNLIMITED. Do not accidentally translate zero into an unrestricted SQL read. Test count-only, empty-page and unlimited requests as well as ordinary pages.

Avoid this mistake Returning any internal table is not a correct implementation of IF_RAP_QUERY_PROVIDER.

Read and verify this answer

SAP: Implementing an Unmanaged Query

SAP: Requesting and Setting Data or Count

SAP: Interface IF_RAP_QUERY_PAGING

ADV-11 How do RAP business events differ from ABAP Objects events?

ABAP Objects events use event handlers within the ABAP execution context. RAP business events describe business occurrences under RAP's event contract; supported local or remote consumption has its own lifecycle. Remote enterprise events require configured infrastructure and transactional publication, with retry and idempotency designed explicitly.

Example and follow-up Use an integration event after a request is durably submitted. A remote consumer should tolerate duplicate delivery and should not approve a request merely because it received an uncommitted local callback.

Avoid this mistake Do not collapse every locally consumed RAP event into synchronous SET HANDLER semantics or promise universal ordering.

Read and verify this answer

SAP: Develop Business Events

ADV-12 How do you extend an existing RAP BO safely?

Inspect its released extension contract and supported extension points before adding fields or behavior. Check model, projection, metadata, authorization, save and service implications. Extension support depends on the BO and release; an accessible source object is not necessarily released for extension.

Example and follow-up For an extra approval reference, verify persistence extension, exposure, validation, update behavior and consumer compatibility end to end. Test the standard flow without the new field as well.

Avoid this mistake Being able to create a CDS extension does not prove every BO behavior or API contract is extensible.

Read and verify this answer

SAP: RAP Extensibility Enablement

ADV-13 How do static and dynamic feature controls differ?

Static restrictions are declared in the behavior definition, for example a readonly or mandatory field. Dynamic controls determine supported operation or field availability from global or instance context. They express business availability, while authorization remains separate. Check which controls the operation and target release support.

Example and follow-up An approved request may dynamically forbid editing. A generated UUID may be statically readonly. Global and instance feature control cannot both be declared for the same operation; choose the applicable contract and implement its generated handler.

Avoid this mistake A dynamic UI state does not prove security, and mandatory metadata does not replace all backend business validations.

Read and verify this answer

SAP feature control

Official references

SAP Learning on authorization

SAP service binding documentation

SAP draft concept

SAP RAP documentation

SAP operation augmentation

SAP action definitions

SAP feature control

SAP RAP side effects

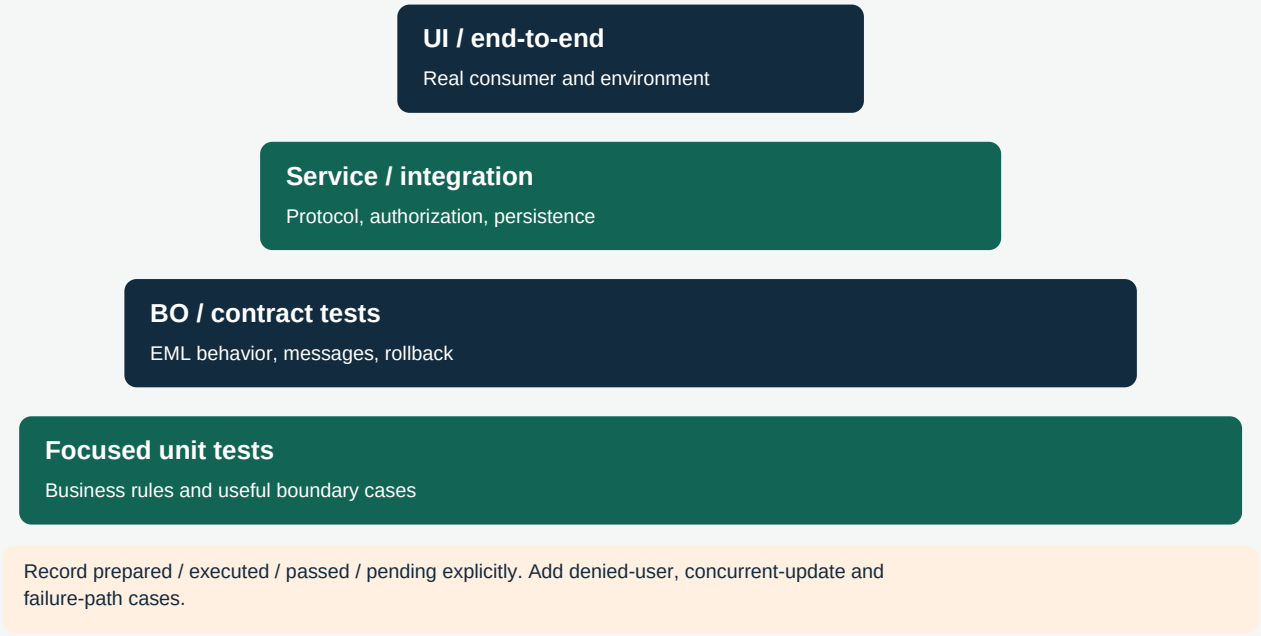
SAP RAP state messages

Senior RAP testing, performance and production

Defend the evidence behind a cloud service: observable behavior, failure recovery, real consumer security, measured performance and reliable delivery. All scenarios are hypothetical; no SAP execution is implied.

Different tests produce different evidence

A passing test double does not prove a deployed service works



Record prepared / executed / passed / pending explicitly. Add denied-user, concurrent-update and failure-path cases.

The widening base suggests many focused checks, but there is no mandatory numerical ratio.

See the visual atlas for the recall prompt, limits and related questions.

OPS-01 How would you test a Submit action without replacing the business object you are supposed to verify?

Keep the Submit implementation and its relevant BO behavior real. Isolate true dependencies such as database tables, external service adapters or other BOs using the appropriate supported doubles. Drive the operation through a meaningful boundary, often EML, and assert state changes, failures and messages. A double of the very action under test would prove the configured double response rather than the business rule.

Example and follow-up A purchase request can be submitted only when every item has an account assignment. Test a valid request and a missing assignment, including persisted outcome after the supported save boundary. If another BO supplies a limit, double that dependency and vary its response.

Avoid this mistake Do not label a test of a mocked Submit response as a test of Submit.

Interviewer probes Which behavior remains real? / How do you prove rejected data was not persisted?

Strong answer criteria Defines the unit/contract under test; Doubles only dependencies; Asserts state plus response semantics

Verification evidence Show the dependency diagram and a failing test when the real Submit check is deliberately broken.

Read and verify this answer

SAP RAP test frameworks and boundaries

OPS-02 When would SQL doubles, CDS doubles and RAP BO doubles give misleading confidence?

They isolate different dependencies. SQL doubles replace table access, CDS doubles support tests of CDS logic, and RAP BO doubles isolate dependent EML behavior. Choose the boundary according to the risk being tested. Generic BO buffer doubles do not execute arbitrary custom provider rules, and access-control behavior needs explicit treatment. Complement isolated tests with targeted integration tests for actual persistence, authorization and service contracts.

Example and follow-up A calculation test may use a SQL double successfully while a dependent action contract changes unnoticed. Conversely, a mocked EML response can test consumer error handling without proving the provider action. Name exactly which conclusion each test supports.

Avoid this mistake A more sophisticated double does not automatically provide broader integration evidence.

Interviewer probes Does this double execute custom determinations? / Which real boundary still needs verification?

Strong answer criteria Matches doubles to dependency type; States excluded behavior; Adds focused integration evidence

Verification evidence Record test scope and verify one representative path through the real provider and service.

Read and verify this answer

SAP RAP test frameworks and boundaries

SAP transactional buffer double support

OPS-03 A bulk EML action processes ten requests and rejects two. What should its test assert?

Assert each input-to-result correlation and the intended atomicity contract, not just the number of messages. Verify which identities appear in FAILED, where reported messages point, and what changes remain buffered or persisted after transaction completion. Cover failures during interaction and save separately. The desired all-or-nothing or permitted partial-result behavior must be explicit and supported by the chosen request boundary.

Example and follow-up Use nonsequential keys and distinct invalid reasons so accidental row-position correlation is visible. Reorder input and response fixtures. Include a successful interaction followed by a save rejection, and ensure the consumer does not report that object as durably complete.

Avoid this mistake Do not equate ten returned messages with ten correct operation outcomes.

Interviewer probes Which failures occur only on save? / Can the caller correlate reordered results?

Strong answer criteria Checks per-instance correlation; Separates interaction and persistence; Tests the declared atomicity rule

Verification evidence Show expected accepted/rejected identities, field messages and final persisted rows for each failure phase.

Read and verify this answer

SAP RAP test frameworks and boundaries

OPS-04 A draft test passes when run alone but fails after another test. What would you inspect?

Inspect shared fixtures, draft and active identities, transactional buffers, locks and double-environment lifecycle. Each test must establish its own relevant state and release/reset supported test resources. Do not use uncontrolled commits to make a suite pass. Verify draft-specific actions and the chosen test framework boundaries rather than assuming an active-instance fixture represents a draft.

Example and follow-up Two tests reuse the same business key but differ in draft context. One leaves a buffer or configured double response behind. Run the pair in reversed order and with distinct fixtures, then fix lifecycle isolation while preserving the scenario being tested.

Avoid this mistake A test that depends on execution order is unreliable evidence for draft behavior.

Interviewer probes Are active and draft transactional keys distinct? / Which setup/teardown resource is shared?

Strong answer criteria Tracks complete transactional identity; Resets supported resources; Checks ordering independence

Verification evidence Run the isolated tests, both orders and the suite; retain evidence of deterministic results.

Read and verify this answer

SAP RAP test frameworks and boundaries

OPS-05 How do you prove optimistic concurrency rather than merely calling the same update twice?

Use independent consumers or sessions that read the same initial version, then perform conflicting updates in a controlled order. Assert that the first permitted write succeeds and the stale second precondition is rejected without lost data. For draft, separately test the exclusive-lock and total-ETag situations. A sequential test that rereads after every update never exercises a stale writer.

Example and follow-up Alice and Ben read the same entity instance with its entity ETag. Alice updates that instance successfully; Ben submits an update to that same instance using the stale ETag and must not overwrite it. Separately test child-only active changes against the configured draft total ETag when resuming an expired draft lock. Keep entity-level conditional update and draft-resume concurrency assertions distinct.

Avoid this mistake Two updates in one shared transaction are not a realistic two-user concurrency test.

Interviewer probes How do you control the interleaving? / Which field protects the whole draft object?

Strong answer criteria Uses independent contexts; Creates a deliberately stale request; Asserts unchanged data after rejection

Verification evidence Capture request versions, responses and final values for the orchestrated conflicting sessions.

Read and verify this answer

SAP total ETag and draft concurrency

OPS-06 Fiori still displays an old total after a successful action. How do you separate UI refresh from backend correctness?

Check the actual operation response and a fresh read of the authoritative BO state. If the backend value is wrong, inspect derivation triggers and transactional reads. If the backend is correct, inspect the action result, side-effect declarations, requested fields and consumer support. Avoid adding a second calculation in the UI that conceals missing backend consistency. Test the real deployed metadata rather than only a local preview.

Example and follow-up Changing an item updates the server total but the header remains stale. Compare a direct read, a new UI session and the original session network requests. The correction may be a targeted refresh contract, not another commit or a full-page reload.

Avoid this mistake A page reload workaround does not explain whether the backend or refresh contract was defective.

Interviewer probes What does a fresh service read return? / Which request should refresh the affected property?

Strong answer criteria Establishes the authoritative value; Traces metadata and network behavior; Keeps business calculation server-side

Verification evidence Record action and follow-up reads with before/after values in the actual consumer.

Read and verify this answer

SAP RAP side effects

OPS-07 A RAP list becomes slow after enabling count and expanding children. What do you measure first?

Decompose the request into data selection, total count, expansion, authorization and any ABAP or remote enrichment. Use available traces and representative parameters to measure rows, calls, database time and application time. Optimize the observed bottleneck while preserving query and security semantics. Restrict unnecessary fields or expansions where the contract permits, and verify changes with the same workload rather than an unrelated microbenchmark.

Example and follow-up A 20-row page triggers an expensive count across millions of authorized records plus child expansion. Removing a SELECT from one handler may be irrelevant. Compare page-only, count-only and expanded requests, including a restricted user whose authorization predicate changes the plan.

Avoid this mistake Do not optimize the shortest visible SELECT before finding the dominant request cost.

Interviewer probes Is count more expensive than the page? / What changes for an authorization-restricted user?

Strong answer criteria Breaks down the actual request; Measures representative volume and identity; Preserves semantics while optimizing

Verification evidence Retain comparable request traces and latency/row/call measurements before and after the change.

Read and verify this answer

SAP SQL analysis in ADT

OPS-08 A custom entity calls a remote API once for every returned row. How would you remove the N+1 pattern?

Look for supported bulk reads, selective remote filters or bounded prefetch keyed by the requested result set. Consider a governed cache or read model only with explicit freshness and scope rules. Avoid fetching the whole remote dataset to save call overhead. Propagate partial and transient failures honestly, protect credentials and tenant boundaries, and define an upper bound on work per request.

Example and follow-up One list page of 50 items makes 51 remote calls. A bulk endpoint can retrieve the needed reference data in one request, but may return partial failures. Test missing references, duplicate keys and service throttling, not only the happy-path call count.

Avoid this mistake An unbounded cache can trade latency for stale or cross-user data leakage.

Interviewer probes What is the maximum request size? / How are partial remote failures represented?

Strong answer criteria Uses bounded bulk access; Defines freshness and scope; Handles partial and throttled responses

Verification evidence Measure remote-call count and transferred rows while testing missing data and rate limits.

Read and verify this answer

SAP: Implementing an Unmanaged Query

OPS-09 A service works in development but fails after transport. What evidence should the handoff have included?

Separate transported repository artifacts from required configuration, communication setup, identities, roles, endpoint exposure and frontend deployment. Verify dependency order and the exact active source/version in the target. A published local service binding or preview in development does not establish a consumable production endpoint. The handoff should identify owners and verifiable readiness checks for each non-code dependency.

Example and follow-up The behavior activates, but an outbound destination or communication arrangement is missing. Another issue is a target role not granting the service. Diagnose using the failure boundary and configuration evidence; do not change business code until the deployed source and configuration are reconciled.

Avoid this mistake A green transport import is not end-to-end service acceptance.

Interviewer probes Which required items are configuration rather than source? / Who verifies the real consuming identity?

Strong answer criteria Inventories code and configuration separately; Checks active target state; Assigns operational owners

Verification evidence Use a target readiness checklist with actual endpoint, role, communication and smoke-test outcomes.

Read and verify this answer

SAP RAP service binding

OPS-10 How would you evolve a released RAP service when an action needs a new mandatory input?

Treat a new mandatory parameter as a potential breaking contract change. Assess existing consumers and choose a compatible optional/defaulted path only if its business meaning is valid; otherwise introduce a deliberate successor operation or version under the supported lifecycle. Keep old behavior operational during migration and provide deprecation guidance. Validate generated clients, metadata-driven UIs and non-UI consumers separately.

Example and follow-up An approval action now requires a legal justification that old clients cannot supply. An empty default may violate the real business rule. Plan a transition with a measurable consumer migration target rather than silently accepting invalid data or breaking every caller at once.

Avoid this mistake Do not call a signature change compatible merely because the server activates.

Interviewer probes Is a default legally/business valid? / How do you know which consumers still use the old contract?

Strong answer criteria Identifies the breaking semantic change; Chooses an explicit transition; Verifies existing consumers

Verification evidence Run old/new consumer contract tests and track observed usage before retiring the old operation.

Read and verify this answer

SAP RAP service binding

OPS-11 A remote approval succeeded, but the RAP caller timed out. How should an operator recover?

Treat the outcome as uncertain until reconciled using stable correlation. Do not blindly resubmit a non-idempotent business action. Identify which system owns each committed effect and use the documented status query, idempotency contract or compensation process. Preserve a durable recovery record and route unresolved cases to an owner. A local rollback cannot erase a remote commit.

Example and follow-up The remote workflow created an approval instance, but its response was lost. A retry should find or reuse that instance rather than create another. Include a later duplicate callback and an approval referring to an outdated local version.

Avoid this mistake A timeout is not evidence that the other system performed no work.

Interviewer probes What is the stable correlation key? / Who resolves an outcome that cannot be queried?

Strong answer criteria Separates unknown from failed outcome; Reconciles before unsafe retry; Handles duplicate and stale callbacks

Verification evidence Simulate lost responses and verify one intended business effect plus a visible recoverable status.

Read and verify this answer

SAP RAP business-object contract

OPS-12 A failed save leaves no business record, but support still needs diagnostics. How would you log it safely?

Distinguish transactional business audit data from operational diagnostics and user-facing messages. Data written in the same rolled-back transaction may disappear, while independent commits from behavior code violate transaction ownership. Use supported logging/monitoring facilities with documented lifecycle and privacy behavior. Correlate failures without exposing secrets or persisting a false successful business event.

Example and follow-up A validation rejects an order and REPORTED provides a user message. Support needs a correlation to an observed request failure. Decide what belongs in the service/application monitoring layer and what must be saved only alongside a successful business transition.

Avoid this mistake Do not add COMMIT WORK inside a handler just to preserve a log entry.

Interviewer probes Should this record survive a rollback? / Is it diagnostic evidence or a business audit event?

Strong answer criteria Separates log purposes and lifetimes; Respects RAP completion ownership; Redacts sensitive diagnostic content

Verification evidence Test success and rollback, then inspect which messages and logs survive through supported mechanisms.

Read and verify this answer

SAP RAP business-object contract

OPS-13 A production incident starts after a release. How do you contain it without making the data worse?

Establish affected scope, current version and the failing transaction boundary before changing code or replaying messages. Stop or limit the damaging path through authorized operational controls, preserve diagnostic evidence and assess already committed effects. Reconcile data and queues before choosing rollback, forward fix or controlled replay. A code rollback is not a general data rollback.

Example and follow-up A new Submit action duplicates downstream approvals. Disable the affected scheduled initiation if authorized, retain correlations and identify duplicate effects. Repair idempotency and reconcile existing approvals before resuming a backlog, with a measured recovery window.

Avoid this mistake Do not rerun the whole backlog while the duplicate-creation defect remains active.

Interviewer probes What effects are already durable? / What proves it is safe to resume processing?

Strong answer criteria Contains before replaying; Separates code from data recovery; Defines a measurable resume criterion

Verification evidence Produce an incident timeline, affected-record reconciliation and a controlled post-fix replay result.

Read and verify this answer

SAP RAP business-object contract

OPS-14 What belongs in a RAP release gate beyond a passing ATC run?

Combine static findings with contract and behavior tests, relevant authorization cases, concurrency and rollback checks, representative performance evidence, dependency/API compatibility and target configuration readiness. Scale the gate to the changed risks. ATC can identify configured code issues but does not prove all business invariants or real integration behavior. Record unexecuted checks explicitly rather than converting missing evidence into a pass.

Example and follow-up A change touches an action signature and an approval event. Required evidence differs from a label-only UI change: include old consumer compatibility, duplicate delivery and negative authorization. Tie each check to the relevant source revision and environment.

Avoid this mistake A universal checklist is not a substitute for checking the risks actually changed.

Interviewer probes Which new failure mode did this change introduce? / Which result is pending rather than passed?

Strong answer criteria Builds a risk-based gate; Separates static and runtime evidence; Records versions and pending checks

Verification evidence Present a concise release matrix linking changed behavior to executed results, owners and remaining risks.

Read and verify this answer

SAP RAP test frameworks and boundaries

SAP RAP feature and release overview

OPS-15 A team advertises 95% coverage while never asserting FAILED or persisted state. How would you improve it?

Treat coverage as a reachability signal, not proof of correct behavior. Identify critical invariants and failure modes, then add assertions that would fail if those rules were broken. Cover rejected operations, message identity, transactional cleanup and unintended side effects. Avoid adding dummy executable code or tests that merely mirror assignments. Use coverage to locate blind spots after meaningful test design.

Example and follow-up A test executes a validation but never checks whether the invalid instance is rejected. Temporarily remove the rejection logic: if the test stays green, it does not prove the rule. Improve the assertion rather than increasing the number of similar happy-path calls.

Avoid this mistake High statement coverage can coexist with a broken business contract.

Interviewer probes Would the test fail if the rule were removed? / Which invariant is more important than the percentage?

Strong answer criteria Prioritizes observable behavior; Uses fault-sensitive assertions; Reports coverage without overstating evidence

Verification evidence Demonstrate tests that fail for deliberately broken rejection or persistence behavior, then restore the implementation.

Read and verify this answer

SAP RAP test frameworks and boundaries

SAP transactional buffer double support

OPS-16 How would you prepare an extension for a platform upgrade?

Inventory the actual used APIs, extension contracts, classifications and target-release changes. Prioritize critical business paths and known exceptions, then run focused regression and consumer tests in a representative target environment. Verify generated metadata, authorization, draft behavior, performance and required configuration where affected. A previously released API reduces a category of risk but does not remove the need to test your own assumptions and integration.

Example and follow-up The extension uses a classic dependency behind a wrapper and a released service consumed externally. Give each a named owner and an upgrade test strategy. A deprecated successor path may require consumer migration before the old dependency can be retired.

Avoid this mistake Do not treat last year's clean ATC report as evidence for the new target release.

Interviewer probes Which dependencies drive the upgrade risk? / What behavior did your code assume beyond the documented contract?

Strong answer criteria Uses an actual dependency inventory; Tests target-specific risks; Tracks exception and deprecation owners

Verification evidence Record target versions, dependency classification and regression results for the highest-impact flows.

Read and verify this answer

SAP RAP feature and release overview

OPS-17 A cloud job processes 500,000 records and times out. How would you redesign it?

Partition work into bounded, restartable units with stable business identities and checkpoints. Use supported background/application-job capabilities for the target environment rather than assuming classic task APIs are available. Control parallelism, database contention and remote limits. Define which units commit independently and how uncertain outcomes are reconciled. Progress reporting should distinguish processed, failed and pending items.

Example and follow-up A pricing recalculation finishes the first 20 partitions before failure. A restart should skip safely completed work and detect conflicting newer updates. Test a crash between the business effect and checkpoint handling so the design does not rely on an unproven exactly-once assumption.

Avoid this mistake Moving a huge synchronous loop into a background job does not solve its transaction or restart design.

Interviewer probes What is one independently recoverable unit? / How do newer user edits interact with retry?

Strong answer criteria Bounds work and contention; Defines durable restart semantics; Uses supported cloud execution APIs

Verification evidence Inject mid-partition failures and verify recovery, resource limits and final reconciliation.

Read and verify this answer

SAP application job catalog and template

OPS-18 How would you communicate senior readiness when you can review code locally but cannot run SAP checks?

Separate what was inspected, prepared and actually executed. A local source review can identify contract defects and propose tests, but cannot establish activation, real authorization, ATC results or measured SAP runtime behavior. Provide a precise target execution handoff with expected outcomes and dependencies. This is stronger evidence than inventing a pass or quoting performance numbers from a different environment.

Example and follow-up For a RAP approval change, supply positive/negative EML cases, a two-session concurrency script, role prerequisites and a service smoke test. State the relevant source revision and what must be returned from the target before claiming acceptance.

Avoid this mistake Do not present prepared test code or a mocked UI as successful SAP integration.

Interviewer probes Which claim can you prove now? / What exact target result would close the remaining risk?

Strong answer criteria Labels evidence honestly; Gives executable acceptance scenarios; Ties results to source and environment

Verification evidence Provide a prepared/executed/pending matrix with specific acceptance criteria and accountable owners.

Read and verify this answer

SAP RAP test frameworks and boundaries

Official references

SAP RAP test frameworks and boundaries

SAP transactional buffer double support

SAP CDS access-control testing

SAP SQL analysis in ADT

SAP RAP service binding

SAP RAP feature and release overview

SAP RAP business-object contract

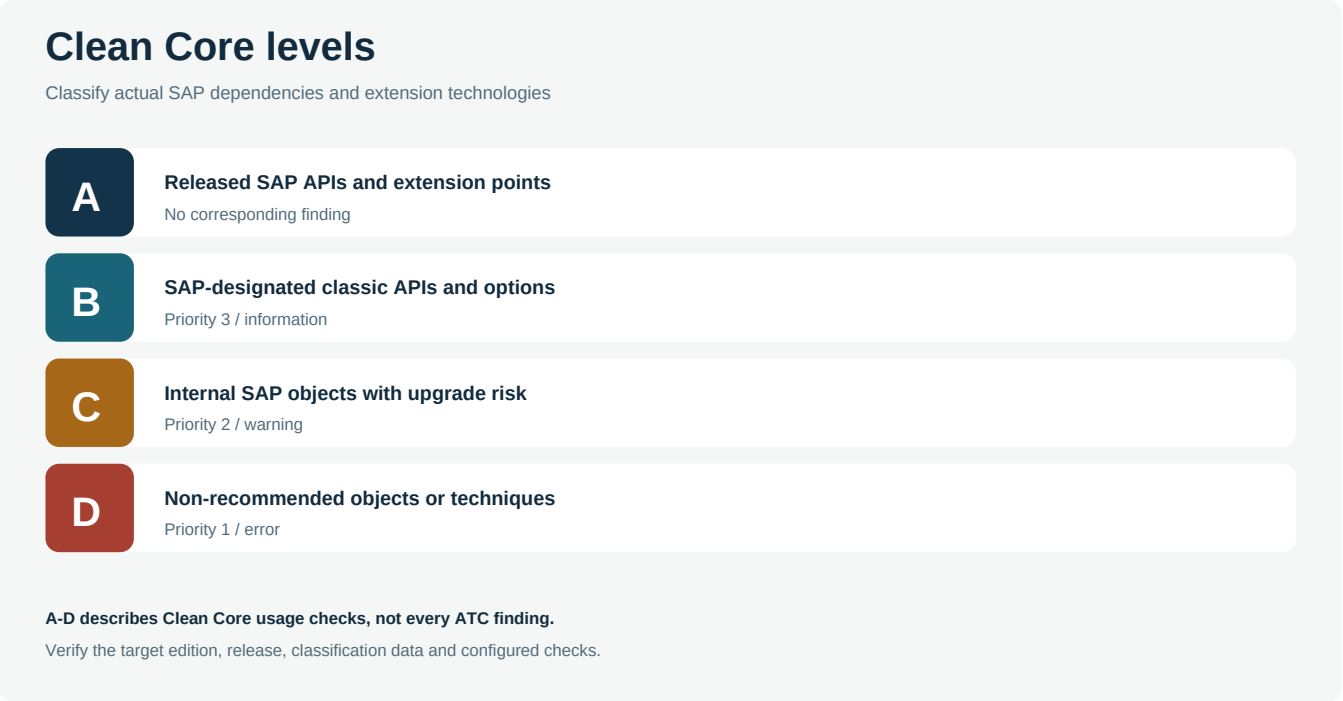
SAP total ETag and draft concurrency

SAP RAP side effects

SAP application job catalog and template

Clean Core levels and extensibility

Clean Core interviews test architectural judgment, not just memorization of A through D. Explain the target edition, actual API and extension technology, upgrade risk and governance. The levels complement the choice of key-user, on-stack or side-by-side extensibility.



Read each level as a classification of the actual SAP dependency or extension technology.

See the visual atlas for the recall prompt, limits and related questions.

CORE-01 What is Clean Core in a practical ABAP answer?

Keep extensions decoupled from SAP internals through supported contracts so business changes and upgrades remain manageable. Start with fit-to-standard, use suitable released interfaces and extension points, and govern exceptions and lifecycle costs. Extensibility is one dimension of Clean Core alongside concerns such as processes, data and integration.

Example and follow-up Explain a business requirement, why standard configuration was insufficient, which extension boundary you chose, and how you test upgrades. This is stronger than saying that all custom code must disappear.

Avoid this mistake Clean Core is not the same as Clean ABAP, zero custom code or moving everything to BTP.

Read and verify this answer

SAP Clean Core best practices

CORE-02 What are Clean Core levels A B C and D?

A uses released SAP interfaces and extension points with their supported contracts. B uses SAP-designated classic APIs and supported classic extension options. C depends on internal SAP objects with upgrade risk. D uses explicitly non-recommended objects or techniques. Always classify actual dependencies and technology in the applicable product context.

Example and follow-up Use A where feasible, consider B for justified classic scenarios, manage C with explicit risk ownership and monitoring, and prioritize remediation of D.

Avoid this mistake A familiar API is not automatically B, and every unreleased object is not automatically D.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-03 How do the levels map to ATC messages?

In the configured Clean Core usage checks, A produces no corresponding finding, B is priority 3 information, C is priority 2 warning and D is priority 1 error. This is the Clean Core mapping, not a universal interpretation of every ATC result from security or performance checks.

Example and follow-up Confirm the check variant, check implementation, classification data and target release before interpreting a result. A system with missing checks can appear green without proving compliance.

Avoid this mistake No ATC findings does not prove that the entire application is secure, correct or fully Level A.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-04 Why is a classic API Level B rather than Level A?

SAP may designate an established interface for classic use without releasing it under the ABAP Cloud contract. That nomination supports a lower-risk classic extension path, but it does not grant unrestricted consumption from ABAP for Cloud Development. Classification and language-version permissions are related but distinct.

Example and follow-up Look up the exact object and applicable use in SAP's classification data. A function module name beginning BAPI is not enough evidence.

Avoid this mistake Do not describe all classic APIs as public-cloud-ready or as formally C1-released.

Read and verify this answer

SAP Clean Core ATC levels and setup

SAP Cloudification Repository

CORE-05 What does responsible Level C use look like?

Level C acknowledges dependence on internals that may change. Identify the missing supported alternative, isolate the dependency, assign ownership, track SAP changes and test affected processes during upgrades. Keep a replacement plan. Organizational exception policy and the target product still govern whether the use is acceptable.

Example and follow-up A required internal read with no suitable alternative should have a named owner and regression coverage. Reassess when SAP releases a successor or changes the classification.

Avoid this mistake C is not an unlimited permission to call anything, nor is it the same risk category as a modification.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-06 What belongs to Level D and what would you do first?

Explicitly non-recommended objects and techniques, including problematic modifications or unsupported direct changes to SAP persistence, represent the highest-risk category. Confirm the actual classification and operation. Replace dangerous write paths or modifications with supported APIs or extension points, using regression tests and a staged rollout.

Example and follow-up For a direct UPDATE of an SAP business table, find the released or supported business API that performs checks and dependent updates rather than reproducing only the table assignment.

Avoid this mistake The fact that a direct update works in development does not establish a supported business transaction.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-07 Are A through D the same as the old three tiers?

No. The earlier model organized ABAP Cloud development, API enablement through wrappers and classic extensions into tiers. The newer levels classify dependency and technology choices more granularly, particularly for private edition and on-premise landscapes. Keep tiers as historical context, not the main current grading scheme.

Example and follow-up A classic extension can qualify at B rather than being dismissed solely for using classic ABAP. A wrapper can still contain a C or D dependency.

Avoid this mistake Do not map tier 1 to A, tier 2 to B and tier 3 to C mechanically.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-08 Does wrapping an unreleased SAP API make it Level A?

A released custom wrapper can provide a controlled interface to a consumer where the platform permits it. Its implementation still has the classification and upgrade risk of the underlying dependency. Evaluate both consumer contract and implementation chain, and retire the wrapper when an appropriate supported alternative exists.

Example and follow-up A wrapper around a nominated classic API can retain a B implementation dependency even when the calling ABAP Cloud code sees a released custom contract.

Avoid this mistake A wrapper is not a mechanism for laundering an internal or prohibited dependency into an entirely Level A solution.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-09 How do public private and on-premise editions change the answer?

Public-edition developer extensibility uses the supported ABAP Cloud model and released contracts. Private edition and on-premise support broader classic scenarios alongside ABAP Cloud, depending on release. The A-D model helps govern those existing dependencies; it does not unlock forbidden language or objects in a restricted environment.

Example and follow-up Start an interview architecture answer by asking which edition and release are in scope. ABAP Cloud is available beyond public cloud and BTP, so location alone is not the development model.

Avoid this mistake Private cloud does not mean that every classic customization is recommended, and ABAP Cloud does not mean only BTP.

Read and verify this answer

SAP Clean Core best practices

SAP ABAP Cloud FAQ

CORE-10 How do you choose key-user on-stack and side-by-side extensibility?

Choose key-user tools for supported business-context adaptations. Choose on-stack developer extensibility when close access to released data and transactional integration is required. Choose side-by-side for a more independent application lifecycle or cross-system process, using supported remote interfaces and explicit distributed consistency.

Example and follow-up An additional field with a supported business context may fit key-user tools. A synchronous local validation may fit a released BAdI. A cross-system approval portal may fit BTP.

Avoid this mistake These are placement and development choices, not automatic A-D grades or fixed implementation times.

Read and verify this answer

SAP Clean Core best practices

CORE-11 Why is side-by-side not automatically Clean Core?

An external application still depends on its integration contracts. Unsupported table access, fragile endpoints or duplicated core business logic can retain upgrade and lifecycle risk even when the code runs on BTP. Decoupling also requires security, observability, versioning and consistency design.

Example and follow-up A remote extension using a stable business API and idempotent events can evolve independently within that contract. It still needs regression tests when the API or business semantics change.

Avoid this mistake Moving custom code out of the ERP server does not erase coupling.

Read and verify this answer

SAP Clean Core best practices

CORE-12 What are release contracts C0 C1 and C2?

In the ABAP release-contract vocabulary, C0 concerns extension, C1 system-internal use and C2 remote use. Check release state, contract and use visibility for the exact object and scenario. A contract describes supported consumption; it does not replace business authorization or establish that every needed operation is available.

Example and follow-up An object suitable for local ABAP consumption is not automatically a remote API. Conversely an HTTP API does not necessarily expose a released local ABAP implementation class.

Avoid this mistake A public class or an I_ CDS name is not proof of a released API contract.

Read and verify this answer

SAP API release contracts

CORE-13 How do you verify an API choice rather than guess?

Check official API discovery and documentation, the target repository's API state and release visibility, and SAP's applicable Cloudification Repository and technology guidance. Confirm the required operation and product version. Use ATC against maintained classification data to validate actual usage.

Example and follow-up Verify a proposed write operation, not merely the existence of a similarly named read CDS view. Record the successor and any restrictions alongside the architecture decision.

Avoid this mistake A search result, object prefix or successful syntax check alone does not establish support for the intended usage.

Read and verify this answer

SAP Clean Core best practices

CORE-14 Can a solution using both A and B dependencies be called entirely A?

No. The lower-grade dependency matters when assessing the containing development and its upgrade risk. Preserve object-level findings and dependency context rather than averaging them away. An A-compliant consumer and a B wrapper are useful separate facts, but they do not remove the B dependency from the solution.

Example and follow-up Report the distribution of findings and critical dependencies. Prioritize remediation by business impact, exposure and cost, not only the number of objects.

Avoid this mistake Percentages of Level A objects should not hide one critical D write path.

Read and verify this answer

SAP Clean Core best practices

CORE-15 What if no released API covers the business requirement?

Recheck standard functionality and supported extension points, confirm the target release, and evaluate another placement or process design. In a landscape allowing classic development, inspect designated classic APIs and govern any remaining internal dependency. Request a supported API through the appropriate SAP channel when needed.

Example and follow-up Document the missing operation, alternatives rejected, risk owner and exit criteria. A temporary exception should have an explicit review point when a successor appears.

Avoid this mistake Do not immediately choose a modification, and do not assume public edition permits a classic workaround.

Read and verify this answer

SAP Clean Core best practices

CORE-16 What would your Clean Core governance pipeline contain?

Maintain a system-specific ATC variant with API-usage and enhancement-technology checks plus appropriate quality and security checks. Review findings, own exceptions with expiry or review conditions, and track remediation. Recheck against target-upgrade information and real regression tests. Automated checks support architecture judgment.

Example and follow-up A transport policy may block selected priorities while allowing reviewed exceptions. Explain how exemptions are approved and revisited rather than silently suppressing all findings.

Avoid this mistake A priority number alone is not a complete technical-debt decision or a substitute for runtime tests.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-17 Is Clean ABAP the same thing as Clean Core?

Clean ABAP concerns understandable and maintainable implementation: naming, responsibilities, interfaces, error handling and tests. Clean Core concerns extension boundaries, supported dependencies and lifecycle stability. They reinforce one another, but a beautifully written class can still modify SAP internals and create high upgrade risk.

Example and follow-up Review a class twice: first its dependencies and extension mechanism, then its code quality and tests. Both dimensions belong in a strong interview example.

Avoid this mistake Refactoring variable names does not change the Clean Core level of an unsupported dependency.

Read and verify this answer

SAP Clean Core best practices

CORE-18 How would you modernize a large custom-code portfolio?

Start with usage and business value, retire genuinely unused code after validation, then classify dependencies and address the highest-risk active objects. Adopt supported APIs and suitable extension patterns incrementally. Measure regressions and upgrade effort, and avoid rebuilding every classic report solely for stylistic uniformity.

Example and follow-up Select one important D dependency, establish baseline behavior, replace it behind a clear interface and verify the end-to-end process. Repeat with evidence rather than promising a universal automatic conversion.

Avoid this mistake RAP is not a mandatory replacement for every report, interface or classic application.

Read and verify this answer

SAP Clean Core best practices

Official references

SAP Clean Core best practices

SAP Clean Core ATC levels and setup

SAP Cloudification Repository

SAP ABAP Cloud FAQ

SAP TechEd Clean Core exercise

SAP API release contracts

Senior Clean Core governance and tradeoffs

Clean Core decisions require evidence about dependencies, supported extension contracts, business impact, and ownership. These scenarios use A-D as extensibility classifications while keeping API release contracts and ATC severities separate. A senior answer should make uncertainty visible, defend a practical decision, and define how the organization will detect and reduce the remaining risk.

Clean Core levels

Classify actual SAP dependencies and extension technologies

A	**Released SAP APIs and extension points** No corresponding finding
B	**SAP-designated classic APIs and options** Priority 3 / information
C	**Internal SAP objects with upgrade risk** Priority 2 / warning
D	**Non-recommended objects or techniques** Priority 1 / error

A-D describes Clean Core usage checks, not every ATC finding.

Verify the target edition, release, classification data and configured checks.

Read each level as a classification of the actual SAP dependency or extension technology.

See the visual atlas for the recall prompt, limits and related questions.

GOV-01 A project claims Level A because its frontend uses RAP. How would you assess the extension?

I would define the assessment scope and inspect the actual dependency and extension paths, including adapters and called custom code. Level A relies on released contracts and the relevant supported development approach; Level B uses classified classic APIs, Level C internal objects, and Level D non-recommended dependencies or techniques. RAP or a Fiori frontend does not determine the classification alone. I would record object-level evidence and avoid presenting a mixed solution as uniformly Level A.

Example and follow-up A RAP handler may call a customer wrapper that ultimately invokes an internal SAP object. The consumer boundary and wrapper can require separate assessment, but the delivery still contains that exposure. A useful report links each finding to the affected business capability and responsible owner, rather than counting only visible service artifacts.

Avoid this mistake RAP, BTP deployment, a customer namespace, or a green UI test is not proof that every underlying dependency is Level A.

Interviewer probes What exactly is included in the claimed Level A scope? / How would you find dependencies hidden behind customer adapters?

Strong answer criteria Defines scope before classification; Traces indirect dependencies; Separates framework choice from API evidence

Verification evidence An object-level dependency inventory with target-system release/classification evidence and the applicable ATC result for the same revision.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-02 An API is released with C1. A team assumes that means Clean Core Level C and permission to expose it remotely. How do you respond?

I would separate the naming systems. C0 is the Extend release contract, C1 is Use System-Internally, and C2 is Use as Remote API; these are purposes and stability contracts, not Clean Core A-D categories. A C1 release must also have the appropriate visibility for its consumer. Remote exposure needs a supported remote contract and communication design. I would inspect the object's API state and documented use rather than infer permissions from the letter C.

Example and follow-up A local released class can support an on-stack application without being a remote API itself. Building a custom remote service around it creates a new owned contract with authentication, authorization, error semantics, and lifecycle obligations. It does not magically convert the underlying object to C2 or inherit every provider guarantee.

Avoid this mistake C0/C1/C2 and Clean Core A/B/C/D are different classifications; release for one purpose does not release every possible usage.

Interviewer probes Which visibility must the API expose for this caller? / What responsibilities arise if you build a custom remote service over a local API?

Strong answer criteria Names contracts accurately; Checks purpose and visibility; Recognizes new provider responsibilities

Verification evidence ADT API-state properties, provider documentation, and a consumer design showing local use or an explicitly owned remote contract.

Read and verify this answer

SAP release contracts, visibility, and API lifecycle states

GOV-03 A team wraps an internal SAP function module and releases the wrapper. Has the technical debt disappeared?

No. A wrapper can isolate callers, translate contracts, and create a replaceable boundary, but it does not change the release status or upgrade exposure of its underlying dependency. I would record both the consumer's contract and the wrapper's implementation risk. Where classic boundaries are allowed, the wrapper needs an owner, regression tests, target-release assessment, and an exit plan. I would also question whether the wrapper adds useful isolation or merely renames the same unstable interface.

Example and follow-up If the wrapper publishes the internal function's structures unchanged, a provider change can still propagate directly to every consumer. A narrower application contract can contain that change. SAP's current level guidance includes wrappers over classified classic APIs in Level B scope; wrapping an internal object does not make that dependency classified classic.

Avoid this mistake A customer release flag does not turn an unreleased SAP dependency into a released SAP API or erase its classification.

Interviewer probes Which unstable details must stay behind the wrapper? / Who replaces the dependency when a released alternative becomes available?

Strong answer criteria Tracks underlying exposure; Designs a narrow stable boundary; Assigns replacement ownership

Verification evidence A wrapper dependency record, contract tests, an owner, and a documented replacement trigger with the current target-release assessment.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-04 An urgent release needs an ATC exemption for an internal dependency. What makes the exception governable?

I would require a specific business need, exact finding and source scope, target edition, alternatives considered, risk owner, and compensating verification. The exception should have an expiry or review trigger and a funded remediation path, not an indefinite blanket waiver. An ATC exemption changes handling of a finding; it does not make unsupported syntax legal, change SAP's API contract, or establish business correctness. The release decision must explicitly accept the residual exposure.

Example and follow-up An exception for one read-only adapter should not silently cover additional write calls introduced later. Tie it to the relevant objects and finding category, rerun checks after changes, and require review when the target release or provider changes. Security and transactional defects need their own assessment rather than being bundled into a clean-core waiver.

Avoid this mistake An approved exemption is not a reclassification to Level A and must not become a general permission to bypass ABAP Cloud restrictions.

Interviewer probes What event invalidates this exception? / How do you prevent the approved scope from growing unnoticed?

Strong answer criteria Bounds scope and lifetime; Names accountable risk ownership; Separates exemption from technical permission

Verification evidence An exception record linked to the exact ATC finding, source revision, release target, regression evidence, owner, and expiry or review trigger.

Read and verify this answer

SAP ATC configuration and transport-release governance

GOV-05 A Level B solution works in Private Edition. Can the same approval justify deploying it in Public Edition?

No. Clean Core classification does not override the target product's available development model, APIs, or extension mechanisms. I would reassess the requirement against Public Edition's supported configuration, key-user and developer extensibility, and remote integration options. A classified classic API acceptable in a private-edition context may not be available there. The governance record should describe that portability gap openly and compare a supported redesign with retaining the current deployment for a justified business reason.

Example and follow-up A classic enhancement or SAP GUI-dependent component may have a stable role in an existing private-edition application but no permitted equivalent in the proposed public-cloud scope. Moving only its UI to BTP does not restore access to an unavailable backend API. Validate the actual business operation end to end.

Avoid this mistake Level B is not a universal deployment permission, and Level A design does not guarantee identical API coverage across editions.

Interviewer probes Which part of the solution is genuinely portable? / What evidence would justify keeping a private-edition dependency?

Strong answer criteria Checks target-edition constraints; Separates classification from portability; Offers supported redesign options

Verification evidence An edition capability comparison with verified target APIs, extension points, and a documented functional gap for each nonportable component.

Read and verify this answer

SAP extensibility options by environment and language version

GOV-06 You have thousands of custom objects and a limited upgrade budget. How would you prioritize remediation?

I would combine usage evidence, business criticality, dependency classification, known target-release changes, and remediation effort. Unused objects may be retirement candidates after validating completeness of the observation window. High-impact Level D or changed internal dependencies usually warrant early investigation, while stable low-risk dependencies can follow a planned backlog. Object counts alone are misleading: one heavily used posting dependency can matter more than hundreds of harmless utilities. Each prioritized item needs an owner and acceptance criteria.

Example and follow-up A year-end report can look unused during a three-month trace window. Deleting it based only on that sample would create a business outage. Conversely, a rarely executed emergency posting tool may still have high operational criticality. Combine telemetry with process-owner confirmation and target-upgrade impact analysis.

Avoid this mistake Do not equate low execution frequency with low business importance or treat every finding as equal remediation work.

Interviewer probes How would you avoid retiring seasonal functionality? / What makes one Level C dependency more urgent than another?

Strong answer criteria Combines usage and business impact; Uses target-release change evidence; Prioritizes actionable owned work

Verification evidence A ranked remediation portfolio with usage-window limitations, business-owner decisions, dependency impact, estimated effort, and target-release tests.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-07 No released API supports a required business operation. What would your escalation contain?

I would describe the business outcome, target product and release, required operation and transaction semantics, expected volume, and the precise missing capability. I would document searched APIs and extension points, explain why standard configuration or a remote alternative does not fit, and provide a minimal reproducible example where useful. The escalation should support an actionable provider decision while the project evaluates a bounded process change, delayed scope, or an edition-appropriate governed alternative.

Example and follow-up Saying 'please release this table' may prescribe a fragile implementation rather than explain the need. A request for a supported command to close a document, including authorization, validation, and failure semantics, gives the provider a clearer requirement. Any interim workaround still needs its own risk and retirement record.

Avoid this mistake An API-gap ticket does not authorize direct table access or promise that the requested API will arrive by the project's deadline.

Interviewer probes What evidence shows that existing APIs really do not fit? / How do you keep delivery moving without hiding the gap?

Strong answer criteria Escalates a business capability; Documents rejected supported alternatives; Defines an explicit interim plan

Verification evidence A capability-gap dossier with release-specific API searches, required semantics, a reproducible scenario, and a decision on interim scope.

Read and verify this answer

SAP extensibility options by environment and language version

GOV-08 The business asks to reproduce a heavily customized legacy process exactly. How do you challenge that without dismissing the requirement?

I would separate mandatory business outcomes from historical implementation choices, then demonstrate how standard configuration and supported extensions meet each outcome. Legal, contractual, and differentiating requirements deserve explicit evidence; familiar screen sequences alone may not. I would quantify the cost of preserving deviations, including upgrade tests and operations, and compare a smaller supported extension or process adjustment. The business owner should decide with concrete examples of retained value, changed behavior, and residual risk.

Example and follow-up A legacy approval screen might combine fields that standard applications split across steps, while the actual control requirement is segregation of duties and an auditable decision. Recreating the old screen through internal hooks can add risk without preserving additional value. A process walkthrough should expose exceptions, not only the happy path.

Avoid this mistake Fit-to-standard is not a slogan for refusing real requirements, and exact visual parity is not always the real business invariant.

Interviewer probes Which requirement is mandatory and who can prove it? / What exception case could invalidate the proposed standard process?

Strong answer criteria Separates outcomes from historical design; Uses concrete process demonstrations; Makes cost and ownership visible

Verification evidence A fit-gap matrix with business-owner acceptance, exception-path demonstrations, and lifecycle cost estimates for retained deviations.

Read and verify this answer

SAP Clean Core dimensions beyond extensions

GOV-09 Your team wants to release a custom API used by three other teams. What must be ready before setting its release state?

I would require a deliberate public surface, supported use cases, error and transaction semantics, authorization expectations, and a named owner prepared to maintain compatibility. Consumers need stable types and behavior, not leaked persistence details. I would capture a delivered API baseline, configure applicable compatibility checks, and define deprecation and successor policies. If the interface is still experimental and coordinated internal reuse is sufficient, a supported component-access arrangement may be more honest than a premature stability promise.

Example and follow-up Removing an optional field or changing a rounding rule can break a consumer even if the developer considers it cleanup. Compatibility snapshots help detect relevant technical changes, while consumer tests protect business semantics. Do not regenerate the baseline merely to silence findings from an incompatible change to a delivered contract.

Avoid this mistake Releasing an API is a provider commitment; broad access permission is not equivalent to a stable contract.

Interviewer probes Which changes can be detected statically and which need consumer tests? / How would you deprecate the API without breaking current users?

Strong answer criteria Defines provider obligations; Protects delivered baselines; Combines technical and semantic checks

Verification evidence An API specification, owner and consumer list, prior-delivery snapshot, compatibility results, and consumer-contract test evidence.

Read and verify this answer

SAP release contracts, visibility, and API lifecycle states

SAP software component dependencies and compatibility snapshots

GOV-10 The direct dependency scan is clean, but an upgrade still breaks the extension. What could the governance model be missing?

I would inspect indirect custom dependencies, dynamic calls, configuration-selected implementations, remote contracts, and data assumptions outside the scanned scope. A scan can also miss relevant objects if its package selection, check version, or classification data is incomplete. I would connect findings to a dependency graph and process-level tests, recording evidence limitations. Clean direct references are useful, but they do not establish that every runtime path or semantic assumption is protected against the target change.

Example and follow-up A custom facade may select an internal implementation through configuration, or a remote service may change independently of the ABAP transport. Another failure can come from expecting one text row where new data produces several. The graph must include configuration and integration edges, with accountable owners and representative runtime coverage.

Avoid this mistake A green direct-reference check is not proof that the whole solution has no unsupported or semantically fragile dependencies.

Interviewer probes How would you discover configuration-driven dependencies? / Which runtime paths must the upgrade rehearsal cover?

Strong answer criteria Explores indirect and dynamic edges; Questions scan completeness; Adds process-level verification

Verification evidence An expanded dependency graph, documented scan scope and tool versions, and upgrade regression evidence covering configured runtime paths.

Read and verify this answer

SAP software component relations and consistency checks

GOV-11 How would you enforce Clean Core in delivery without blocking every legacy transport immediately?

I would establish a trustworthy baseline, prevent new unapproved exposure, and prioritize remediation of existing risk through owned work. Gates should use the applicable language checks, API and enhancement checks, compatibility checks, and security or correctness tests for the target environment. Existing exceptions need bounded review rather than silent suppression. I would record the variant, priorities, scope, and baseline policy so a green gate has a reproducible meaning and developers receive feedback before release day.

Example and follow-up A transport that changes a legacy report's layout should not automatically trigger an unplanned rewrite of the entire application, but it also must not introduce a new internal write dependency unnoticed. Separate unchanged baseline exposure from new findings, then require review when an edit changes the risk of an existing exception.

Avoid this mistake A baseline is not approval for unlimited new debt, and transport blocking thresholds are configured policy rather than a universal property of every ATC check.

Interviewer probes What happens when a touched legacy dependency becomes riskier? / How do you stop teams from weakening the shared check variant?

Strong answer criteria Distinguishes baseline from new risk; Uses reproducible early gates; Governs exception and policy changes

Verification evidence A versioned gate policy, representative pass/fail cases, a protected check-configuration record, and a baseline-to-current findings comparison.

Read and verify this answer

SAP ATC configuration and transport-release governance

GOV-12 A vendor upgrade changes an object used by an approved Level C extension. How do you organize the response?

I would identify affected consumers and business processes, compare the old and new provider behavior, and determine whether a released alternative now exists. The approved exception should trigger reassessment for this target release. I would reproduce the failure in an upgrade environment, adapt the isolated boundary if still justified, and run focused integration and regression tests. Delivery must coordinate vendor and customer changes, with rollback or business recovery defined for persistence or contract changes.

Example and follow-up An internal object's unchanged signature can conceal different defaulting or validation behavior. Conversely, an activation failure may reveal a straightforward type change. Both require more than a syntax fix: test postings, authorizations, messages, and affected data transitions against the intended business outcome, using the actual upgraded system revision.

Avoid this mistake A previous exception or successful prior upgrade does not prove compatibility with the next vendor release.

Interviewer probes How do you distinguish an API change from a data-dependent defect? / Would you keep the exception if a suitable released successor is now available?

Strong answer criteria Reassesses release-specific exposure; Tests semantic behavior; Coordinates recovery and replacement

Verification evidence A target-upgrade impact record, reproduced failure, reviewed boundary change, and process regression results tied to both vendor and custom revisions.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-13 What would a credible two-year custom-code modernization roadmap look like?

I would organize the roadmap by business capability and lifecycle risk, with early retirement of confirmed unused functionality and focused replacement of high-exposure dependencies. New development should follow supported ABAP Cloud contracts where suitable, while valuable classic capabilities can coexist under explicit governance. Each phase needs measurable outcomes, owners, dependencies, and operational acceptance. The roadmap must reserve capacity for API changes and upgrades rather than promise that every object will be rewritten in RAP by a fixed date.

Example and follow-up One phase might remove an unused interface, another replace internal reads with a released query, and another redesign a transaction that owns an unstable posting path. These changes produce different benefits and require different tests. Counting converted classes alone can reward cosmetic work while leaving the highest-risk dependency untouched.

Avoid this mistake Modernization is not measured reliably by RAP object count or by replacing all classic code regardless of business value.

Interviewer probes What quarterly outcome would demonstrate reduced upgrade risk? / How do you fund remediation that has no visible new UI feature?

Strong answer criteria Prioritizes capability outcomes; Preserves justified coexistence; Uses measurable risk reduction

Verification evidence A capability-based roadmap with owners, usage decisions, target dependency reductions, migration acceptance criteria, and budgeted upgrade work.

Read and verify this answer

SAP Clean Core dimensions beyond extensions

GOV-14 A classified classic API meets the need, while a released API covers only half the operation. Would you accept Level B?

In a target environment that permits the classic approach, I would compare the supported business completeness and lifecycle risk of both options. A properly classified Level B API can be a defensible choice; it is not automatically a defect to rewrite. I would verify that the complete dependency path and extension technique qualify, avoid misleading Level A claims, and preserve a migration option. The decision must not override Public Edition or ABAP Cloud language restrictions.

Example and follow-up Splitting one business operation across an incomplete released API and direct table updates can be worse than using a complete classified classic business API. The comparison should include transaction integrity, supported fields, authorization, maintainability, and future product direction. Confirm classification of the specific object instead of generalizing from the word BAPI.

Avoid this mistake Not every old API is Level B, and a Level B assessment does not make it directly callable from restricted ABAP Cloud code.

Interviewer probes What makes the exact API qualify as classic rather than internal? / What would cause you to revisit this decision after an upgrade?

Strong answer criteria Makes an evidence-based B decision; Preserves transaction completeness; Respects target and language constraints

Verification evidence Specific object-classification evidence, a complete operation test, a comparison with released alternatives, and a scheduled reassessment trigger.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-15 A required report uses an internal read object with no current replacement. Would you accept Level C, and under what conditions?

I would first verify the actual gap, data semantics, authorization, and deployment constraints. If a permitted Level C dependency remains necessary, I would isolate its use, limit the public contract, record the target-release exposure, and assign an owner for provider changes and replacement. Read-only access lowers some transaction risks but does not eliminate confidentiality, correctness, or upgrade risk. Acceptance should be specific, time-bounded or trigger-bound, and supported by representative result and authorization tests.

Example and follow-up An internal view may change filtering or joins without causing a syntax error, yielding duplicate rows or exposing unintended data. A regression fixture should check business totals, row identity, and denied access, not merely successful execution. Where release-specific change information is available, use it as an input rather than a complete compatibility guarantee.

Avoid this mistake Read-only does not mean risk-free, and Level C is not a blanket exemption for all internal objects or non-recommended Level D techniques.

Interviewer probes What result change could escape an activation check? / Which precise event forces this exception back into review?

Strong answer criteria Treats read semantics as consequential; Isolates and owns exposure; Defines focused compatibility evidence

Verification evidence A bounded exception with dependency classification, authorization and result baselines, target-upgrade checks, and replacement-review triggers.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-16 How would you assess a solution that uses a released CDS query, a classified classic BAPI, an internal class, and a modification?

I would assess the four paths separately and confirm their actual usage and applicable classifications on the target system. The released query can support Level A use when consumed under the right contract; the classified classic API introduces Level B scope, the internal dependency Level C scope, and a non-recommended modification can introduce Level D exposure. I would not average these into a reassuring score. I would identify affected capabilities, alternatives, criticality, and the highest-risk remediation first.

Example and follow-up The modification may be in an unused historical path, while the internal class supports daily financial posting. Both facts matter to sequencing, but neither changes the factual dependency record. Removal requires usage validation and process-owner approval; replacing a dependency requires proving the same intended business operation and error behavior.

Avoid this mistake A single letter for a mixed solution can hide scope; classification and remediation priority are related but are not the same decision.

Interviewer probes Which capability would you remediate first and why? / What evidence is required before removing the modification?

Strong answer criteria Classifies paths rather than averages; Separates exposure from priority; Uses business and usage evidence

Verification evidence A scoped assessment table linking each dependency to classification evidence, process impact, usage, ownership, and a justified remediation decision.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-17 The sponsor says Clean Core is a developer concern and rejects funding for integration monitoring and data cleanup. How would you negotiate?

I would connect the proposal to operational and business outcomes: failed interfaces, unreliable master data, unsupported software, and complex process variants can prevent safe upgrades even when custom code is well structured. Clean Core includes more than extension code. I would present concrete incidents or scenario costs, propose a minimum owned control set, and distinguish mandatory controls from improvements that can be phased. The sponsor should explicitly decide which residual business risks remain and who operates them.

Example and follow-up A technically released order API still fails repeatedly when customer mappings are invalid and nobody owns the error queue. Developers cannot solve that through an ATC exemption or another wrapper. A practical agreement assigns mapping ownership, alert response, reconciliation, and cleanup priorities with measurable service expectations.

Avoid this mistake A strong extensibility classification does not certify clean data, standard processes, secure integrations, or effective operations.

Interviewer probes Which control delivers the highest near-term business value? / Who is accountable when an interface remains technically successful but business processing is incomplete?

Strong answer criteria Connects non-code controls to outcomes; Negotiates proportional phased scope; Assigns business and operational ownership

Verification evidence A responsibility matrix, costed minimum control plan, measurable operational targets, and documented acceptance of deferred risks.

Read and verify this answer

SAP Clean Core dimensions beyond extensions

GOV-18 An interview panel shows an A-D chart with ATC priorities. What would you clarify before treating it as a release gate?

I would explain the chart's intended check context, then ask which variant, check implementations, priorities, classification data, object scope, and target release produced the findings. A mapping used by an API-usage check is not a universal meaning of every ATC priority. Other findings can concern security, correctness, or performance. A clean-core gate should preserve that distinction and state what a pass proves, what exemptions exist, and which runtime evidence remains necessary.

Example and follow-up Two projects can show different findings because one omitted an enhancement check or used older classification data, not because its code is cleaner. Even a matching configuration cannot prove business transaction correctness. Save the run metadata and compare like-for-like scopes before using counts or severity trends in a management dashboard.

Avoid this mistake Do not interpret every priority 1 finding as Level D, every priority 2 as Level C, or absence of findings as proof of complete Level A coverage.

Interviewer probes What would you store so this assessment can be reproduced? / How would you distinguish an exempted finding from a resolved dependency?

Strong answer criteria Qualifies severity mapping by check; Requires reproducible assessment scope; Separates static findings from runtime proof

Verification evidence The exact ATC variant and check configuration, classification/version information, source and scope identifiers, exemptions, and complementary runtime validation results.

Read and verify this answer

SAP ADT release notes: ABAP_CLEAN_CORE_READINESS check variant

Official references

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

SAP release contracts, visibility, and API lifecycle states

SAP software component dependencies and compatibility snapshots

SAP software component relations and consistency checks

SAP ATC configuration and transport-release governance

SAP ADT release notes: ABAP_CLEAN_CORE_READINESS check variant

SAP extensibility options by environment and language version

SAP Clean Core dimensions beyond extensions

ABAP foundations

Practice explaining data structures and transactional correctness before discussing frameworks. These answers distinguish internal memory, database persistence, and application locks. Classic update-task examples apply to supported Standard ABAP environments; ABAP Cloud and RAP impose additional API and transaction restrictions.

Find key 50: three internal-table access paths

The same seven records; choose the table and key for the actual access pattern.

STANDARD

Linear free-key example

Row	Key	Check
1	30	1
2	70	2
3	10	3
4	20	4
5	50	match
6	40	
7	60	

Inspect rows until the match.

SORTED

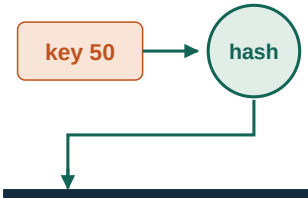
Ordered key lookup

Row	Key	Probe
1	10	
2	20	
3	30	
4	40	1st
5	50	3rd
6	60	2nd
7	70	

Illustrative probes: 40, 60, 50.

HASHED

Complete unique key



Bucket	Record key
...	...
h(50)	50 [match]
...	...

Bucket route is conceptual.
No primary index access.

Also consider secondary keys, inserts, memory, and range access.

Not a timing benchmark: sorted probes and hash buckets illustrate the lookup idea.

A free-key linear search, an ordered lookup, and a complete hashed-key lookup find the same record through different access paths.

See the visual atlas for the recall prompt, limits and related questions.

ABAP-01 How do you choose between standard, sorted, and hashed internal tables?

I choose from the access pattern. A standard table suits indexed access and sequential processing. A sorted table maintains key order and supports efficient key and range access. A hashed table suits repeated complete-key lookup and requires a unique primary key. I also consider write volume, memory, and secondary keys before choosing.

Example and follow-up For an order import, lt_orders can be a standard table when input order matters. A hashed lookup by unique order ID avoids repeated linear scans. A sorted table by customer and order ID supports processing each customer's contiguous group.

Avoid this mistake A hashed primary key is unique; that does not mean every field value must be unique or that arbitrary partial-key searches are constant time.

Read and verify this answer

SAP internal table reference and examples

ABAP-02 What are primary, secondary, default, and empty internal table keys?

The primary key belongs to the table definition. Secondary keys add alternative sorted or hashed access paths, with memory and maintenance costs. The default key is derived from the row type and may omit fields I expected to matter. For an index-oriented standard table without key semantics, I explicitly use an empty key.

Example and follow-up A standard table can preserve arrival order and still expose a sorted secondary key for customer access. I name the secondary key in the access when needed, and measure whether its repeated-read benefit justifies the overhead of frequent inserts or changes.

Avoid this mistake An internal table key is not a database index, and an empty key does not remove the standard table's index.

Read and verify this answer

SAP internal table reference and examples

ABAP-03 When is BINARY SEARCH correct, and how do table expressions handle missing rows?

Binary search on a standard table requires ascending order compatible with the search fields. If code can break that order, a sorted table is safer. A table expression normally raises CX_SY_ITAB_LINE_NOT_FOUND for a missing row. I use line_exists for existence only, or handle absence explicitly when retrieving data.

Example and follow-up If absence means an optional description, VALUE with OPTIONAL may be appropriate. If an order must exist, returning an initial structure would hide a defect. When I need the row, I avoid an existence check followed by a second identical lookup.

Avoid this mistake Adding BINARY SEARCH to an unsorted table can return incorrect results; modern table-expression syntax does not automatically optimize an unsuitable key.

Read and verify this answer

SAP internal table reference and examples

ABAP-04 How do a work area, field symbol, and data reference differ?

Reading into a work area normally gives me a value to process independently. A field symbol aliases an assigned data object, so changing a table line through it changes that line directly. A data reference stores a reference to a data object and is dereferenced explicitly. I choose the mechanism for clarity and required mutation.

Example and follow-up LOOP AT lt_orders ASSIGNING FIELD-SYMBOL(<ls_order>) is useful for changing non-key attributes in place. With INTO ls_order, changing ls_order alone does not update the table. I check assignment or binding before accessing optional aliases and references.

Avoid this mistake Do not change protected table-key components through an alias; key restrictions depend on table category and the access in progress.

Read and verify this answer

SAP internal table reference and examples

SAP ABAP Keyword Documentation

ABAP-05 What is the difference between a domain, data element, structure, and table type?

A domain describes technical value properties such as type, length, and possible fixed values. A data element adds semantic meaning, field labels, and documentation. A structure groups components without defining persistent storage. A table type describes an internal table's row type, category, and keys. A database table is a separate persistence object.

Example and follow-up Two fields can share a technical domain while using different data elements because their business meanings and labels differ. A reusable table type might describe a collection of request items, but declaring that type does not create database records.

Avoid this mistake A DDIC structure is not a database table, and domain fixed values are not a universal substitute for application validation.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-06 How do a foreign key, check table, and value table differ in classic DDIC?

A foreign key models a relationship between fields and a check table. It supports Dictionary semantics and applicable input checks, but I do not assume it creates a database-enforced constraint. A domain's value table is a proposal for defining such a relationship; defining the value table alone does not install the full check.

Example and follow-up A country field can reference the country customizing table through a defined foreign-key relationship. An arbitrary programmatic database write must still preserve business validity. I also distinguish a foreign key from a secondary database index, which is an access structure.

Avoid this mistake Foreign key does not mean secondary key or secondary index, and a value table alone does not guarantee valid database content.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-07 How do you explain an SAP LUW versus a database LUW?

A database LUW is the database transaction completed by a database commit or rollback. An SAP LUW groups a business change across application processing, potentially spanning multiple database LUWs in classic dialog processing. The application coordinates consistency using appropriate persistence, update mechanisms, and logical locks. The transaction owner decides when the business operation is complete.

Example and follow-up In a classic document change, user interaction can span dialog steps. Holding ordinary database locks across that entire interaction is unsuitable, so the application uses SAP enqueue locks and a defined save process. RAP instead provides a framework-controlled transaction and save sequence.

Avoid this mistake A database commit caused by a work-process boundary is not permission to split a business operation into independently committed fragments.

Read and verify this answer

SAP LUW reference and examples

ABAP-08 Who should issue COMMIT WORK, and what does AND WAIT guarantee?

The transaction owner should commit, rather than a reusable helper committing unexpectedly. In classic update-task processing, COMMIT WORK triggers registered update processing. AND WAIT waits for high-priority update completion: sy-subrc is 0 on success and 4 if that update fails. Without AND WAIT, COMMIT WORK sets sy-subrc to 0 without reporting the asynchronous update outcome. Neither form guarantees that every asynchronous follow-up in the landscape has completed successfully.

Example and follow-up A reusable posting service called within a larger business operation must respect the caller's transaction contract. In RAP behavior code I do not insert COMMIT WORK or BAPI_TRANSACTION_COMMIT; framework restrictions and the actual legacy API's commit behavior must be checked.

Avoid this mistake COMMIT WORK AND WAIT is not a distributed transaction guarantee and does not authorize commits inside RAP handlers.

Read and verify this answer

SAP LUW reference and examples

SAP: COMMIT WORK and update result

ABAP-09 Why do SAP enqueue locks exist in addition to database locks?

SAP enqueue locks express logical ownership of a business object across application processing. Database locks protect database operations within database transactions. Applications cooperating on the same object must use a consistent enqueue protocol, handle conflicts, and release or transfer ownership at the correct point. Lock scope and update ownership matter in classic transactions.

Example and follow-up For a document edit, acquire the appropriate object lock, read consistent state, validate, and save under the intended transaction protocol. Handle user cancellation and error paths as carefully as success. Classic diagnosis can include SM12; cloud applications use supported tooling and APIs.

Avoid this mistake An enqueue lock does not physically prevent unrelated SQL code from updating a table if that code ignores the locking protocol.

Read and verify this answer

SAP LUW reference and examples

ABAP-10 How do you choose and propagate class-based exceptions?

I use CX_STATIC_CHECK for an explicit recoverable failure contract, CX_DYNAMIC_CHECK when the API intentionally relies on runtime propagation rules, and CX_NO_CHECK for failures that should propagate implicitly. I catch where I can recover or translate meaningfully. When wrapping a dependency failure, I preserve its original exception through PREVIOUS.

Example and follow-up A repository failure might become an application-specific document-read exception while retaining the technical cause. I do not add an empty CATCH merely to satisfy a signature, and I do not replace an established framework error contract without considering its caller.

Avoid this mistake NO_CHECK does not mean an exception cannot be caught; DYNAMIC_CHECK does not mean it can cross any procedure interface without declaration.

Read and verify this answer

SAP exception category and propagation rules

ABAP-11 Which modern ABAP expressions are useful, and when should you avoid them?

Inline declarations, VALUE, CORRESPONDING, COND, CONV, and string templates can make intent clearer and reduce temporary variables. I use them when the supported release and language version allow them. A deeply nested expression that hides branching or expensive work is harder to debug, so I prefer named intermediate steps when they explain the business logic.

Example and follow-up CORRESPONDING maps components by name unless explicit mapping changes that behavior. CONV converts a value; it does not validate every business rule. A short expression for selecting a default is useful, while a dense REDUCE replacing several unrelated calculations may be less readable.

Avoid this mistake Modern expression syntax is not synonymous with ABAP SQL and does not automatically improve runtime performance.

Read and verify this answer

SAP internal table reference and examples

SAP ABAP Keyword Documentation

ABAP-12 How do you handle amounts, quantities, dates, and timestamps correctly?

I keep amounts with currency and quantities with their unit, using suitable decimal types and the application's conversion rules. I distinguish a date from a timestamp and preserve time-zone context when converting instants for users. Formatting is a presentation concern; calculations and interfaces need stable semantics, precision, and documented rounding behavior.

Example and follow-up Adding 100 EUR to 100 USD without conversion produces a meaningless total. A deadline stored as an instant can cross a local date boundary for another user. Test currency decimal differences, daylight-saving transitions, and rounding at the intended business boundary.

Avoid this mistake A numeric field alone does not explain its currency or unit, and a local date-time string is not an unambiguous global instant.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-13 When would you use RTTI, RTTC, or dynamic ABAP?

RTTI inspects runtime type information; RTTC creates type descriptions used to construct data objects dynamically. They are valuable for generic frameworks whose structure is genuinely unknown beforehand. For normal business logic I prefer static types, because they make contracts, navigation, checks, and maintenance easier. Dynamic names and selections also require explicit validation.

Example and follow-up A configurable export framework may need to inspect component metadata and construct a selected output shape. An ordinary invoice service with a stable schema normally does not. Validate allowed fields rather than turning arbitrary user text into dynamic SQL fragments.

Avoid this mistake Dynamic programming is not inherently more reusable, and escaping values alone does not validate arbitrary dynamic SQL identifiers or syntax.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-14 How would you investigate an ABAP failure without changing production behavior?

I start with the exact operation, inputs, time, user, and system context, then correlate application logs, error details, and a dump if one exists. I reproduce with controlled data and inspect the failing call path. A fix should address the cause and preserve authorization, locking, and transaction boundaries, with a focused regression scenario.

Example and follow-up In classic systems ST22 can identify a dump and its call stack; debugging or tracing must use appropriate permissions and scope. A background or update failure may run under a different user and execution context from the original dialog request.

Avoid this mistake Suppressing an exception or granting broad authorization may hide the symptom while leaving the defect and data-integrity risk intact.

Read and verify this answer

SAP ABAP Keyword Documentation

Official references

SAP internal table reference and examples

SAP LUW reference and examples

SAP exception category and propagation rules

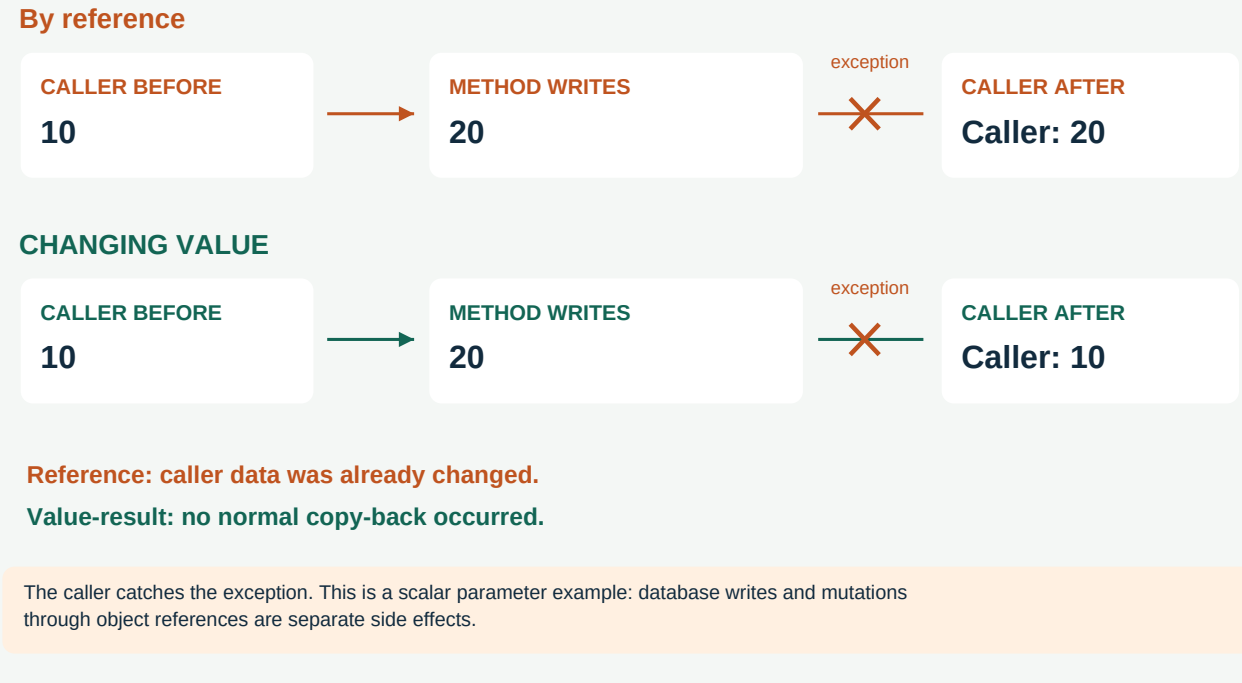
SAP ABAP Keyword Documentation

Practical ABAP: values, expressions, and failure paths

Use these worked scenarios after the basic language questions. They test the consequences of parameter passing, aliasing, partial updates, and failure handling rather than syntax recall alone. Modern additions require a supporting target release and language version; examples are teaching scenarios, not executed SAP evidence.

An exception separates reference and value-result paths

A plain CHANGING integer starts at 10; the method sets 20, then raises an exception



The caller catches the exception. This is a scalar parameter example: database writes and mutations through object references are separate side effects.

Compare the same assignment under two transfer modes. The final caller value is visible evidence of the difference.

See the visual atlas for the recall prompt, limits and related questions.

BASE-01 What changes when a method receives a CHANGING parameter by value instead of by reference?

By reference, the formal parameter aliases the caller's data, so allowed writes affect it immediately. With VALUE, a local value is changed and copied back on normal completion. An exception leaving the method prevents that normal copy-back. This difference affects error guarantees, but it does not undo database writes or changes made through referenced objects.

Example and follow-up Suppose a method changes a plain integer from 10 to 20 and then raises a class-based exception. A reference CHANGING parameter can leave 20 visible to the caller; a value-result parameter retains the caller's 10. Test both failure and success paths before changing an established signature.

Avoid this mistake Pass-by-value is not a transaction rollback and does not clone objects behind reference parameters.

Interviewer probes How would this change with an object reference parameter? / Can an IMPORTING reference variable still expose a mutable object?

Strong answer criteria Separates parameter direction from transfer mode; Explains exceptional copy-back behavior; Identifies external side effects

Verification evidence A small method-call test showing caller values after normal return and exception.

Read and verify this answer

SAP: Pass by value

BASE-02 A copied structure contains a string, a nested table, and an object reference. Is it an independent snapshot?

The string and nested table are value components: changing their copied contents does not intentionally mutate the source value. The reference component is different: copying it preserves a reference to the same object. Mutating that object's state through either structure is observable through the other. A true snapshot therefore needs an explicit policy for references and mutable reachable objects.

Example and follow-up For an approval snapshot, copying ls_request preserves the item values but does not freeze an attached mutable pricing object. Store the calculated price as a value, use an immutable collaborator, or implement deliberate cloning with defined identity and cycle handling. Internal sharing optimizations do not change these language-level semantics.

Avoid this mistake A deep ABAP type is not a promise that ordinary assignment recursively clones the entire object graph.

Interviewer probes Which components need copying for an audit snapshot? / How would you test sharing without inspecting memory addresses?

Strong answer criteria Distinguishes value components from references; Preserves object identity semantics; Defines snapshot ownership

Verification evidence Mutation tests for a copied string, nested row, and referenced object's attribute.

Read and verify this answer

SAP: Sharing between dynamic data objects

BASE-03 A large temporary internal table is no longer needed. When would you choose CLEAR or FREE?

For a headerless internal table, both remove its rows. CLEAR keeps the initial memory requirement available, while FREE also releases that initial allocation. I use CLEAR when the same table will soon be populated again and FREE when retaining its allocation has no useful purpose. Neither deletes the declared variable or guarantees an immediate reduction in operating-system memory.

Example and follow-up In a package loop, repeatedly freeing and rebuilding a modest buffer may add allocation work. After a one-off large reconciliation, releasing the unused table can reduce retained application memory. Measure used and allocated memory in the relevant session before drawing conclusions from process memory alone.

Avoid this mistake Do not claim CLEAR retains every byte ever allocated, or that FREE destroys referenced objects still reachable elsewhere.

Read and verify this answer

SAP: CLEAR

BASE-04 A dynamic ASSIGN fails, but IS ASSIGNED is still true. How can the program update the wrong field?

A failed dynamic assignment can leave the field symbol pointing at its previous target. IS ASSIGNED then says only that some assignment exists, not that the latest lookup succeeded. I check the statement's result immediately and avoid using the alias after failure. On a supporting release, ELSE UNASSIGN makes the failure state explicit; otherwise I unassign before attempting the lookup.

Example and follow-up A mapper assigns component PRICE successfully, then requests nonexistent component PRCIE. If it tests only IS ASSIGNED, a subsequent write may overwrite PRICE. A regression test must include a successful lookup followed by an invalid one using the same field symbol.

Avoid this mistake An assigned field symbol is not evidence that the most recent dynamic ASSIGN succeeded.

Read and verify this answer

SAP: ASSIGN, ELSE UNASSIGN

BASE-05 A READ TABLE fails, yet a later IF sy-subrc = 0 takes the success branch. What do you inspect?

I inspect every intervening statement that can set sy-subrc. It is a shared runtime field, not a result permanently attached to READ TABLE. I branch immediately or preserve the result in a named local variable before further work. For modern expressions, I use their documented exception or predicate behavior instead of assuming they set sy-subrc like classic statements.

Example and follow-up A failed lookup is followed by a successful FIND used to prepare logging text. Testing sy-subrc afterward now tests FIND. Include a missing-record case with successful logging preparation; merely testing an immediate lookup in isolation would miss the actual sequence.

Avoid this mistake Not every statement resets sy-subrc, and a table expression is not a drop-in sy-subrc-producing READ TABLE.

Read and verify this answer

SAP: SY-SUBRC

BASE-06 An import must reject precision loss rather than silently round or truncate. Where does EXACT help?

EXACT requests a lossless assignment or calculation and raises an appropriate exception when the requested result cannot be represented without loss. CONV applies ordinary conversion rules and is not equivalent validation. I place the check at a typed boundary, translate conversion failures into useful input feedback, and still validate domain rules such as permitted signs, identifiers, and currency.

Example and follow-up Converting a runtime text longer than a fixed character target can otherwise discard characters. For a two-decimal packed result, EXACT can reject a calculation such as one divided by three. That is useful when rounding is forbidden; when rounding is required, make the business rounding step explicit instead.

Avoid this mistake EXACT does not prove that a technically representable value is an allowed business value.

Read and verify this answer

SAP: EXACT examples

BASE-07 Two invoice totals differ by one cent although both use decimal types. How do you diagnose rounding?

I compare the sequence of calculations, intermediate precision, and the prescribed rounding point and mode. Rounding every line and then summing can differ from summing precise values and rounding once. Currency metadata does not automatically make an arbitrary calculation compliant. I retain currency context, use appropriate decimal types, and encode the agreed rule explicitly with boundary tests.

Example and follow-up Under an agreed half-up, two-decimal rule, two positive line results of 0.005 become 0.01 each if rounded separately, totaling 0.02. Their unrounded sum is 0.010 and rounds to 0.01. The correct amount depends on the specified line-versus-document rule.

Avoid this mistake Choosing decfloat34 or formatting two decimals does not settle the business rounding policy.

Interviewer probes How do negative half-way values affect the selected mode? / Where is currency conversion applied relative to tax rounding?

Strong answer criteria Traces intermediate precision; Names rounding stage and mode; Keeps currency semantics

Verification evidence Worked positive/negative boundary examples with independently calculated expected totals.

Read and verify this answer

SAP: Rounding function ROUND

BASE-08 Why can replacing MOVE-CORRESPONDING with CORRESPONDING erase an existing structure field?

An ordinary CORRESPONDING constructor creates a new result, so target components absent from the source become initial. MOVE-CORRESPONDING between structures leaves unmatched target components unchanged. BASE can supply starting values for a constructor result before matching components overwrite them. I specify the desired merge contract rather than assuming all name-based assignments preserve existing state.

Example and follow-up ls_existing contains status Approved and a note. The source contains only a new note: plain CORRESPONDING clears the unmatched status; BASE preserves it. But if the source type contains status with an initial value, that matching component still overwrites the base. Field presence requires a separate update contract.

Avoid this mistake BASE preserves unmatched components; it does not mean ignore every initial source value or implement PATCH automatically.

Read and verify this answer

SAP: Working with structures

BASE-09 Unsorted orders must be processed once per customer with their member rows. How does GROUP BY change the loop?

LOOP AT with GROUP BY first forms groups and then executes the group loop. Equal group keys are collected even if rows are not adjacent, unlike traditional control-level processing that relies on ordering. I choose explicit group-key binding when it improves clarity and use LOOP AT GROUP for members. Group order and member order still need a deliberate contract.

Example and follow-up Input customers A, B, A creates two groups, with both A orders together. The group body executes twice, not once per order. WITHOUT MEMBERS is suitable when only group keys are needed, but then a later member loop is not available.

Avoid this mistake Grouping an internal table is not database aggregation, and its default group order is not automatically business sort order.

Read and verify this answer

SAP: LOOP AT itab, GROUP BY

BASE-10 A FILTER expression is proposed for a standard table with an empty primary key. Is that enough?

For the basic FILTER form, access must use a suitable sorted or hashed key. A standard table can work when it has an appropriate secondary key selected with USING KEY; an empty primary key alone is insufficient. I also check that the predicate meets the selected key's rules. A FOR expression or a clear loop may fit an arbitrary predicate better.

Example and follow-up A standard table retains arrival order but has a sorted secondary key by customer. FILTER USING KEY can select that customer. A complex OR predicate over unrelated columns should not be forced into a key-based FILTER shape or justified by an unmeasured speed claim.

Avoid this mistake FILTER is not a general-purpose synonym for every WHERE expression supported by LOOP or FOR.

Read and verify this answer

SAP: FILTER, basic form

BASE-11 A REDUCE expression calculates a maximum, but all-negative input returns zero. What went wrong?

The accumulator's initial value is part of the algorithm. Starting a maximum at zero invents a candidate that may not exist in an all-negative input set. I define empty-input semantics and initialize from a valid first member or represent the absence of a result explicitly. I also choose accumulator types deliberately so intermediate overflow or rounding cannot hide behind concise syntax.

Example and follow-up For values -8 and -3, a zero-initialized maximum remains zero although the correct maximum is -3. Empty input may require no result or a domain exception, not the same number. The expression should remain readable enough that reviewers can identify initialization, iteration, and state update.

Avoid this mistake REDUCE does not supply mathematically correct initial values or empty-set semantics on the developer's behalf.

Read and verify this answer

SAP: Table reductions, structured result

BASE-12 A report accepts a filter value and a sort-column name. Is escaping quotes sufficient for safe SQL?

No. Data values and SQL identifiers are different inputs. I bind values through host variables in static ABAP SQL where possible. For configurable identifiers or dynamic clauses, I restrict them to a server-controlled allowlist and validate the supported syntax. Escaping a string value cannot authorize a table name, column name, operator, or an entire user-supplied WHERE fragment.

Example and follow-up A search value containing an apostrophe should remain a value. A sort request is mapped from an accepted business option to a known column; arbitrary SQL text is rejected. Authorization predicates are built independently and cannot be removed by user filter or ordering choices.

Avoid this mistake Input that is safely quoted as a literal is not therefore safe as SQL syntax or an authorized data access.

Interviewer probes How would you test an input that is valid syntax but unauthorized? / Which parts of the SQL shape can remain completely static?

Strong answer criteria Separates values and identifiers; Uses explicit allowed SQL shapes; Preserves authorization predicates

Verification evidence Negative tests for quoted values, forbidden columns, injected operators, and unauthorized records.

Read and verify this answer

SAP: Prevent SQL injection

BASE-13 A resource release is placed only in CLEANUP. Why can it remain unreleased after success?

ABAP CLEANUP is part of exception unwinding, not a finally block that runs after every TRY. It runs for the relevant protected context when an exception propagates to an outer handler; it is not the normal-success path. I arrange explicit release on successful completion and on relevant exceptional exits, while respecting the actual owner's lock or resource lifetime.

Example and follow-up An application-owned temporary handle is opened, processing succeeds, and ENDTRY is reached: CLEANUP alone does not close it. Test success, an exception caught locally, and one propagated outward. Cleanup failures also need handling so they do not obscure the original failure or escape a CLEANUP block.

Avoid this mistake Do not assume CLEANUP automatically runs after a CATCH in the same TRY, or release framework-owned locks prematurely.

Interviewer probes What changes when the exception is caught inside the same TRY? / Who owns release when a caller supplied the resource?

Strong answer criteria Distinguishes CLEANUP from finally; Covers normal and exceptional paths; Respects ownership

Verification evidence Instrumented resource tests show one correct release on every intended exit.

Read and verify this answer

SAP: Exception system response

BASE-14 An optional limit parameter is omitted in one call and explicitly set to zero in another. How can a method distinguish them?

IS SUPPLIED tests whether an actual parameter was provided, independently of its value. IS INITIAL tests the value and cannot distinguish an omitted zero-defaulted argument from an explicit zero. I use presence only when the API deliberately assigns different meaning to omission. If a wrapper always forwards a parameter, it must preserve the intended presence contract explicitly.

Example and follow-up For a pagination helper, omission may mean use the configured limit, while zero may mean return no rows. A wrapper that forwards its own initial variable makes the downstream parameter supplied. Test the direct calls and the wrapper path, rather than silently changing zero into the default.

Avoid this mistake OPTIONAL does not automatically preserve absent-versus-present semantics across another method boundary.

Read and verify this answer

SAP: IS SUPPLIED

BASE-15 A single-line INSERT INTO TABLE hits an existing unique primary key. Does it update the existing row?

No. For this internal-table insertion case, the new line is not inserted and sy-subrc indicates the duplicate. The existing row is not an implicit update target. I decide whether the operation means reject, merge, or replace, then implement that policy explicitly. I do not generalize this return-code behavior to every constructor, secondary-key conflict, or database insertion form.

Example and follow-up An import contains order 7 twice with different amounts. A unique hashed table prevents duplicate keys, but silently ignoring the second line can conceal inconsistent source data. Compare payloads if duplicates are allowed only when identical; otherwise report both the key and conflicting input position.

Avoid this mistake Uniqueness enforcement is not a complete deduplication policy, and INSERT is not an automatic upsert.

Read and verify this answer

SAP: Populating internal tables

BASE-16 An authorization helper returns an empty range table. Why is WHERE field IN range unsafe as the only access restriction?

An initial selection range represents no restriction for IN, so it must not be used to encode 'the user may see nothing.' I define empty authorization results separately and return no rows or reject the operation before querying. Nonempty ranges combine inclusion and exclusion rules; their semantics differ from a plain list whose empty membership set would match nothing.

Example and follow-up If a user has no permitted plants, the application must not execute an otherwise unrestricted read with an empty plant range. Keep optional user filters separate from mandatory authorization constraints. Test no permissions as well as one permitted plant and exclusions.

Avoid this mistake An empty ABAP selection range is not the same as an SQL list containing zero allowed values.

Interviewer probes How do you distinguish an optional filter from mandatory authorization? / What result do you expect from a range containing only exclusions?

Strong answer criteria Recognizes initial-range semantics; Fails closed for no permissions; Separates user filtering from security

Verification evidence A denied-user test proves zero data and zero aggregate leakage with an empty authorization result.

Read and verify this answer

SAP: Comparison with selection tables

Official references

SAP: Pass by value

SAP: Sharing between dynamic data objects

SAP: CLEAR

SAP: ASSIGN, ELSE UNASSIGN

SAP: SY-SUBRC

SAP: EXACT examples

SAP: Rounding function ROUND

SAP: Working with structures

SAP: LOOP AT itab, GROUP BY

SAP: FILTER, basic form

SAP: Table reductions, structured result

SAP: Prevent SQL injection

SAP: Exception system response

SAP: IS SUPPLIED

SAP: Populating internal tables

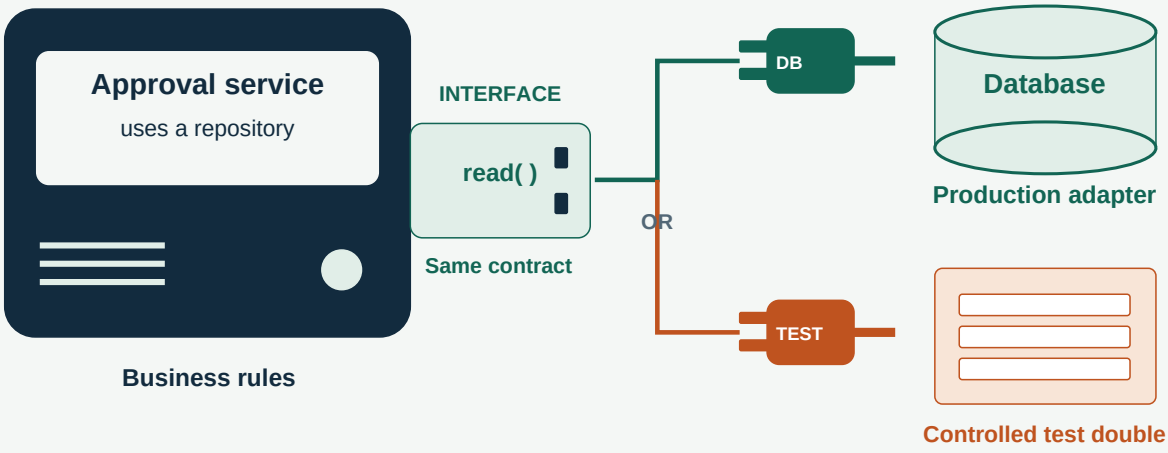
SAP: Comparison with selection tables

OO design and Clean ABAP

Explain object orientation through responsibilities, replaceable dependencies, and observable behavior. Memorized definitions become useful when you can name a concrete tradeoff. The examples are hypothetical design exercises, not claims about personal project experience; project naming conventions are separate from the SAP language.

One contract; production and test can plug in

Constructor injection supplies a collaborator. Polymorphism selects its implementation at runtime.



A test checks service behavior; a real integration check still verifies database behavior.

Inject one suitable implementation per instance. An adapter does not make an unreleased API released.

The service depends on a repository contract. Production and test implementations fit the same socket; the composition boundary supplies the appropriate collaborator.

See the visual atlas for the recall prompt, limits and related questions.

OO-01 Explain abstraction, encapsulation, inheritance, and polymorphism with one example.

For a delivery service, abstraction exposes a send operation while hiding transport details. Encapsulation protects its internal state and invariants. Inheritance can specialize a genuine base contract. Polymorphism lets the caller invoke send through an interface while different implementations deliver by different channels. These ideas help changes stay local to the responsible component.

Example and follow-up A notification coordinator can receive an email sender or a test sender through the same interface. The coordinator should not inspect the concrete class to decide how to send; doing that would weaken the polymorphic design.

Avoid this mistake Encapsulation means protecting valid behavior and state, not merely generating getters and setters for every attribute.

Read and verify this answer

SAP ABAP object orientation examples

OO-02 What can an ABAP interface contain, and when would you use one?

An ABAP interface defines a contract implemented by classes. It can declare methods and other components such as types, constants, attributes, and events; it is not limited to abstract methods. I normally keep behavioral interfaces narrow so callers depend on the capability they need and tests can supply a substitute implementation.

Example and follow-up A price-provider interface can expose a calculation operation without publishing database access. Two implementations need not be similar classes overall; sharing the required business contract is enough. Ordinary interfaces and specialized BAdI interfaces have different restrictions.

Avoid this mistake An interface is not an abstract class, and a collection of similarly named methods alone does not establish a meaningful common contract.

Read and verify this answer

SAP interfaces and implementation semantics

OO-03 How do ABSTRACT and FINAL affect classes and methods?

An abstract class cannot be instantiated directly and may provide shared implementation. It does not have to contain an abstract method. An abstract method has no implementation there and must be implemented before a concrete subclass can be instantiated. FINAL prevents further subclassing for a class, or further redefinition for a method.

Example and follow-up An abstract document processor could implement logging while requiring a concrete processing step. If no supported specialization is intended, a final class communicates that choice. Marking a class final does not stop it implementing interfaces or receiving collaborators.

Avoid this mistake Do not import another language's virtual keyword or method-overloading rules into ABAP; use ABAP's actual redefinition and signature rules.

Read and verify this answer

SAP ABAP object orientation examples

OO-04 How do upcasting and downcasting work in ABAP Objects?

Upcasting views an object through a compatible superclass or interface reference and preserves the same object. Downcasting requests a more specific compatible type and can fail at runtime if the actual object does not support it. I use a type check or handle CX_SY_MOVE_CAST_ERROR when a downcast is genuinely necessary.

Example and follow-up A reference typed to a payment interface can point to a card implementation. Converting it back to the card type does not create a new card object. Frequent downcasts often indicate the interface is missing a required capability or the caller knows too much.

Avoid this mistake Casting a reference changes the accessible static view, not the object's dynamic type or stored data.

Read and verify this answer

SAP ABAP object orientation examples

OO-05 What is the difference between instance and static methods and constructors?

An instance method acts on an object and can access its instance state. A static method belongs to the class and has no implicit current instance. The instance constructor initializes a newly created object. The class constructor initializes class-level state when the runtime first requires it in the internal session; I avoid unpredictable heavy work there.

Example and follow-up A class can contain static and instance methods together. Restricting creation or declaring a class abstract affects instantiation; merely adding CLASS-METHODS does not. Constructor injection makes mandatory dependencies visible before business methods are called.

Avoid this mistake ABAP does not make a class uninstantiable simply because it has static methods, and class data is not automatically shared system-wide.

Read and verify this answer

SAP ABAP object orientation examples

OO-06 When do you prefer composition over inheritance?

I prefer composition when behavior varies independently or I want to replace a dependency without coupling classes to a hierarchy. Inheritance is appropriate when the subclass really preserves the base contract and specialization is intentional. Composition makes ownership and collaboration explicit and avoids a deep hierarchy used only to share a few helper methods.

Example and follow-up A report service can contain a repository, an authorization policy, and a formatter. Different implementations can be supplied independently. Making every report inherit from a large superclass may entangle unrelated persistence, output, and authorization decisions.

Avoid this mistake Code reuse alone is weak justification for inheritance; a subclass that violates caller expectations is not a sound substitute.

Read and verify this answer

SAP Clean ABAP style guide

SAP ABAP object orientation examples

OO-07 How do dependency injection and dependency inversion differ?

Dependency inversion is a design principle: business policy should depend on suitable abstractions rather than concrete infrastructure details. Dependency injection is a way to supply a collaborator from outside, commonly through a constructor. Together they make dependencies visible and replaceable, but injection alone does not guarantee a useful abstraction or a good responsibility split.

Example and follow-up An approval service receives a repository interface. Production composition supplies a database implementation; an ABAP Unit test supplies a controlled double. The service should not immediately instantiate its own concrete repository and ignore the injected dependency.

Avoid this mistake Injecting a concrete class everywhere may aid construction while still leaving the business layer tightly coupled to infrastructure.

Read and verify this answer

SAP Clean ABAP style guide

OO-08 How would you apply SOLID without overengineering?

I use SOLID as questions about change. Does each component have a coherent responsibility? Can variation be added without repeatedly editing central logic? Do substitutes preserve expectations? Are interfaces small enough for their clients? Do business rules depend on abstractions? I introduce structure to solve an observed need, not to maximize the number of classes.

Example and follow-up Separating tax calculation from database retrieval can support focused tests and independent changes. Creating an interface for a tiny stable helper with no substitution need may add little value. Discuss one concrete change scenario rather than reciting five slogans.

Avoid this mistake Single responsibility does not mean one method per class, and open/closed does not mean existing code must never be changed.

Read and verify this answer

SAP Clean ABAP style guide

OO-09 When are Strategy and Factory useful?

Strategy packages interchangeable behavior behind a common contract, such as choosing a pricing rule. A factory centralizes construction when callers should not know concrete classes or setup details. They often work together: configuration selects a strategy that a factory creates. A simple static creation helper is not automatically the inheritance-based Factory Method pattern.

Example and follow-up For domestic and export pricing, the coordinator receives a pricing strategy. The composition boundary decides which implementation is appropriate. If only one simple construction path exists, direct construction outside the business logic may be clearer than an elaborate factory hierarchy.

Avoid this mistake Do not call every method that returns an object Factory Method without distinguishing a simple factory from the formal pattern.

Read and verify this answer

SAP ABAP object orientation examples

SAP Clean ABAP style guide

OO-10 What do Adapter and Facade solve in SAP integration code?

An adapter translates one contract into another, allowing a client to use an existing service through its preferred interface. A facade offers a simpler operation over a subsystem with several steps. For a legacy posting API, either pattern can isolate mapping and error translation while keeping its transaction, authorization, and release limitations visible.

Example and follow-up A posting facade might coordinate validation and mapping; an adapter can translate an application's request into a legacy function module's structures. The caller still needs to know whether persistence is synchronous and who owns commit and rollback.

Avoid this mistake Wrapping an unreleased SAP object does not by itself make that underlying dependency released, cloud compliant, or clean-core Level A.

Read and verify this answer

SAP Clean ABAP style guide

OO-11 What are the limitations of Singleton in ABAP?

A conventional singleton restricts construction and returns a stored instance through a class-level accessor. Its scope is normally the ABAP internal session, not the whole SAP landscape. It can introduce hidden dependencies, persistent mutable test state, and difficult substitution. I use it only when that lifetime and access model are deliberate requirements.

Example and follow-up A configuration cache might need invalidation when customizing changes. A singleton that retains yesterday's assumptions or leaks state between tests becomes a correctness problem. Often an explicitly constructed shared service with an injected interface gives clearer ownership.

Avoid this mistake A singleton does not synchronize business updates across sessions, work processes, application servers, or users.

Read and verify this answer

SAP ABAP object orientation examples

OO-12 How do ABAP OO events differ from durable business events?

An ABAP OO event notifies registered handlers in the current runtime context and can decouple a publisher from listeners. A durable business event used for workflow or external integration needs a separate delivery mechanism, transaction coordination, monitoring, and retry design. Similar terminology does not mean a local event provides durable asynchronous messaging.

Example and follow-up An ALV interaction can raise an event handled by a controller registered with SET HANDLER. Starting an external approval process after a document save requires the supported business-event or integration mechanism, including handling duplicates and failures.

Avoid this mistake RAISE EVENT by itself does not persist a message or guarantee workflow creation after a commit.

Read and verify this answer

SAP ABAP object orientation examples

OO-13 What would you change first in a difficult legacy ABAP method?

I first identify its observable behavior, callers, and transaction responsibilities. Then I make small, reviewable improvements such as naming unclear concepts, separating a calculation from I/O, and removing repeated logic. I add focused tests around risky behavior before larger restructuring. Preserving the established contract matters more than turning every statement into modern syntax.

Example and follow-up A safe extraction might isolate discount calculation while keeping posting and commit behavior unchanged. In this project, new tables use lt_, structures ls_, and constants co_; these are team preferences, not mandatory ABAP syntax or universal Clean ABAP rules.

Avoid this mistake A broad stylistic rewrite without behavior evidence can introduce regressions and make review harder, even if the result looks more modern.

Read and verify this answer

SAP Clean ABAP style guide

OO-14 How do you make an OO design testable, and what should ABAP Unit assert?

I separate business decisions from database, clock, authorization, and external-service access through appropriate seams. Tests exercise observable behavior with controlled inputs and meaningful assertions. I cover boundaries, failures, and interactions that matter to the contract. Coverage helps identify untested areas, but it does not prove that assertions are correct or that real integration works.

Example and follow-up For an approval policy, test below, at, and above a threshold, plus a missing approver. A repository double can simulate not-found and technical failure. Separate integration checks must verify the real persistence and authorization behavior in the target system.

Avoid this mistake A test that only asserts that a method returned, or mirrors its implementation step by step, provides weak evidence.

Read and verify this answer

SAP Clean ABAP style guide

OO-15 How does MVC separate responsibilities?

The Model represents application data and relevant behavior. The View presents information and collects interaction. The Controller interprets user interaction and coordinates model and view changes. Frameworks implement the pattern differently; MVC is a responsibility split, not a fixed network sequence that every application follows.

Example and follow-up In SAPUI5, a controller can handle a button press while a bound model supplies view data. Backend business rules still belong in the appropriate service or BO, not only in the browser controller.

Avoid this mistake MVC does not mean copying all backend business logic into the UI controller or assuming the view must always be manually rebuilt.

Read and verify this answer

SAPUI5: Use the MVC Concept

Official references

SAP Clean ABAP style guide

SAP ABAP object orientation examples

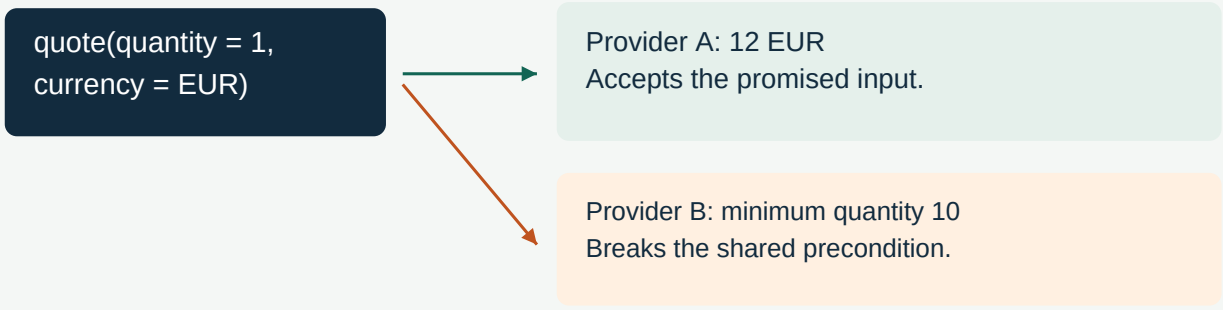
SAP interfaces and implementation semantics

Practical OO: contracts, state, and substitutability

Work through these decisions after the OO fundamentals. The emphasis is on observable promises: valid state, compatible substitutes, honest failure, ownership, and useful tests. Design recommendations are applications of the cited language rules and SAP guidance, not claims that the compiler enforces a complete business contract.

Same call shape does not prove the same behavior

Shared quote contract: accept every positive quantity; return the requested currency



Change request	Contract-preserving direction
Different inherited implementation	REDEFINITION keeps the signature
Optional simulation capability	Separate capability; honest unsupported result

The interface supports common dispatch across unrelated classes. It does not enforce business substitutability. Shared code alone does not justify a superclass.

A valid quantity-one call exposes a behavioral violation invisible in matching parameter types. The lower table separates redefinition from a genuinely new capability.

See the visual atlas for the recall prompt, limits and related questions.

OODEEP-01 A method named reserve returns a reservation ID. Which promises must be explicit beyond its parameter types?

I define accepted inputs, the observable success result, allowed state changes, and failure behavior. The caller needs to know whether the reservation is merely buffered or durable, whether repetition is safe, and who owns transaction completion. Those promises belong in the contract and its tests. A matching signature alone cannot tell another developer whether combining two operations is safe.

Example and follow-up One implementation returns an ID before save; another commits immediately. Both compile against the same interface but are not interchangeable for a caller that reserves two items atomically. Document the intended boundary in ABAP Doc and test failure of the second item against the first one's state.

Avoid this mistake RETURNING an identifier does not itself promise committed persistence, and a method name cannot replace a transaction contract.

Interviewer probes What state is allowed after reserve raises an exception? / What is the result when the same business request is retried?

Strong answer criteria Defines success and failure state; States transaction ownership; Defines repeat-call semantics

Verification evidence Contract tests plus documentation covering buffered success, persistence boundary, and repeated requests.

Read and verify this answer

SAP Learning: Documenting ABAP code

OODEEP-02 A new price-provider implementation rejects quantities below ten although the interface accepts any positive quantity. Is it a valid substitute?

No, if the published contract accepts every positive quantity, the new provider strengthens the precondition and breaks callers that validly request one unit. I either preserve the shared contract, expose the narrower capability separately, or change the contract through a managed migration. I compare outputs, errors, and side effects as well as input types when assessing substitutability.

Example and follow-up A checkout calls the provider through an interface with quantity one and expects a quote. A subclass or implementation that raises an unsupported-quantity error for that valid input violates the agreed behavior even though activation succeeds. Run the same contract scenarios against each implementation.

Avoid this mistake Implementing an interface or passing a cast check proves type compatibility, not behavioral substitutability.

Interviewer probes Can a substitute silently return a different currency? / Which common tests would run against every provider?

Strong answer criteria Identifies strengthened precondition; Checks postconditions and side effects; Uses shared behavioral tests

Verification evidence A contract suite exercises minimum positive quantity, currency, failures, and repeat calls for both providers.

Read and verify this answer

SAP Learning: Using interfaces

OODEEP-03 Two unrelated services share a capability and some validation code. Should their callers depend on an abstract superclass?

I separate the public capability from implementation reuse. An interface can express the capability without imposing a class hierarchy. Shared validation can be a collaborator. An abstract base class is justified when there is a stable specialization relationship and a useful shared lifecycle or template, not simply similar code. I also consider that a class has only one superclass.

Example and follow-up A document exporter and a notification sender may both support cancellation but have unrelated lifecycles. A narrow cancellation interface lets a coordinator address either, while shared policy is composed where needed. For a genuine family of parsers with enforced shared steps, an abstract template may be appropriate.

Avoid this mistake An abstract superclass is not automatically a better contract because it can carry reusable implementation.

Interviewer probes Which invariant would the superclass enforce? / How would the design accommodate a class already inheriting from a framework base?

Strong answer criteria Separates contract from reuse; Tests the is-a relationship; Accounts for independent variation

Verification evidence A responsibility sketch shows the caller's minimal interface and the owner of shared validation.

Read and verify this answer

SAP Learning: Defining interfaces

OODEEP-04 A subclass needs a different implementation and an extra mandatory input for an inherited method. Can REDEFINITION add it?

No. Redefinition replaces an eligible inherited instance method's implementation while preserving its interface. ABAP does not use same-name, different-signature method overloading to solve this. If the additional input changes the operation's contract, I introduce a distinct operation or a deliberate compatible contract evolution. The existing inherited operation must still satisfy callers using the base type.

Example and follow-up A base method formats a document. A specialized format requires an extra profile: provide a separately named operation, or configure a validated collaborator before use if that fits the existing contract. Requiring callers to downcast merely to make the inherited method usable exposes a broken abstraction.

Avoid this mistake Redefinition is not method overloading, and optional parameters do not automatically make every behavioral change compatible.

Read and verify this answer

SAP: Extending a subclass

SAP: ABAP Doc class example and method uniqueness

OODEEP-05 A local test class needs access to private state. What does granting friendship expose, and what does it not imply?

Friendship grants access to the granting class's protected and private components. It is unilateral, so the reverse access is not automatic. A friend of a superclass is not automatically a friend of its subclasses; subclasses of a friend can also gain access. I keep the grant narrow and first ask whether the test can assert the public behavior instead.

Example and follow-up A focused final local test class may need a legacy injection seam. Making every production attribute public is broader than necessary. Granting friendship to a widely implemented interface is also much broader than naming one final class, because implementations participate in the friendship relationship.

Avoid this mistake Do not summarize friendship as simply inherited or not inherited; the direction and which class is subclassed matter.

Read and verify this answer

SAP: Friendship between classes

OODEEP-06 Two references point to one resource-owning object. Does clearing one reference close the resource or destroy the object?

Clearing one reference changes that variable to the null reference; another reference can still keep the object reachable. Memory reclamation by the garbage collector is distinct from business resource management. If a connection, registration, or lock needs timely release, I define an explicit ownership and release protocol and execute it on the relevant completion and failure paths.

Example and follow-up A coordinator and cache both hold a reference to the same session object. Clearing the coordinator's reference leaves the cache entry usable. An explicit close can be appropriate for the owner, but then other holders need defined behavior after closure, and a borrowed resource must not be closed accidentally.

Avoid this mistake Reference clearing is not an object-level close operation and must not be used as a lock-release guarantee.

Interviewer probes Who may close a shared or borrowed object? / What should a second close or a call after close do?

Strong answer criteria Separates reachability and resource lifetime; Identifies owner versus borrower; Defines closed-state behavior

Verification evidence Tests cover multiple aliases, explicit close, repeated close, and use after closure.

Read and verify this answer

SAP: Initializations

OODEEP-07 An object can be created with a missing repository and fails much later on its first business call. What would you change?

I make mandatory construction requirements explicit and establish a valid state before the object is usable. Constructor injection can require the repository, with a meaningful failure if the supplied reference is invalid. Complex or fallible setup can be coordinated by a factory. I avoid publishing a half-initialized object or making unrelated business operations depend on a hidden setup call.

Example and follow-up An approval service requires a repository and a policy. A public zero-argument constructor followed by set_repository creates an invalid intermediate state. Validate the mandatory dependencies at creation; if opening an external session is a separate lifecycle step, expose and test that lifecycle explicitly rather than surprising every constructor caller.

Avoid this mistake Constructor injection does not validate an initial reference automatically, and heavy I/O is not required for good initialization.

Interviewer probes When would a factory improve error handling over constructor work? / How do you prevent this object escaping before setup succeeds?

Strong answer criteria Defines construction invariants; Rejects missing dependencies early; Separates setup from business work

Verification evidence Creation tests prove invalid dependencies fail immediately and valid instances need no undocumented setup call.

Read and verify this answer

SAP Learning: Encapsulation and consistency

OODEEP-08 A test passes alone but fails after another test changes CLASS-DATA. Why does NEW not reliably isolate it?

Creating another instance initializes that instance's state; it does not reset class-level mutable state. Static attributes persist within the relevant internal session and may be shared by every instance of that class there. I identify and remove unnecessary global state or provide an explicit, controlled test reset. Cache keys and invalidation must also reflect the real context.

Example and follow-up One test loads a policy cache, then another expects changed customizing. Recreating the service leaves the static cache populated. An injected policy provider with test-owned lifetime can avoid this coupling; if caching is a genuine requirement, test refresh, invalidation, and relevant user or tenant context separately.

Avoid this mistake CLASS-DATA is not per instance, but it is also not a globally synchronized cache across application servers.

Interviewer probes Does a class constructor rerun for every NEW? / Which context must appear in a security-sensitive cache key?

Strong answer criteria Identifies static lifetime; Separates test isolation from construction; Defines cache scope and invalidation

Verification evidence The suite passes individually, together, and with varied order without stale shared state.

Read and verify this answer

SAP: CLASS-METHODS

OODEEP-09 A repository raises a low-level exception. Should a business service catch CX_ROOT and return an empty result?

Usually not: an empty result can turn technical failure into a false business fact. I catch failures at a boundary that can recover or translate them into the service's declared meaning. When wrapping an exception, I preserve the original through previous so diagnosis remains possible. Expected absence, business rejection, and technical failure need distinguishable outcomes.

Example and follow-up A pricing repository connection failure must not look like 'no price exists' and trigger a free item. Translate the unavailable dependency into a meaningful service error while retaining its cause. Test not-found separately from unavailable, and ensure failure does not leave a misleading success result.

Avoid this mistake Broad catch-and-default handling is not resilience when it changes an unknown outcome into valid-looking data.

Interviewer probes When is catching CX_ROOT justified at an application boundary? / What information belongs in user feedback versus technical diagnostics?

Strong answer criteria Keeps absence distinct from failure; Translates at an abstraction boundary; Preserves causal exception

Verification evidence Tests assert distinct not-found and technical-failure paths, including the previous exception chain.

Read and verify this answer

SAP Learning: Working with exception classes

OODEEP-10 A getter returns a reference to a mutable internal collection. Has private visibility protected the object's invariants?

Private storage alone is insufficient if the getter hands out a reference that lets callers mutate the same data or object. I define whether the return value is a snapshot, a read capability, or intentionally shared mutable state. For an invariant-bearing aggregate, I prefer operations that validate changes and expose appropriate values rather than unrestricted aliases to internals.

Example and follow-up A cart validates positive item quantities in add_item, but get_items returns a writable data reference to its actual table. A caller can then insert a negative quantity without calling add_item. Returning a table value removes that table alias; reference components inside its rows still require separate ownership decisions.

Avoid this mistake A copied reference is not a copied referent; changing the static view alone does not create immutable state.

Interviewer probes Does returning a table value solve reference components inside its rows? / Which mutation should remain an operation of the owning object?

Strong answer criteria Finds alias-based invariant bypass; Chooses explicit ownership semantics; Considers nested references

Verification evidence Attempted external mutation cannot bypass the owner's quantity rule or alter a promised snapshot.

Read and verify this answer

SAP: Assignment rules for reference variables

OODEEP-11 A widely used provider interface needs an optional new capability. How do you avoid forcing unrelated implementations into fake behavior?

I assess both callers and implementers before extending the contract. A separate capability interface may let interested clients request the new operation without requiring every existing provider to pretend it supports it. If the main interface must evolve, I define semantics, compatibility, and rollout deliberately. A no-op implementation is valid only when doing nothing genuinely satisfies the documented contract.

Example and follow-up A price provider gains simulation support, but one external provider cannot simulate. Adding simulate everywhere and returning an empty success hides a capability gap. A simulation interface and explicit capability selection can keep pricing callers unchanged; tests verify unsupported capability handling at the composition boundary.

Avoid this mistake Adding a method is not automatically harmless merely because existing callers do not invoke it.

Interviewer probes How will existing implementers activate after the interface change? / When would a separate version be clearer than optional capability discovery?

Strong answer criteria Assesses caller and implementer impact; Avoids fake successful behavior; Defines migration and unsupported capability

Verification evidence An implementation inventory and compatibility tests cover old clients and providers without simulation.

Read and verify this answer

SAP: Interfaces in ABAP Objects

OODEEP-12 An OO unit test mocks calculate_total and then asserts the mocked total. What behavior has it proved?

It has mainly proved the test configuration, not the calculation. I keep the behavior named as the code under test real and replace dependencies that make its inputs unpredictable, such as a price source or clock. Then I assert independently expected results and important failure behavior. A separate orchestration test may mock a calculator only when orchestration, rather than arithmetic, is its subject.

Example and follow-up To test a discount calculation, provide controlled prices and exercise the real calculation at the threshold and around it. To test whether checkout requests a calculation before posting, the calculator can be a collaborator double. Naming the test's responsibility makes that difference explicit.

Avoid this mistake The same component can be a real subject in one test and a doubled dependency in another; coverage alone does not settle test quality.

Interviewer probes Which defect should make this test fail? / What real integration evidence remains necessary after a double-based unit test?

Strong answer criteria Names the actual tested responsibility; Doubles only dependencies; Uses independent assertions

Verification evidence A deliberately changed business rule makes its calculation test fail, while orchestration tests remain separately meaningful.

Read and verify this answer

SAP: Adding test doubles

Official references

SAP Learning: Documenting ABAP code

SAP Learning: Using interfaces

SAP Learning: Defining interfaces

SAP: Extending a subclass

SAP: ABAP Doc class example and method uniqueness

SAP: Friendship between classes

SAP: Initializations

SAP Learning: Encapsulation and consistency

SAP: CLASS-METHODS

SAP Learning: Working with exception classes

SAP: Assignment rules for reference variables

SAP: Interfaces in ABAP Objects

SAP: Adding test doubles

Practical ABAP design patterns and trade-offs

Use patterns to explain a concrete design decision, not to recite a catalog. These original senior scenarios build on OO-09 through OO-12 and distinguish pattern mechanics from RAP transaction, API-release and security contracts. SAP's pattern examples are exploratory demonstrations, not production templates. Verify syntax and dependencies on the target release. Recommendations below are contextual analysis, not claims that SAP mandates a particular application architecture.

One variation: replace a rule or specialize a step?

Worked choice: approval policy varies by country; process ownership must stay stable

STRATEGY / composition

Approval coordinator -> rule interface

Country A rule
Limit: 500 EUR

Country B rule
Limit: 900 EUR

700 EUR: A rejects; B accepts under this fixture.

TEMPLATE METHOD / inheritance

1. **Read inputs**
2. **Calculate policy**
3. **Return decision**

Stable skeleton; subclass supplies the chosen step.

Assembly decision	Boundary to keep
Factory selects policy	Validate country and policy version
Constructor receives rule	Rule returns a decision; does not commit

The amounts are fictional policy fixtures. Choose a real variation point; a reusable calculation alone does not require an inheritance hierarchy.

The same 700 EUR request receives different policy decisions through the same Strategy contract. Template Method instead fixes a call skeleton and exposes deliberate subclass steps.

See the visual atlas for the recall prompt, limits and related questions.

PAT-01 Approval rules vary by country, while the application workflow should remain stable. Strategy or Template Method?

Prefer an injected Strategy when approval calculation is the variation and the caller should select or replace it without changing its own class hierarchy. Template Method fits a deliberately stable algorithm skeleton with controlled subclass extension steps. Here, I would keep workflow coordination separate from country rules and pass explicit context to the selected rule. Inheritance becomes justified only if its protected extension contract and lifecycle are genuinely stable requirements.

Example and follow-up A country rule should return a decision rather than commit, send email or modify hidden global state. Test the same coordinator with several rule implementations. If countries also vary the process sequence, revisit the workflow model instead of adding increasingly powerful hooks that subclasses can use to violate invariants.

Avoid this mistake Choosing inheritance solely to share code, or allowing a rule implementation to take over transaction ownership.

Interviewer probes Which variation changes at runtime rather than through subclassing? / How would you prevent a new rule from changing the workflow's transaction boundary?

Strong answer criteria Separates algorithm variation from coordination; Defends composition versus inheritance contextually; Keeps rules substitutable and independently testable

Verification evidence Run identical coordinator contract tests with two country rules and a test double; assert decision and side-effect boundaries.

Read and verify this answer

SAP ABAP pattern examples: Strategy

SAP ABAP pattern examples: Template Method

PAT-02 A posting integration needs matching serializer and sender implementations for each provider. Is one static factory sufficient?

A simple factory can centralize selection for a small fixed set of implementations. Abstract Factory becomes useful when callers must obtain a compatible family, such as a provider-specific serializer and sender, without mixing families. Factory Method instead delegates creation through a subclass-overridable creation operation. I would choose the smallest construction design that protects the actual compatibility rule, rather than calling every method returning an object the Factory Method pattern.

Example and follow-up For two stable providers, an explicit factory returning a validated pair may suffice. If multiple related products evolve independently, a provider-factory interface can make their compatibility visible. Fail clearly for unknown configuration and test family consistency; silently selecting a default provider can post to the wrong destination.

Avoid this mistake Confusing a static creation helper with inheritance-based Factory Method or introducing an Abstract Factory for one trivial object.

Interviewer probes What concrete invalid combination must construction prevent? / When would adding a creator subclass be better than changing a configuration map?

Strong answer criteria Distinguishes three creation approaches; Names a real product-family invariant; Includes invalid configuration and evolution costs

Verification evidence Test each supported provider family and an unknown provider; verify no serializer/sender cross-pairing can be constructed.

Read and verify this answer

SAP ABAP pattern examples: Abstract Factory

SAP Factory Method pattern

PAT-03 A legacy posting API has awkward parameters and a three-step prepare/check/post sequence. Would you build an Adapter, a Facade, or both?

Use an Adapter to translate the application's posting contract into the legacy API's types, identities and error representation. A Facade can expose a coherent business operation over the three underlying steps. They may coexist, but their boundaries must preserve authorization, failure information and transaction ownership. Neither pattern makes an unreleased API eligible for ABAP Cloud use or removes a nested commit. I would reject a superficially clean wrapper that hides those constraints.

Example and follow-up Separate mapping tests from orchestration tests. For example, verify that a zero amount remains explicit, API warnings do not become unconditional success, and a failed check prevents posting. Expose an uncertain remote outcome distinctly from a definitive rejection so callers can reconcile instead of duplicating work.

Avoid this mistake Treating any wrapper as both a supported API and a transaction adapter without inspecting its dependencies.

Interviewer probes Which errors belong to the domain contract and which retain technical detail? / What evidence would make the legacy API unsuitable despite a neat interface?

Strong answer criteria Separates contract translation from subsystem orchestration; Preserves transactional and release constraints; Tests meaningful error and value mapping

Verification evidence Trace all three steps with injected failures and inspect dependency release state; assert no hidden commit or swallowed business rejection.

Read and verify this answer

SAP ABAP pattern examples: Adapter

SAP ABAP pattern examples: Facade

PAT-04 You want timing, diagnostic logging and retries around an outbound service. Would stacked Decorators be safe?

Decorators can add behavior around the same service interface without creating subclasses for every combination. I would make order explicit and preserve the wrapped contract, especially exceptions, authorization context and return values. Retrying is not a harmless decoration: it requires operation-specific idempotency and classification of uncertain outcomes. I would enable retries only where the service contract supports them, and decide whether timing and logging describe each attempt or the overall logical operation.

Example and follow-up A logging decorator outside a retry decorator records one logical call; inside it, it can observe individual attempts. Both can be useful if correlated clearly. Redact sensitive payloads and ensure diagnostic failure does not silently change a successful business result unless the required audit contract explicitly demands that behavior.

Avoid this mistake Adding automatic retries to every POST or assuming decorator order cannot change observable behavior.

Interviewer probes Which wrapper should count attempts and which should measure total latency? / How does a timeout after remote success affect the retry policy?

Strong answer criteria Preserves the component interface contract; Reasons about wrapper ordering; Separates retries from idempotency guarantees

Verification evidence Test wrapper orders, a transient read failure, uncertain write completion and logging failure; inspect returned outcome and number of remote effects.

Read and verify this answer

SAP ABAP pattern examples: Decorator

SAP Clean ABAP: explicit failure flow

PAT-05 Three handlers can route a request to an approver. The first matching rule should win. How would you implement and test Chain of Responsibility?

Model the decision contract explicitly: handled, not applicable and failed are different outcomes. An ordered Chain of Responsibility fits first-match routing when each handler can accept the request or pass it onward. Validate configuration order and define the no-match outcome. If the business instead needs every validation result, use an explicit validation pipeline or collection rather than stopping at the first successful handler and assuming the rest of the checks ran.

Example and follow-up Put the most specific route ahead of a general fallback only when that precedence is intended. Avoid hidden database writes in a routing handler; return a decision that the transaction coordinator applies. Log the rule identity so support can explain why the request reached a particular approver.

Avoid this mistake Treating an exception as 'not applicable', or confusing first-match routing with execute-all validation.

Interviewer probes What happens when two handlers match? / How is a configuration error different from a valid request with no applicable route?

Strong answer criteria Defines routing outcomes and precedence; Distinguishes chain from validation pipeline; Keeps decisions explainable and side effects controlled

Verification evidence Test overlapping rules, no match, handler failure and reordered configuration; assert exactly which handlers execute and which route is chosen.

Read and verify this answer

SAP ABAP pattern examples: Chain of Responsibility

PAT-06 A local ABAP event updates an ALV, and the team proposes another handler to start an external workflow. What boundary has changed?

The local Observer arrangement is suitable for notifying listeners in the current execution context. Starting a durable external workflow introduces commit, delivery, retry and recovery requirements that an ordinary ABAP Objects event does not supply. I would keep local display notification separate from a committed business transition and use an appropriate transactional publication mechanism for the integration. Multiple listeners also require an explicit policy for exceptions and lifecycle registration.

Example and follow-up Registering multiple listener instances, failing to deregister obsolete listeners, or retaining state across tests can produce surprising local notifications. For the workflow, duplicate delivery and delayed events require durable consumer correlation and state checks. Do not infer asynchronous durability merely because the publisher does not directly name the receiving class.

Avoid this mistake Assuming SET HANDLER creates a durable subscription or that Observer guarantees exactly-once business effects.

Interviewer probes Which part of the design survives an application-server restart? / What if one local listener fails after another already changed its state?

Strong answer criteria Distinguishes in-process notification from durable messaging; Coordinates external effects with committed state; Defines listener and consumer failure handling

Verification evidence Test local listener registration separately from rejected save, duplicate event and delayed workflow delivery cases.

Read and verify this answer

SAP Observer pattern

SAP RAP business events

PAT-07 A command object represents Submit and can be queued or retried. Does using Command make execution idempotent or undoable?

No. Command packages an operation and its inputs; durability, authorization, idempotency and compensation are additional contracts. I would assign a stable business operation identity, decide how it is stored with the state transition, and define responses to repeated execution or conflicting payload reuse. An undo method is only meaningful where a valid inverse exists. A posted document may require a compensating business operation rather than deletion or an in-memory rollback.

Example and follow-up A command held only in memory is not a reliable job queue. When persistence is required, store a versioned data contract and reconstruct an eligible executor; do not treat an arbitrary object reference as a durable payload. Recheck the relevant authorization and business state at execution time.

Avoid this mistake Assuming execute can safely run twice or that every business command has an exact inverse.

Interviewer probes What result should a retry return after the first response was lost? / How would you evolve a persisted command payload across application releases?

Strong answer criteria Separates encapsulation from execution guarantees; Defines durable identity and replay behavior; Uses compensation where reversal is not exact

Verification evidence Simulate duplicate execution, conflicting payload reuse, lost response and executor upgrade; assert one effective transition and traceable recovery.

Read and verify this answer

SAP ABAP pattern examples: Command

SAP LUWs in ABAP

PAT-08 A repository interface improves testability, but its RAP implementation reads managed persistence directly. What can go wrong?

The seam may be useful while the implementation is transactionally incorrect. A direct table read can miss creates, updates and deletes still in the RAP buffer. Define whether the repository contract means persisted state or the current transactional view; for transaction-sensitive BO logic, use the appropriate EML behavior access. Do not add a generic repository that writes managed tables and bypasses RAP checks. Test doubles must reproduce the relevant contract, not merely return convenient rows.

Example and follow-up A reporting query and a validation inside a modify flow can legitimately need different read semantics. Give those contracts distinct names or responsibilities. Test an update followed by a read before save, plus create and delete cases; an isolated unit test alone will not reveal a wrong persistence shortcut.

Avoid this mistake Equating a mockable interface with correct behavior or recreating RAP's transactional buffer in a second general-purpose cache.

Interviewer probes Which caller actually needs persisted rather than buffered state? / What integration test would fail if the repository used SELECT instead of EML?

Strong answer criteria Defines explicit read semantics; Respects RAP ownership and buffer visibility; Combines seam tests with real integration evidence

Verification evidence Compare unsaved create/update/delete reads through the adapter with the BO contract, then run rollback and save paths on the target system.

Read and verify this answer

SAP ABAP pattern examples: data-access isolation examples

SAP RAP implementation contract

PAT-09 A singleton caches the current approver and makes tests pass alone but fail as a suite. How do you redesign its lifetime?

I would remove request-specific mutable state from the singleton and make dependencies explicit. A conventional ABAP singleton is scoped to its internal session, not to all users or the whole landscape, and its lifetime can exceed one test or business operation. Prefer a freshly constructed, context-aware service or a cache with explicit key, scope and invalidation. If a shared instance is justified, specify immutability or cleanup rules and provide deterministic substitution in tests.

Example and follow-up Caching the 'current user' implicitly is particularly dangerous for reusable jobs or delegated processing. Passing business context into the operation makes the dependency visible. A reset method can stabilize legacy tests temporarily, but it does not repair a production lifetime that is inconsistent with the data being cached.

Avoid this mistake Claiming singleton means one system-wide instance or solving hidden state solely by forcing test order.

Interviewer probes What exact lifetime is required by the cached value? / How would two simulated users in one test expose the defect?

Strong answer criteria Scopes singleton state accurately; Removes hidden request context; Distinguishes test cleanup from production correctness

Verification evidence Run tests in multiple orders and reuse one session across different users or contexts; assert no stale decisions and deterministic dependency replacement.

Read and verify this answer

SAP ABAP pattern examples: Singleton

SAP Clean ABAP: state lifetime

PAT-10 A service obtains every dependency from a global factory inside its methods. How would you introduce dependency injection without building a framework?

At a composition root near the application boundary, construct the service with its required interface-typed dependencies. Keep wiring separate from business decisions so tests can provide controlled collaborators and new dependencies are visible. A factory can help build a complicated dependency graph, but repeated global lookup inside domain methods hides that graph. Start with straightforward constructor injection for required services; do not introduce a container when explicit construction remains readable.

Example and follow-up Framework-owned classes may constrain construction, so create a small delegated application service or use a supported seam instead of inventing framework constructor parameters. Ensure test setup cannot accidentally fall back to a live posting client. Use integration tests to verify production wiring separately from pure decision tests.

Avoid this mistake Replacing one hidden global singleton with a service locator and calling the result fully decoupled.

Interviewer probes Which object owns construction in a framework-created RAP handler? / How would a newly added dependency become visible to existing tests?

Strong answer criteria Separates composition from domain logic; Uses explicit dependencies with framework awareness; Verifies production wiring and safe test substitution

Verification evidence Instantiate the application service with doubles, assert no live dependency access, and test the production factory's intended object graph independently.

Read and verify this answer

SAP Clean ABAP: dependency injection

SAP ABAP test isolation examples

PAT-11 A small report has one stable calculation. A reviewer requests Strategy, Abstract Factory and Repository 'for clean architecture'. How do you respond?

I would ask which real variation, dependency boundary or testing obstacle each abstraction resolves. A small named method with explicit inputs can be the better current design. Add a seam where an external dependency obstructs meaningful tests, and introduce polymorphism when distinct behaviors actually need independent evolution. Pattern count is not a quality metric. I would preserve readability and leave a concrete refactoring trigger instead of building speculative extension layers.

Example and follow-up For example, a pure tax-base calculation with one supported rule can remain a function-like method with boundary tests. A remote rate source may deserve an interface now because it fails independently. If a second rule later appears, extract a Strategy without pretending the initial absence of one was a design defect.

Avoid this mistake Equating more classes with greater maintainability or rejecting all patterns because the current implementation is short.

Interviewer probes Which expected change would justify the next abstraction? / What testing benefit can you demonstrate before adding another interface?

Strong answer criteria Connects abstraction to a concrete force; Balances simplicity with testable external boundaries; Defines an observable refactoring trigger

Verification evidence Compare change scope and tests for the current design and proposed abstraction; retain only boundaries with a demonstrated maintenance or testing benefit.

Read and verify this answer

SAP Clean ABAP: abstraction level

SAP Clean ABAP: focused responsibilities

PAT-12 A factory selects country approval Strategies from configuration. How do you keep that combination safe, explainable and testable?

Use a small explicit mapping from supported configuration values to eligible Strategy implementations, then inject the selected strategy into the coordinator. Validate missing and unsupported configuration instead of silently defaulting to a permissive rule. Keep strategy selection separate from its business decision and preserve a common result contract. If dynamic construction is used, it still needs controlled class selection and release-compatible dependencies; it is not a way around ABAP Cloud API restrictions.

Example and follow-up Record which policy version produced the decision when business audit requires it. Test selection independently from each rule and run a common contract suite across strategies. Configuration changes must not make an in-flight approval unpredictably switch policy; decide whether it uses current policy or a recorded policy version.

Avoid this mistake Allowing arbitrary class names from untrusted input or implementing a 'default approve' path when configuration lookup fails.

Interviewer probes What should happen to a pending request after the configured policy changes? / Which tests distinguish factory-selection failure from a correct rule rejecting the request?

Strong answer criteria Separates selection and execution; Fails safely on configuration errors; Defines versioned policy and common contracts

Verification evidence Test supported and unknown countries, policy changes during pending approvals and identical contract assertions against every Strategy.

Read and verify this answer

SAP ABAP pattern examples: Strategy

SAP ABAP pattern examples: object creation alternatives

Official references

SAP ABAP pattern examples: Strategy

SAP ABAP pattern examples: Template Method

SAP ABAP pattern examples: Abstract Factory

SAP Factory Method pattern

SAP ABAP pattern examples: Adapter

SAP ABAP pattern examples: Facade

SAP ABAP pattern examples: Decorator

SAP Clean ABAP: explicit failure flow

SAP ABAP pattern examples: Chain of Responsibility

SAP Observer pattern

SAP RAP business events

SAP ABAP pattern examples: Command

SAP LUWs in ABAP

SAP ABAP pattern examples: data-access isolation examples

SAP RAP implementation contract

SAP ABAP pattern examples: Singleton

SAP Clean ABAP: state lifetime

SAP Clean ABAP: dependency injection

SAP ABAP test isolation examples

SAP Clean ABAP: abstraction level

SAP Clean ABAP: focused responsibilities

SAP ABAP pattern examples: object creation alternatives

ABAP SQL, HANA, and performance

Start with correct result semantics, then measure execution cost. Be ready to explain row multiplication, empty selection drivers, NULL values, and transaction-sensitive buffering before recommending a faster query. HANA-specific techniques and tool availability depend on the target edition and release.

A join repeats the header once for every matching item

Example: sum the order totals before and after a one-to-many join. All amounts are EUR.

ORDERS: 2 rows

Order	Header total	Currency
10	100	EUR
20	80	EUR

ITEMS: 3 rows

Order	Item	Product
10	1	A
10	2	B
20	1	C

JOIN ON order ID

RESULT: 3 rows

Order	Item	Header total	Currency
10	1	100	EUR
10	2	100	EUR
20	1	80	EUR



Fix the grain: aggregate at the intended business level before combining results.
SUM(DISTINCT amount) is not a general fix: different orders can have equal totals.

Outer join caveat: a WHERE predicate on the missing right side can remove unmatched left rows.

Joining two order headers to three items repeats order 10's total, inflating a naive sum from EUR 180 to EUR 280. For outer joins, matching conditions in ON can preserve unmatched left rows, while right-side conditions in WHERE can remove them.

See the visual atlas for the recall prompt, limits and related questions.

SQL-01 How do ABAP SQL, Native SQL, CDS, and AMDP differ?

ABAP SQL provides database access integrated with ABAP types and supported platform semantics. CDS defines reusable data models with additional semantics. Native SQL exposes database-specific capabilities with more responsibility on the developer. AMDP manages database procedures through ABAP repository objects, commonly using SQLScript on HANA. I choose the simplest supported option that meets the requirement.

Example and follow-up A filtered aggregate usually needs ABAP SQL or CDS, not a database procedure. A complex HANA-specific operation may justify AMDP after measurement. Release availability, client handling, authorization, and dependency release status must be checked for the chosen path.

Avoid this mistake ABAP syntax around an AMDP method does not turn its SQLScript body into ABAP SQL or automatically apply CDS access control.

Read and verify this answer

SAP ABAP SQL examples and semantics

SAP AMDP use and selection guidance

SQL-02 What is the empty-table trap in FOR ALL ENTRIES?

If the driver internal table is empty, FOR ALL ENTRIES can cause the WHERE condition to be ignored and read all rows in scope. I guard the selection with a nonempty check and define the empty-input result explicitly. I also consider duplicate driver values, result duplicate elimination, and whether a join expresses the requirement better.

Example and follow-up For lt_customer_ids, an empty input should usually produce an empty result rather than all customers. Deduplicating repeated driver keys can reduce work. Because duplicate result rows are eliminated, omitting distinguishing columns may collapse records the application intended to keep separate.

Avoid this mistake An empty FAE driver does not mean no rows; a separate filter in the same WHERE clause is not a safe replacement for the guard.

Read and verify this answer

SAP FOR ALL ENTRIES selection semantics

SQL-03 When would you use a join instead of FOR ALL ENTRIES?

I favor a join when related database sources and their matching rules can be expressed clearly in one set-based operation. FOR ALL ENTRIES can be useful when driver data already exists in ABAP and the release or access constraints support it. Neither is universally faster; I compare semantics, transferred rows, buffering, and the measured execution plan.

Example and follow-up An order-to-item join naturally returns one row per matching item, while FAE may eliminate duplicates. If the caller needs orders only, EXISTS or a suitable projection may express the requirement more directly. Benchmark realistic volumes, including sparse and skewed keys.

Avoid this mistake Replacing FAE with a join can change duplicates and row counts even when the new query runs faster.

Read and verify this answer

SAP ABAP SQL examples and semantics

SAP FOR ALL ENTRIES selection semantics

SQL-04 How can a LEFT OUTER JOIN accidentally behave like an inner join?

A left outer join preserves unmatched left rows and supplies NULL for the missing right side. A WHERE condition requiring a right-side value can reject those unmatched rows. If the condition belongs to matching, I put it in ON; if it is intended to filter the final result, WHERE may be correct.

Example and follow-up To list every order with its optional English description, the language condition often belongs in ON. Putting description-language equals English in WHERE can remove orders with no matching description. The intended business result determines placement, not a mechanical rule.

Avoid this mistake Moving conditions between ON and WHERE is not always semantics-preserving for outer joins.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-05 What is the difference between SQL NULL and an ABAP initial value?

NULL represents missing or unknown SQL information; an ABAP initial value is a defined value for its type. SQL predicates use three-valued logic, so comparison to NULL does not behave like comparison to zero or an empty string. I use IS NULL, COALESCE, or supported null indicators when the distinction matters.

Example and follow-up After an outer join, a missing amount and a real zero amount are different business facts. Plain transfer into an ABAP field can make both appear initial unless the query or result handling preserves the distinction deliberately.

Avoid this mistake COALESCE replaces NULL, not every initial or blank value, and WHERE column = NULL is not a correct null test.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-06 How do GROUP BY, WHERE, and HAVING work together?

WHERE selects input rows before grouping. GROUP BY defines the groups, and aggregate expressions calculate a result per group. HAVING filters those groups using group-level conditions. I push suitable filtering and aggregation to the database while preserving currency, unit, and join semantics, rather than moving a large raw dataset to ABAP unnecessarily.

Example and follow-up For sales totals by company and currency, first restrict the posting period, group by company and currency, then apply a threshold to each total. Joining items to several partners before summing may multiply each item amount and produce an inflated total.

Avoid this mistake Database aggregation is not automatically correct if different currencies are mixed or joins multiply the rows being aggregated.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-07 What do cardinality and uniqueness mean for join correctness?

Cardinality describes how many matching rows a relationship can produce. The complete join condition and actual data determine the result. Incorrect to-one assumptions can hide multiplication or mislead optimization. I verify keys and optionality and test missing and multiple matches. A declared CDS key or association cardinality alone does not repair inconsistent data.

Example and follow-up Joining a document only by its number can be wrong if company code or year also belongs to its identity. Language-dependent text and time-dependent validity records are common sources of unexpected extra rows. Test those dimensions explicitly.

Avoid this mistake DISTINCT can conceal an incomplete join condition while leaving aggregates or selected attributes logically wrong.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-08 Does SELECT SINGLE return a predictable first row?

SELECT SINGLE is suitable when the conditions identify the required single record or any matching record is genuinely sufficient. Without unique conditions it does not define a business ordering. If I need the latest or highest-priority record, I use a supported limited selection with an explicit deterministic ORDER BY and a tie-breaker.

Example and follow-up For the latest status, order by the event timestamp and a unique sequence identifier before limiting to one row. A query without ordering may return a different matching row after an index, plan, or data distribution changes.

Avoid this mistake The primary key definition or a convenient current execution plan is not an implicit ORDER BY guarantee.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-09 What do you optimize first in a slow data selection?

I first establish the correct result and a reproducible baseline. Then I inspect unnecessary rows and columns, repeated calls, missing filters, costly joins, and excessive round trips. I prefer reducing work at the data source before micro-optimizing ABAP expressions. Each change is measured with representative data and checked for unchanged business semantics.

Example and follow-up A loop issuing one SELECT for each of thousands of items may spend time on repeated database calls. A set-based fetch can help, but it must preserve authorization and transaction consistency. Fetching every column or an unbounded result can dominate memory and transfer cost.

Avoid this mistake A query that is fast on ten development rows may be unsuitable for millions of production rows or a different key distribution.

Read and verify this answer

SAP ABAP SQL examples and semantics

SAP HANA secondary indexes

SQL-10 How do ST05, SAT, SQL Monitor, and database plans complement each other?

ST05 helps inspect traced database access for a scoped execution. SAT shows ABAP runtime distribution and call behavior. SQL Monitor helps find important SQL across a broader workload. A database execution plan explains how the database processes a statement. I select the tool according to whether the bottleneck is database work, ABAP processing, or call volume.

Example and follow-up For a slow report, identify the expensive processing segment, trace a representative run, and compare executions, rows, and time. In cloud editions the available tools and permissions differ; use the supported ADT or monitoring equivalent for that environment.

Avoid this mistake The slowest individual SQL statement may not be the largest total cost if another cheap statement is executed thousands of times.

Read and verify this answer

SAP analyzing SQL statements in ADT

SQL-11 Does SAP HANA remove the need for indexes and performance tuning?

No. HANA's column storage, compression, and execution engine change the tradeoffs, but large scans, poor selectivity, row multiplication, and expensive intermediate results still matter. Secondary indexes can help specific access patterns and also consume memory and maintenance effort. I evaluate the actual table organization, workload, and execution evidence before proposing an index.

Example and follow-up A selective lookup repeatedly filtering the same attribute combination may benefit from a supported secondary index. That is different from assuming a SQL Server-style clustered index controls physical row ordering. HANA column-store index structures have their own documented behavior.

Avoid this mistake Do not say HANA needs no indexes or that every primary key is a clustered index with a predictable physical row order.

Read and verify this answer

SAP HANA secondary indexes

SQL-12 When is table buffering appropriate?

Buffering can benefit small, relatively stable data read frequently through access patterns supported by the ABAP table buffer. It is usually unsuitable for highly volatile transactional data or requirements needing immediate cross-instance freshness. I check the table's buffering configuration and statement behavior, because some query forms bypass the buffer and buffering changes consistency tradeoffs.

Example and follow-up A small customizing table may be a candidate after checking change frequency and access. An order status updated continuously is different. The ABAP table buffer is separate from HANA caches and internal tables held by an individual application session.

Avoid this mistake Buffering is not universally faster, and reading buffered data does not guarantee immediate visibility of every change across application servers.

Read and verify this answer

SAP table buffering and synchronization

SQL-13 What are normalization and denormalization, and how do they affect ABAP design?

Normalization organizes relational data around dependencies to reduce update, insert, and delete anomalies. First normal form avoids repeating groups; second removes partial dependency on a candidate key; third addresses unsuitable transitive dependencies of non-key attributes. Denormalization deliberately duplicates or precomputes data for a measured need and requires an explicit consistency strategy.

Example and follow-up Keeping a customer's current address on every order row creates update problems if those copies are intended to remain synchronized. An immutable address snapshot on a historical invoice can instead be a deliberate business requirement. Clarify the semantics before calling duplication a defect.

Avoid this mistake NoSQL is not automatically faster or better for large data, and normalization is not a rule against all intentionally stored business snapshots.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-14 When is AMDP justified, and what risks must you address?

I consider AMDP when supported ABAP SQL or CDS cannot express a necessary database-specific operation well, or measurement shows that keeping a substantial calculation on HANA avoids repeated large transfers. I account for SQLScript skills, testability, dependencies, client and authorization handling, and the release's restrictions. Database pushdown must preserve business correctness, not just reduce ABAP code.

Example and follow-up A complex analytical calculation may justify an AMDP or CDS table function after simpler options are evaluated. A plain filtered SELECT normally does not. Database logic must not assume that selecting underlying tables automatically enforces the DCL of a CDS entity.

Avoid this mistake AMDP is not a universal performance upgrade, and an AMDP method should not introduce an independent commit into an application transaction.

Read and verify this answer

SAP AMDP use and selection guidance

Official references

SAP ABAP SQL examples and semantics

SAP FOR ALL ENTRIES selection semantics

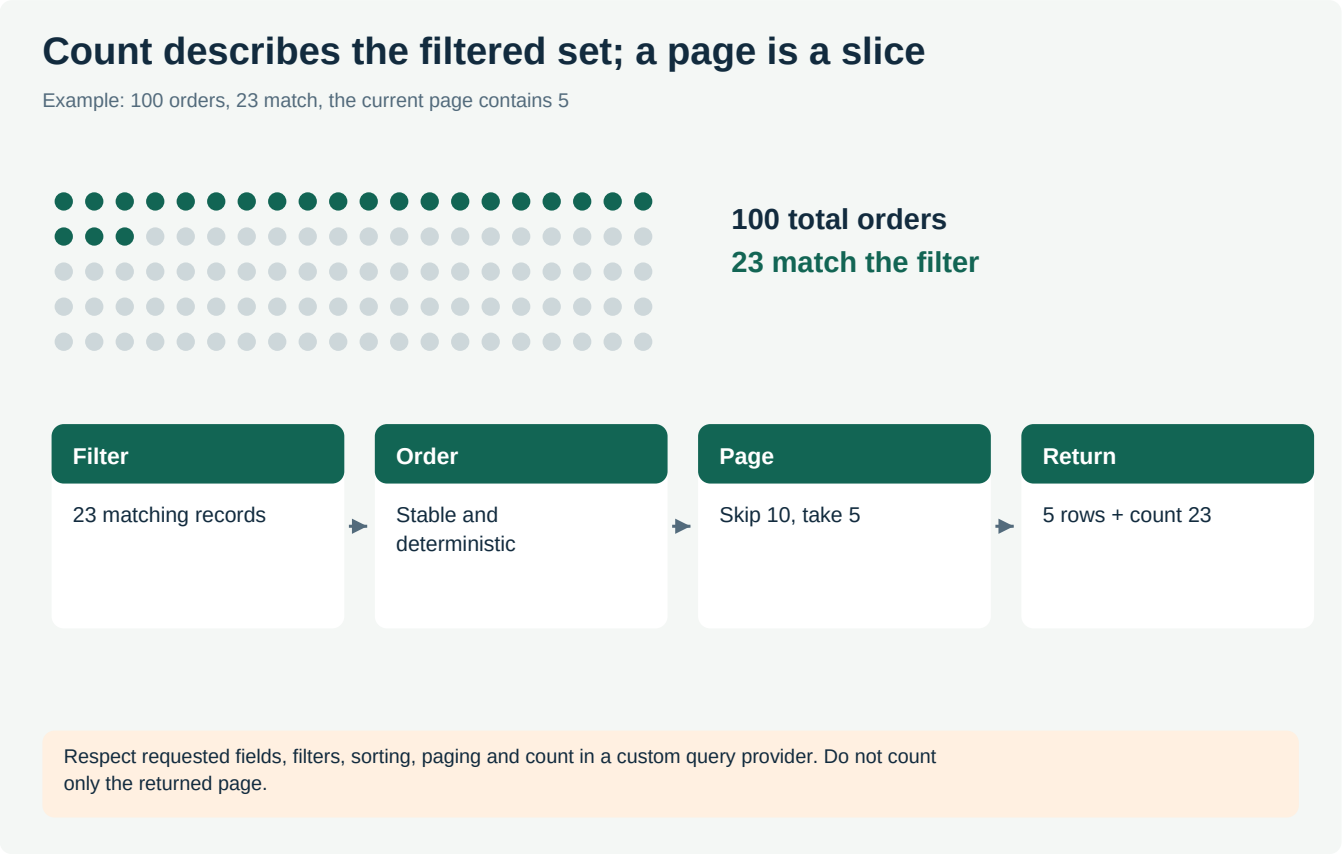
SAP HANA secondary indexes

SAP AMDP use and selection guidance

SAP analyzing SQL statements in ADT

OData Gateway and Fiori integration

Separate the OData protocol from SAP implementation technologies. Practice explaining a request, its transaction boundary and the returned metadata or error. Keep legacy Gateway knowledge while choosing supported modern bindings for new services.



Count and paging answer different questions. Stable ordering matters when the client requests successive pages.

See the visual atlas for the recall prompt, limits and related questions.

ODATA-01 When would you choose OData V4 rather than V2?

Prefer V4 for new supported transactional RAP services, while checking the actual Fiori elements or integration consumer and backend release. Keep V2 where required by an existing contract or consumer. V4 has first-class actions and functions and different protocol details; it is not simply a faster V2 switch.

Example and follow-up Draft, determinations and validations are RAP capabilities rather than exclusive V4 features. Verify draft and non-draft support for the specific UI template and SAPUI5 version.

Avoid this mistake Neither RAP nor draft automatically implies that V2 is impossible; Gateway is not exclusively V2.

Read and verify this answer

SAP Learning on OData

SAP Learning on UI service bindings

ODATA-02 What does $metadata describe?

It describes the service contract: entity types, properties, keys, relationships, operations and relevant annotations. Clients use it to understand types and available operations. Metadata is separate from instance data, and the actual endpoint must be checked after model or binding changes.

Example and follow-up A missing action button may result from an unexposed projection action, missing metadata or UI support. Inspect the contract before debugging the handler implementation.

Avoid this mistake A database field is not automatically visible in the service just because it exists in persistence.

Read and verify this answer

OASIS OData V4 protocol

ODATA-03 What are the main query options?

$select restricts properties, $filter restricts rows, $orderby sorts, $top and $skip page, and $expand includes related entities. Count and search syntax and support differ by protocol and service. A provider must apply options consistently and avoid returning unnecessarily large payloads.

Example and follow-up Request a small field set, a selective filter and a stable key-based order. Use free-text $search only where the service exposes and implements that capability.

Avoid this mistake The existence of an OData option in the standard does not guarantee that every SAP service implements it.

Read and verify this answer

OASIS OData V4 protocol

ODATA-04 How do paging and total count work in V2 and V4?

Apply filtering before count and paging, and ensure a stable order. V2 commonly uses $inlinecount=allpages, while V4 uses $count=true for an inline total; the /$count resource returns a count. Server-driven paging can return a next link that clients should follow.

Example and follow-up A total of 500 with a page of 20 is valid. Returning 20 as the total because only 20 rows were fetched breaks page navigation. Concurrent changes may still affect offset-based paging.

Avoid this mistake Do not invent one generic SET_COUNT method for every Gateway implementation; use the actual response contract.

Read and verify this answer

OASIS OData V4 protocol

ODATA-05 What is the difference between actions and functions?

In V4 a function is side-effect-free in the protocol sense and is used for retrieval or calculation. An action may have side effects and is invoked with POST. Operations can be bound to an entity or collection or be unbound. V2 exposes operation patterns through function imports.

Example and follow-up Get a calculated preview through a function when the supported service contract fits. Submit a request with an action that checks the business transition.

Avoid this mistake An HTTP GET should not create or approve business documents.

Read and verify this answer

OASIS OData V4 protocol

ODATA-06 How do ETags prevent lost updates?

A service returns a version token. The client supplies If-Match with a modifying request, and the service checks whether the current version matches. A stale token normally produces HTTP 412. The client should reload or resolve the conflict instead of silently overwriting newer changes.

Example and follow-up Test with two clients reading the same version, then updating sequentially. In RAP, configure the relevant ETag fields and update semantics correctly.

Avoid this mistake If-Match with a wildcard can defeat the intended stale-version protection; timestamps alone do not secure authorization.

Read and verify this answer

OASIS OData V4 protocol

ODATA-07 Is a batch request one database transaction?

Not necessarily. A batch groups HTTP operations; it is not automatically one atomic transaction. Multipart batch formats use changesets for atomic change operations. OData 4.01 JSON batch uses atomicity groups. These mechanisms depend on protocol version and provider support; read operations and separate groups can have different outcomes. Deep create describes creating a related structure and requires backend support for a coherent transaction.

Example and follow-up For an order and items, test a failing child and verify that the intended atomic group does not partially persist. Inspect each inner response even when the outer batch request succeeds. Check the service's protocol version and provider capabilities; a V4 endpoint does not automatically support every V4.01 batching feature.

Avoid this mistake HTTP 200 for a batch does not mean every operation inside it succeeded.

Read and verify this answer

OASIS OData V4 protocol

OASIS OData 4.01 JSON Format: Batch Requests

ODATA-08 What are MPC_EXT and DPC_EXT in classic Gateway?

The model-provider extension class extends the service model and metadata. The data-provider extension class implements or redefines runtime data operations and business error handling. Generated base classes should not host customer changes. This is a classic Gateway development approach, distinct from RAP behavior implementation.

Example and follow-up In a SEGW service, trace GET_ENTITYSET for collection retrieval and CREATE_ENTITY or a deep-create implementation for writes. Keep reusable domain logic outside the provider adapter where feasible.

Avoid this mistake Putting every rule directly in DPC_EXT makes testing and reuse harder; MPC_EXT is not intrinsically unimportant.

Read and verify this answer

SAP Learning on OData

ODATA-09 How do authentication authorization and CSRF differ?

Authentication establishes the caller. Authorization controls what that caller may access or change. CSRF protection mitigates unwanted requests using a user's browser session. SAP Gateway write flows commonly require a fetched CSRF token and matching session context; exact requirements depend on the endpoint and authentication flow.

Example and follow-up For a 403, inspect whether the failure is a missing token, service permission or business authorization. Never fix it by indiscriminately removing checks.

Avoid this mistake CSRF is not a universal OData-standard requirement and a valid token does not grant business permission.

Read and verify this answer

SAP Learning on OData

ODATA-10 How do you design dates media and errors?

Preserve the service's protocol-specific date and timestamp types and timezone meaning. Media streams require the appropriate binary endpoints, content type and content handling. Errors need suitable HTTP status, stable codes and useful messages without exposing internals. Classic Gateway and RAP use different implementation contracts.

Example and follow-up Use a date for a date-only business value and a timezone-aware instant for a recorded event. In Gateway use its exception and message-container APIs; in RAP use FAILED and REPORTED appropriately.

Avoid this mistake Edm.Date is a V4 type; do not assume identical date serialization in V2.

Read and verify this answer

OASIS OData V4 protocol

ODATA-11 How do you troubleshoot a slow or failing Fiori request?

Capture the failing network request and response, user, timestamp and correlation information. Check service metadata and relevant Gateway or application logs, then trace SQL and ABAP work in an authorized test context. Separate UI rendering, network payload, backend processing and authorization failures.

Example and follow-up A huge $expand can dominate response time even if each SQL statement is fast. Measure request duration, row counts and payload size before changing the model.

Avoid this mistake A working ADT preview does not prove a deployed app's routing, identity, metadata cache and backend authorization are correct.

Read and verify this answer

SAP Learning on OData

ODATA-12 How do you evolve an existing API safely?

Treat keys, names, types, operation semantics and error behavior as consumer contracts. Prefer compatible additions where consumers tolerate them, and version breaking changes. Test old consumers and publish a transition plan. A parallel RAP service may coexist with a legacy Gateway service during modernization.

Example and follow-up Renaming a field or changing a key can break saved UI state and integrations. Inventory consumers and retire the old contract only after migration evidence.

Avoid this mistake Building a new RAP service does not automatically migrate an existing V2 client or preserve its semantics.

Read and verify this answer

SAP Learning on UI service bindings

Official references

SAP Learning on OData

SAP Learning on UI service bindings

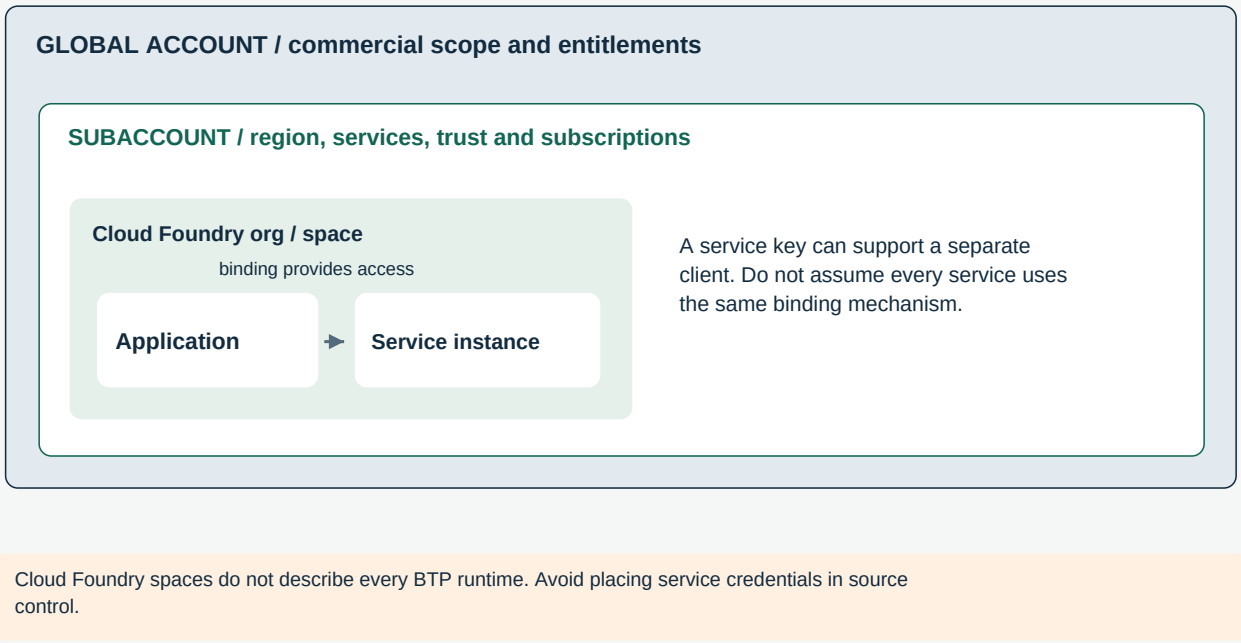
OASIS OData V4 protocol

BTP and cloud integration for ABAP developers

Focus on what an ABAP developer must design, configure or troubleshoot. Separate the ABAP development model from its hosting location, and distinguish Cloud Foundry app mechanics from the managed ABAP environment.

BTP organization and runtime are different axes

A schematic Cloud Foundry example



Cloud Foundry spaces do not describe every BTP runtime. Avoid placing service credentials in source control.

Read the nesting as an organizational example, then distinguish the application, service instance and credentials.

See the visual atlas for the recall prompt, limits and related questions.

BTP-01 What is BTP and how is ABAP Cloud different?

SAP BTP is a platform of services and environments for applications, integration and related capabilities. ABAP Cloud is a development model with restricted language and released contracts. The managed BTP ABAP environment is one place to use that model; supported S/4HANA editions also provide it.

Example and follow-up Explain whether your extension is deployed on the ERP stack or in a separate ABAP environment and how it communicates with the business system.

Avoid this mistake Do not call BTP ABAP simply a normal application container deployed with cf push.

Read and verify this answer

SAP ABAP Cloud FAQ

BTP-02 How do global accounts subaccounts entitlements and spaces relate?

A global account organizes commercial and administrative scope. Subaccounts separate regional and administrative application contexts. Entitlements allocate permission and quota to consume services. In an enabled Cloud Foundry environment, organizations and spaces structure app and service deployment. These are different layers, not synonyms.

Example and follow-up Check the service plan, region, entitlement and required roles before provisioning. The exact provisioning flow depends on the selected service and current account setup.

Avoid this mistake An entitlement is not a runtime container, and every BTP service does not require the same CF-space workflow.

Read and verify this answer

SAP BTP documentation

BTP-03 What is the difference between IaaS PaaS and SaaS?

IaaS supplies infrastructure, PaaS supplies a managed application platform, and SaaS supplies an application. The key distinction is who operates each part of the stack. In an interview explain your responsibility for code, configuration, identity and data rather than memorizing a provider list.

Example and follow-up A managed ABAP environment lets the developer focus on ABAP applications within service constraints; a SaaS business application provides an already implemented business capability.

Avoid this mistake SAP HANA Cloud is a managed database service, not a generic example of a finished business SaaS application.

Read and verify this answer

SAP BTP documentation

BTP-04 What are a service instance binding and service key?

A service instance is a provisioned service resource. A binding connects an application to that instance and supplies connection material. A service key provides credentials for a supported external client use case. A binding does not universally mean that a separately managed service-key artifact was created.

Example and follow-up In Cloud Foundry, binding data is commonly provided through VCAP_SERVICES; other delivery methods exist. Restart or restage as required by the platform and application to consume changes.

Avoid this mistake default-env.json is a local development convention, not the universal Cloud Foundry credential transport.

Read and verify this answer

Cloud Foundry service bindings

BTP-05 What do containers buildpacks droplets and routes do?

A buildpack supplies language-runtime support during staging, producing a runnable app artifact. Containers isolate running app instances. Routes direct traffic to applications, and multiple instances can serve the same route. Availability also depends on instance count, placement, dependencies and failure behavior.

Example and follow-up A blue-green deployment switches traffic between app versions, but schema compatibility and in-flight requests still need a rollout plan.

Avoid this mistake A container is not a standalone virtual machine, and one application does not always mean exactly one container.

Read and verify this answer

Cloud Foundry: how applications are staged

BTP-06 What do destinations and Cloud Connector solve?

A destination centralizes connection and authentication configuration for a supported consumer. Cloud Connector provides controlled connectivity from the cloud to permitted on-premise resources through its configured tunnel. Both require appropriate identity and authorization; neither makes an unreleased backend API supported.

Example and follow-up Trace a failed call through destination configuration, connectivity, exposed resource, certificate or token, and backend permission. Avoid copying credentials into application source.

Avoid this mistake Cloud Connector is not simply an unrestricted inbound opening of the entire corporate network.

Read and verify this answer

SAP BTP documentation

BTP-07 How do remote transactions differ from local RAP transactions?

A remote API call generally has a separate transaction boundary. A local rollback does not automatically undo a remote commit. Design orchestration with idempotency, explicit statuses, retries and compensation where needed. Do not assume distributed ACID behavior merely because both systems use SAP technology.

Example and follow-up If payment creation succeeds remotely but the local response times out, retry with a stable idempotency key or reconcile before creating another payment.

Avoid this mistake Calling HTTP inside an action does not make the remote change part of the local RAP database transaction.

Read and verify this answer

SAP BTP documentation

BTP-08 What makes event-driven integration reliable?

Publish business events consistently with committed state, use durable delivery where required, handle duplicates and define retry and dead-letter behavior. Consumers should use stable identifiers and be idempotent. Ordering, retention and delivery guarantees depend on the configured broker and protocol.

Example and follow-up A workflow starter records the request-event identifier so redelivery does not create a second approval process. Reconcile unprocessed events with business state.

Avoid this mistake Do not promise exactly-once end-to-end processing or global ordering without a specific proven design.

Read and verify this answer

SAP BTP documentation

BTP-09 Which tools matter to an ABAP developer on BTP?

ADT in Eclipse is the primary ABAP development environment. Fiori tooling such as supported SAP Fiori tools supports UI work. BTP cockpit and service-specific tools manage entitlements, identities, connections and deployment. Use the tool appropriate to the runtime rather than one generic build path.

Example and follow-up Know how to establish an ABAP project, find a released API, run ATC and tests, and troubleshoot a service consumer. Check installed tool versions against the target system.

Avoid this mistake The historical name SAP Cloud Platform and a Neo-versus-CF-only diagram are not a complete current BTP model.

Read and verify this answer

SAP ABAP Cloud FAQ

BTP-10 How would you secure and operate a side-by-side extension?

Use supported authentication, least-privilege authorization and managed credentials. Separate environments, trace requests across systems, monitor latency and failures, and define backup or recovery and deployment procedures. Treat API compatibility and dependency lifecycle as operational responsibilities as well as development concerns.

Example and follow-up A production-ready design explains who owns a failing integration, how to replay safely and how to revoke a credential. Include those points in architecture answers.

Avoid this mistake Successful connectivity from a developer machine is not evidence of secure production operation.

Read and verify this answer

SAP BTP documentation

Official references

SAP ABAP Cloud FAQ

Cloud Foundry service bindings

Cloud Foundry environment variables

SAP BTP documentation

Cloud Foundry: how applications are staged

RICEFW, enhancements, and workflow

Use this segment to connect classic delivery skills with current S/4HANA extension choices. Explain the business contract, operational recovery, and edition constraints before naming a transaction or framework. Classic SAP GUI transactions below are diagnostic examples for environments where they exist, not public-cloud instructions.

One business effect, even if the message arrives twice

Idempotency needs a key, a state check and safe persistence

Delivery	Business event ID
Attempt 1	EVT-204
Retry	EVT-204

→ **Consumer transaction**

Check event identity and current state. Apply an allowed transition and record processing safely together.

Current state	Incoming request	Result
Pending / new event	Approve	Apply once
Approved / same event	Approve again	Return prior outcome or no-op
Changed / stale version	Old approval	Reject or reconcile; do not overwrite

A check-then-write race is not solved by an in-memory set. Enforce the concurrency and persistence boundary.

The event identity and current business state decide whether to apply, ignore or reject the request.

See the visual atlas for the recall prompt, limits and related questions.

RICEFW-01 What does RICEFW mean, and how would you scope a deliverable?

RICEFW means Reports, Interfaces, Conversions, Enhancements, Forms, and Workflows. I scope the business outcome, users, source and target data, volume, dependencies, error handling, security, and acceptance criteria. I also identify the deployment edition, released extension options, operational ownership, and cutover needs. The acronym organizes work but does not decide the technology.

Example and follow-up For a supplier import, define supported fields, validation rules, duplicate handling, reconciliation totals, retry behavior, and the approved API. A count of one conversion object is not enough to estimate mapping, cleansing, testing, and recovery effort.

Avoid this mistake RICEFW is a delivery classification, not a requirement to build custom code when standard functionality already fits.

Read and verify this answer

SAP migration cockpit: objects, mapping, simulation and monitoring

RICEFW-02 How do you choose between a standard API, IDoc, migration tooling, and BDC?

I first identify the supported business interface and deployment constraints. Standard APIs, IDocs, or migration tooling can preserve application validation and provide documented contracts. In classic environments BDC can be a fallback for a suitable screen-based process, but it is sensitive to screen flow and configuration. Volume, completeness, error recovery, and supportability drive the choice.

Example and follow-up For a one-time migration, supported migration objects may be preferable to building a permanent integration. BDC is not a general option for ABAP Cloud. A BAPI's existence does not prove that it is released for the target development model or supports every needed field.

Avoid this mistake Do not call every BAPI clean-core Level B or assume every API is suitable for mass load without checking its contract.

Read and verify this answer

SAP migration cockpit: objects, mapping, simulation and monitoring

SAP IDoc control record, message type, basic type, and extension

RICEFW-03 How do BAdIs, customer exits, and user exits differ?

A BAdI exposes an object-oriented extension contract, with capabilities such as filters or multiple implementations depending on its definition. Classic customer exits include function, screen, and menu exits managed through enhancement projects. User exit often refers to older application-specific routines. I inspect the actual extension, its timing, and its supported release status before choosing it.

Example and follow-up SMOD describes classic enhancement components and CMOD manages customer enhancement projects. Some SD USEREXIT routines are a different older technique. A released BAdI is preferable when it covers the requirement, but the name BAdI alone does not establish cloud availability or upgrade guarantees.

Avoid this mistake CMOD/SMOD customer exits and all routines named USEREXIT are not interchangeable, and not every BAdI is filter-dependent or multi-use.

Read and verify this answer

SAP explicit and implicit enhancement options

RICEFW-04 What is the difference between implicit and explicit enhancement options?

Explicit enhancement options are deliberately provided in source, such as enhancement points or sections. Implicit options exist at predefined positions without an explicit marker there. A point adds implementation code, while a section can replace a defined section's implementation. Both are enhancement mechanisms; explicit enhancement does not automatically mean directly modifying SAP's original source.

Example and follow-up An enhancement spot organizes explicit options, while an enhancement implementation supplies customer code. A section replacement deserves particular care because it can bypass standard behavior. Availability and suitability differ from the released extension contracts expected in ABAP Cloud.

Avoid this mistake The distinction is not 'implicit preserves SAP code, explicit modifies it'; source modification is a separate concept.

Read and verify this answer

SAP explicit and implicit enhancement options

RICEFW-05 How would you design a reliable interface?

I define the business contract, authentication, validation, mapping, response semantics, and operational recovery. Synchronous interfaces suit immediate results; asynchronous processing can decouple systems and tolerate temporary outages. I make duplicate handling, correlation identifiers, timeouts, retries, and reconciliation explicit. Choosing OData, SOAP, IDoc, or another supported API follows those requirements and the landscape.

Example and follow-up A network timeout after posting creates an uncertain outcome: blindly retrying may post twice. A business idempotency key and a way to query the original outcome help. Retry policies should distinguish transient technical failures from permanent validation errors.

Avoid this mistake Transport-level success or an accepted message does not prove that the target business transaction was posted successfully.

Read and verify this answer

SAP IDoc communication status and application outcome

RICEFW-06 How would you troubleshoot a failed IDoc in a classic system?

I correlate the business document with the IDoc, inspect its control record, data, status history, and detailed messages, then identify configuration, mapping, master-data, authorization, or application errors. I resolve the cause before controlled reprocessing. I also determine whether the business effect already occurred and how duplicates are prevented before repeating a posting.

Example and follow-up WE02 or WE05 can support inspection, and BD87 can support reprocessing where authorized and appropriate. Partner profiles, ports, process codes, and application logs may explain the failure. The exact processing path depends on direction and configured integration.

Avoid this mistake Changing a status or repeatedly reprocessing without checking business effects can hide the cause or create duplicates.

Read and verify this answer

SAP IDoc control and message structure

RICEFW-07 What makes a data conversion safe and supportable?

I separate extraction, cleansing, mapping, validation, loading, and reconciliation. A conversion needs stable source identifiers, restart behavior, error records, and business totals that can be checked. Trial loads establish throughput and uncover data-quality issues. Cutover includes sequencing, freeze assumptions, ownership, and recovery plans, using supported business interfaces rather than uncontrolled table updates.

Example and follow-up For document migration, reconcile counts and values by company and currency, not only a total row count. Distinguish rejected records from accepted records whose follow-up processing is pending. Repeating a failed package must not recreate already posted documents.

Avoid this mistake A technically successful load is not evidence that balances, relationships, and business status are correct.

Read and verify this answer

SAP migration cockpit: objects, mapping, simulation and monitoring

RICEFW-08 How do you choose between SAPscript, Smart Forms, and Adobe output forms?

I start from the application's supported output framework, release, and document requirements. SAPscript and Smart Forms remain relevant in existing landscapes. Adobe-based output forms are common in newer scenarios and require the appropriate rendering setup. S/4HANA output management coexists with other frameworks; I do not assume a universal BRF+ migration for every application.

Example and follow-up Separate output determination from form layout and data provision. Test language, fonts, multi-page item tables, totals, attachments, recipients, and reprints. An invoice rendering correctly once does not verify email routing, spool behavior, or all business variants.

Avoid this mistake All S/4HANA output is not automatically BRF+ plus Adobe, and an output form need not be an interactive form.

Read and verify this answer

SAP S/4HANA output control and framework coexistence

RICEFW-09 How do classic Workflow, Flexible Workflow, and SAP Build Process Automation differ?

Classic SAP Business Workflow offers technical workflow modeling in the ABAP landscape. Flexible Workflow builds on SAP workflow concepts and offers configuration-oriented approval scenarios, often through Manage Workflows apps. SAP Build Process Automation runs on BTP and supports broader automation and integration. I choose according to the delivered scenario, landscape, extensibility, and operational needs.

Example and follow-up For a supported purchase approval scenario, first evaluate the delivered Flexible Workflow features. For a cross-system custom process, a BTP option may fit. A custom RAP application does not automatically receive a complete approval workflow merely because it has a submit action.

Avoid this mistake Flexible Workflow is not a completely unrelated replacement engine, and identical features are not available in every application scenario.

Read and verify this answer

SAP workflow solution overview

SAP flexible workflow scenario and configuration concepts

RICEFW-10 What are a workflow container, task, work item, and agent?

The container holds process context, and bindings map data between workflow components. A task defines work to perform; a work item is a runtime instance. Agent determination identifies the responsible processor, considering the scenario's rules and restrictions. A complete process also needs decisions, deadlines, exception handling, and a reliable update to the business object.

Example and follow-up For an approval, the context might carry a request identifier and amount rather than a stale copy of every field. If no valid approver is found, the design needs an explicit error or fallback path; silent continuation would violate the approval requirement.

Avoid this mistake A notification email is not the work item itself, and being selected as an agent does not replace business authorization checks.

Read and verify this answer

SAP workflow solution overview

RICEFW-11 How should a RAP submit action start an approval process reliably?

The action should validate the requested transition and update the business object through its transaction contract. Starting an external process must be coordinated with successful persistence using supported event or integration mechanisms. I design correlation, retries, and idempotency so rollback does not leave an orphan approval and duplicate delivery does not create duplicate workflows.

Example and follow-up A conceptual flow is Draft or Created to Submitted, committed event delivery, workflow creation, then an authorized approval callback. Actual draft activation is a separate RAP operation unless the application explicitly orchestrates it. Available event mechanisms depend on release and deployment.

Avoid this mistake Calling an external workflow endpoint before save succeeds can start approval for a document that never committed.

Read and verify this answer

SAP flexible workflow scenario and configuration concepts

RICEFW-12 How do you handle duplicate events, stale approvals, and callback retries?

I assume retries can occur and make the consumer identify an already processed business event or process instance. Approval callbacks validate the current document state, version, actor, and correlation before applying a permitted transition. A repeated successful callback should produce a stable outcome. Conflicting or stale decisions need an explicit business response and audit trail.

Example and follow-up If a submitted request is withdrawn or materially changed, an old approval must not silently approve the new version. Store enough correlation to distinguish request versions and workflow instances. Deadline escalation also needs a clear rule for a concurrent original approver decision.

Avoid this mistake Exactly-once business behavior does not follow automatically from a queue, an event name, or an HTTP success response.

Read and verify this answer

SAP flexible workflow scenario and configuration concepts

RICEFW-13 How would you diagnose a workflow that did not start or is stuck?

I follow the chain from committed business change to event delivery, receiver or start-condition selection, workflow creation, agent determination, task execution, and callback. I inspect context bindings and logs at the failing boundary. In classic systems event traces and work-item administration help; in cloud scenarios I use the supported workflow administration and integration monitors.

Example and follow-up Where available, SWEL shows recorded event trace entries after tracing is enabled, and SWI1 helps inspect work items and logs. No work item can mean a missing event or failed start condition; an existing item without a recipient points to a different problem.

Avoid this mistake An absent trace entry proves little if tracing was disabled, and a session breakpoint may not catch background processing under another user.

Read and verify this answer

SAP workflow solution overview

RICEFW-14 How would you test a workflow or enhancement end to end?

I test the happy path and rejected, missing, duplicate, concurrent, and recovery cases. For workflow that includes no approver, substitution, deadline escalation, stale decisions, and failed callbacks. For enhancements I verify when the extension runs, the data it may change, and its effect on the surrounding transaction. Evidence must include the real integration boundaries.

Example and follow-up A useful approval test changes a request after submission and attempts an old decision, then checks state and audit history. A mocked endpoint can verify request composition, but only a real authorized integration test can establish delivery, identity, and callback behavior.

Avoid this mistake A successful unit test or inbox screenshot alone does not prove business authorization, persistence, and reliable process completion.

Read and verify this answer

SAP flexible workflow scenario and configuration concepts

RICEFW-15 Which S/4HANA changes can affect existing ABAP developments?

Check target-release simplification items and custom-code findings before changing the application. Relevant areas include inventory data modeling around MATDOC, the Universal Journal around ACDOCA, Business Partner and CVI, and application-specific output and transaction changes. Compatibility mechanisms and migration prerequisites vary; do not infer behavior from a table name alone.

Example and follow-up Inventory direct reads, writes and called transactions, then verify each affected business process. Business Partner integration does not mean every customer and supplier table has simply vanished. Compare functional results and performance on the actual target release.

Avoid this mistake Replacing a legacy table name with MATDOC or ACDOCA mechanically is not a migration strategy.

Read and verify this answer

SAP S/4HANA 2025 upgrade guide

Official references

SAP explicit and implicit enhancement options

SAP S/4HANA output control and framework coexistence

SAP workflow solution overview

SAP flexible workflow scenario and configuration concepts

SAP S/4HANA 2025 upgrade guide

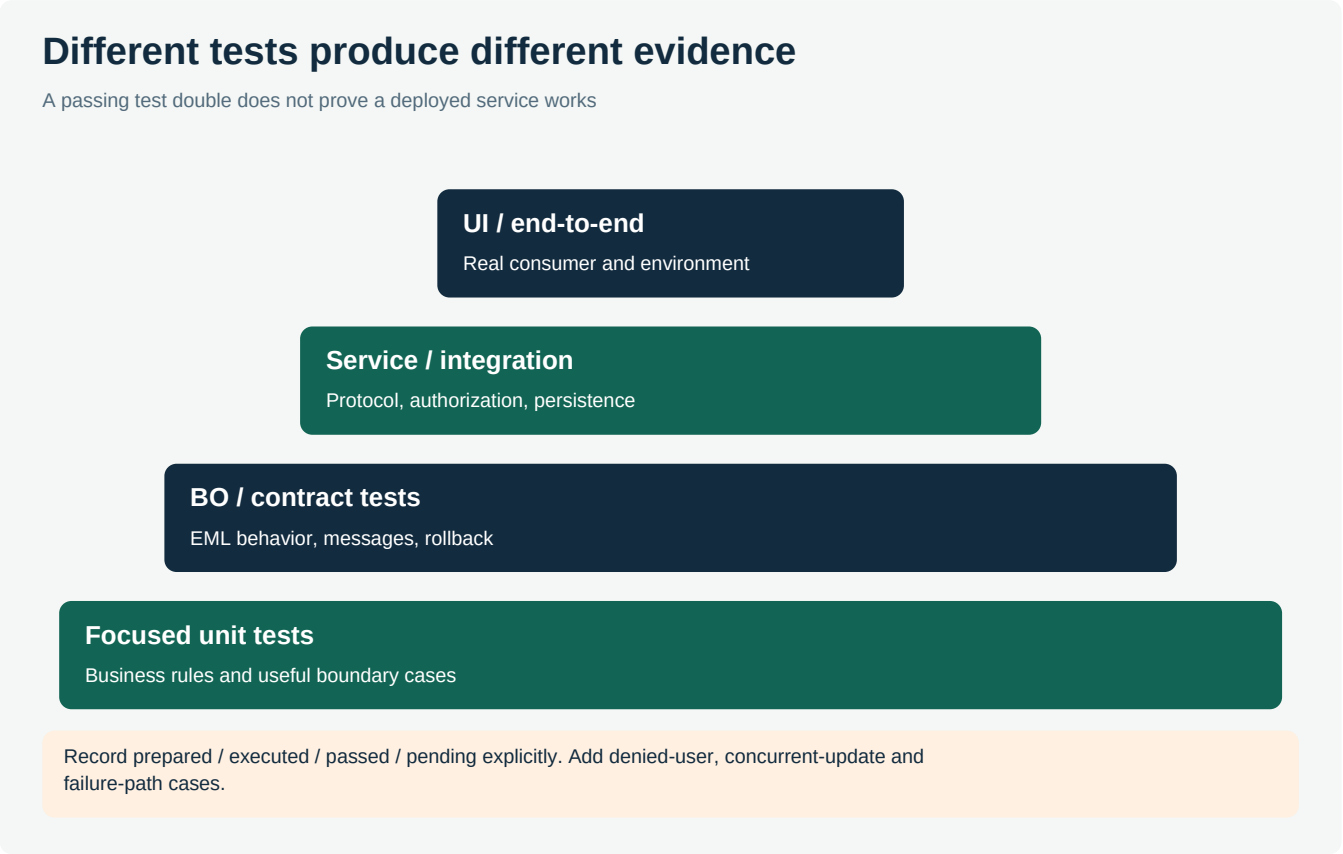
SAP migration cockpit: objects, mapping, simulation and monitoring

SAP IDoc control record, message type, basic type, and extension

SAP IDoc communication status and application outcome

Testing security and delivery

Strong candidates explain how they know a solution works. Distinguish isolated unit tests, database or CDS tests, BO integration tests and real UI or service tests. State precisely which environment and behavior each result covers.



The widening base suggests many focused checks, but there is no mandatory numerical ratio.

See the visual atlas for the recall prompt, limits and related questions.

TEST-01 What makes a useful ABAP Unit test?

Test observable behavior with controlled inputs and explicit expected results. Keep tests deterministic, isolated and understandable. Replace external dependencies at deliberate boundaries rather than relying on shared production data. Include relevant negative and boundary cases, not just successful construction or trivial getters.

Example and follow-up For a date validator, test a valid interval, equal dates and an invalid interval. Name the test after the business rule so a failure explains what broke.

Avoid this mistake High statement coverage alone does not prove that assertions or business cases are meaningful.

Read and verify this answer

SAP ABAP Unit documentation

TEST-02 How does dependency injection help?

An object receives collaborators through a defined interface instead of creating all dependencies internally. Tests can supply a fake repository, clock or service client, while production supplies the real implementation. The interface should express the required behavior, not merely reproduce a large concrete class.

Example and follow-up A retry policy can be tested with a client that returns a timeout followed by success, without calling a live service or waiting real minutes.

Avoid this mistake Mocking everything, including the object being tested, removes the behavior the test should verify.

Read and verify this answer

SAP Clean ABAP testing guidance

TEST-03 How would you test a RAP business object?

Use suitable EML-driven tests to exercise the real BO behavior with controlled persistence and dependencies. Cover allowed and rejected operations, response messages, save failures, authorization and concurrency where appropriate. Add service-level tests for protocol behavior and UI tests for user flows. Framework availability depends on the target release.

Example and follow-up A Submit test checks final state, FAILED and REPORTED, and persistence only after the intended save. A separate test confirms that an invalid request does not persist.

Avoid this mistake A mock Fiori success or a doubled BO does not establish that the real RAP implementation works.

Read and verify this answer

SAP RAP documentation

TEST-04 How do SQL and CDS test doubles help?

SQL and CDS test-double frameworks isolate supported database dependencies and allow deterministic fixtures. Choose the framework matching the component under test, configure dependencies accurately and clean up the environment. They do not automatically cover authorization, network protocols or real database performance.

Example and follow-up For a CDS aggregation, insert a small dataset with multiple currencies and duplicate join targets to expose modeling errors. Verify both expected rows and absence of unwanted rows.

Avoid this mistake Passing a double-based test is not a HANA execution-plan benchmark.

Read and verify this answer

SAP ABAP Unit documentation

TEST-05 What does ATC cover and what remains outside it?

ATC performs configured static quality checks and can govern supported API and enhancement usage. Its findings depend on check variants and available components. Runtime correctness, business acceptance, deployment and full security still need other evidence. Review exemptions and update checks for target releases.

Example and follow-up Keep syntax and activation checks, unit tests, ATC and end-to-end acceptance as separate results so each failure has a clear owner and meaning.

Avoid this mistake Do not say that an ATC pass proves an application is error-free or fully Clean Core compliant.

Read and verify this answer

SAP RAP documentation

TEST-06 What security tests would you include?

Test both permitted and forbidden reads and operations with realistic user roles and organizational scopes. Check direct API calls, mass operations and hidden fields, not only the UI. Verify authentication, DCL and behavior authorization separately and inspect error messages for unintended data exposure.

Example and follow-up A user from company code A must not retrieve B's records by changing a URL or use an action on B's known key.

Avoid this mistake Feature control, UI.hidden and disabled buttons are not sufficient security boundaries.

Read and verify this answer

SAP Learning on RAP authorization

TEST-07 How do you test concurrency and rollback?

Reproduce conflicting edits with independent consumers and verify stale-version rejection and lock behavior. Inject an expected failure before persistence and confirm no unintended partial result. For remote dependencies test timeout, retry and compensation separately because local rollback cannot undo all external effects.

Example and follow-up Read the same ETag twice, save from one client and submit the other write. Then simulate a failed child operation within the intended atomic group.

Avoid this mistake Sequential happy-path tests cannot establish concurrency safety.

Read and verify this answer

SAP RAP documentation

TEST-08 What would you include in a transport and deployment handoff?

Identify repository objects, dependencies, configuration, authorization and service activation or communication steps. State target versions, checks executed and pending acceptance. Preserve the deployed source revision and rollback or recovery approach. A transport import is one step in delivery rather than the complete user experience.

Example and follow-up For a RAP service, include DCL, behavior, metadata and binding dependencies plus the UI deployment and role setup. Verify the actual production-like endpoint after import.

Avoid this mistake Local binding publication or a successful source copy is not proof of cross-system deployment.

Read and verify this answer

SAP RAP documentation

TEST-09 How would you investigate a production defect?

Capture the symptom, scope, timing, affected revision and reproducible input. Use authorized logs, dumps and traces to isolate the failing layer, preserving sensitive data. Reproduce safely, add a regression case, implement the smallest sound fix and compare behavior before rollout.

Example and follow-up For an intermittent duplicate document, examine retries and transaction boundaries rather than immediately adding a UI delay. Distinguish a code defect from configuration or data issues.

Avoid this mistake A plausible explanation without evidence should not be presented as a confirmed root cause.

Read and verify this answer

SAP RAP documentation

TEST-10 How should you describe tests you have not run?

Separate prepared test cases from executed results. State the environment, revision, actual outcome and remaining validation. If you only inspected source locally, say so. This makes the handoff actionable and shows engineering judgment rather than overstating confidence.

Example and follow-up A sample answer can say: I prepared EML scenarios and reviewed the source; activation, ABAP Unit and ATC still need the target SAP system.

Avoid this mistake Never claim a passing SAP syntax check, measured coverage or successful activation without an actual run.

Read and verify this answer

SAP ABAP Unit documentation

Official references

SAP Clean ABAP testing guidance

SAP Learning on RAP authorization

SAP RAP documentation

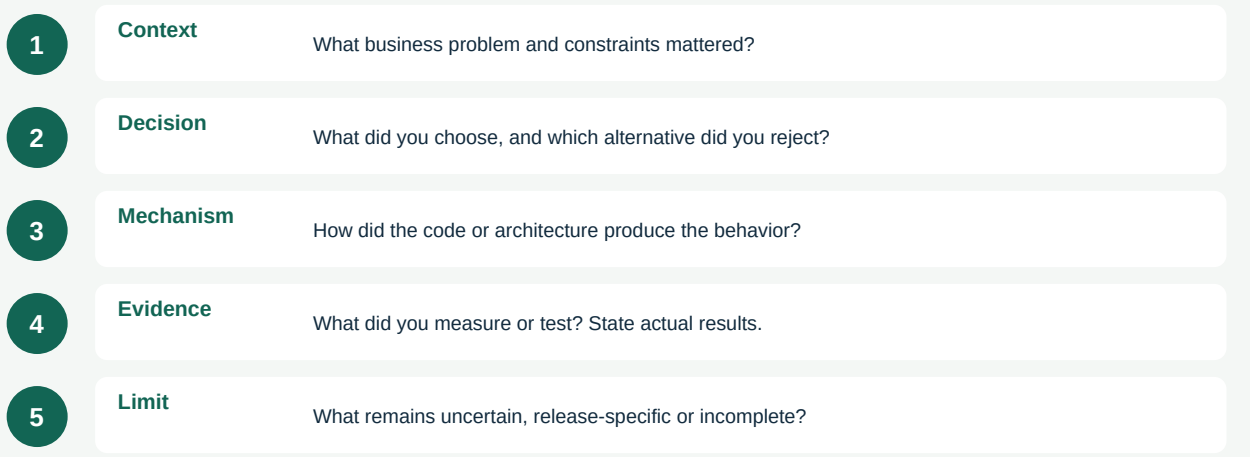
SAP ABAP Unit documentation

Interview answers and project discussion

Use these structures with your real experience. Keep the first answer to about one minute, then expand when asked. Be explicit about the part you owned, the constraints and the evidence for the outcome.

A strong interview answer has a visible chain of evidence

Build a truthful story that an interviewer can follow



Use your own experience. If you only prepared a test or proposed a design, say so explicitly.

Practice a 60-second version first, then expand the mechanism and evidence when prompted.

See the visual atlas for the recall prompt, limits and related questions.

STORY-01 How would you introduce yourself for an ABAP role?

Connect your actual ABAP experience to the role's needs: core language and data handling, the SAP landscape you worked in, representative delivery responsibilities and the modern topics you can discuss confidently. Give one concrete example and distinguish production experience from learning exercises.

Example and follow-up Use this sequence: current focus, two relevant strengths, one real project result, and why this role fits. Replace every general statement with a truthful detail before rehearsing.

Avoid this mistake Do not invent years, clients, performance metrics or RAP production experience.

STORY-02 How do you explain a project you owned?

Start with the business problem and users, then state your responsibility, constraints and design choice. Describe one important trade-off, how you tested it and the outcome. Finish with what you would improve now. Keep background short enough that the interviewer can ask technical follow-ups.

Example and follow-up Prepare one classic ABAP example and one modern service example if you have them. If the modern example is a lab, identify it as a lab and explain its limits.

Avoid this mistake Listing transactions and technologies without explaining the result does not show ownership.

STORY-03 How do you answer a question when the SAP release is unknown?

State the concept that is stable, identify the release-sensitive point and ask for the target edition or version. Explain how you would verify availability in official documentation and the target repository. Offer a supported alternative rather than pretending one syntax works everywhere.

Example and follow-up For a RAP feature, separate business intent from exact BDL syntax. For an API, check release state and the operation needed instead of guessing from its name.

Avoid this mistake Saying it depends without explaining what it depends on is not a useful answer.

STORY-04 How do you discuss a performance improvement?

Describe the initial symptom, representative workload and measurements. Explain the bottleneck found, the change made and the measured result with correctness checks. Mention relevant data volume and constraints so the result is meaningful. Use only numbers you can support from your own work.

Example and follow-up Prepare a story involving SQL round trips, selective access or internal-table keys, and be ready to explain why a different approach was rejected.

Avoid this mistake Do not claim a speedup from code appearance alone or memorize an invented percentage.

STORY-05 How would you defend an architecture trade-off?

Compare options against requirements such as transaction consistency, API availability, consumer compatibility, latency and operational ownership. State the selected option, its cost and the conditions that would change your decision. Show how the design will be validated.

Example and follow-up For on-stack versus side-by-side, discuss the local LUW requirement and remote failure handling before product preferences.

Avoid this mistake Always use RAP or always use BTP is not an architecture argument.

STORY-06 What should you ask the interviewer?

Ask which SAP editions and releases the team supports, how work is split between classic ABAP and RAP, what Clean Core policy they enforce, and how they test and deploy. Discuss the expected ownership of CDS, UI, integration and production support.

Example and follow-up Choose three questions that affect your daily work and learning priorities. Ask for an example of a recent technical trade-off rather than only a technology list.

Avoid this mistake Do not assume the role title tells you whether the work is maintenance, migration or new development.

HTTP, REST, JSON and XML

Explain the protocol, architectural style and payload independently. Practice concrete requests, payload edge cases and ABAP integration failure handling.

Read an HTTP exchange as a contract

Illustrative HTTP/1.1 layout; HTTPS protects the transport

REQUEST: client to service

GET /orders/4711 HTTP/1.1

Host: api.example.test

Accept: application/json

Method + target identify the request.

Accept asks for a representation.

Credentials are intentionally omitted.

RESPONSE: service to client

HTTP/1.1 200 OK

Content-Type: application/json

ETag: "v7"

{"id":"4711","status":"Open"}

ETag is an opaque version validator.

Transport / HTTP → **Representation / JSON**

REST is an architectural style; JSON and XML are payload formats. OData adds service conventions on top of HTTP.

Read the method, target, status and headers before interpreting the body. The sample domain is deliberately non-production.

See the visual atlas for the recall prompt, limits and related questions.

HTTP-01 What is the difference between HTTP, REST, OData and a payload format?

HTTP supplies request/response semantics. REST is an architectural style with constraints such as stateless interactions and a uniform interface. OData specifies conventions for data services. JSON and XML represent data; neither makes an endpoint RESTful.

Example and follow-up Explain four separate choices for an order API: transport semantics, architectural constraints, service conventions, and representation. In REST, stateless means each request carries the context needed to understand it, not that business data cannot persist.

Avoid this mistake Do not use REST, HTTP and JSON as interchangeable names.

Read and verify this answer

HTTP semantics: IETF RFC 9110

REST constraints: Fielding dissertation

HTTP-02 What is inside an HTTP request and response?

A request identifies a method and target, carries headers, and may contain a body. A response carries a status, headers and optional content. HTTPS adds transport protection; the business contract still determines what the operation means.

Example and follow-up For an order update, inspect the target resource, method, credentials, Content-Type, conditional headers, body and resulting status. The familiar HTTP/1.1 text layout is a teaching model; HTTP/2 and HTTP/3 frame messages differently.

Avoid this mistake A successful TCP/TLS connection does not prove that the business request succeeded.

Read and verify this answer

HTTP semantics: IETF RFC 9110

HTTP-03 How do GET, POST, PUT, PATCH and DELETE differ?

GET retrieves a representation. POST submits content for resource-specific processing. PUT creates or replaces the state of the identified target under its contract. PATCH applies a described modification. DELETE requests removal of the target resource association.

Example and follow-up For a partial update, inspect the accepted patch document or service-specific update contract. An omitted field, an explicit null and an empty string can mean different things. PATCH is not inherently idempotent; a particular patch can be designed to be.

Avoid this mistake Do not treat PUT as an arbitrary partial update or assume that every POST creates exactly one row.

Read and verify this answer

HTTP semantics: IETF RFC 9110

PATCH method: IETF RFC 5789

HTTP-04 What do safe and idempotent mean, and how do they affect retries?

Safe methods have read-oriented requested semantics. Idempotency concerns the intended effect of repeated identical requests, not identical response codes. PUT and DELETE are idempotent by their HTTP semantics; POST and PATCH need specific application guarantees for safe repetition.

Example and follow-up After a timeout on CreateOrder, the client may not know whether the server committed. Use a documented idempotency mechanism or query a stable business correlation before retrying. Repeated DELETE can return different statuses while the final resource state remains absent.

Avoid this mistake Do not blindly retry a potentially committed business creation because the response was lost.

Read and verify this answer

HTTP semantics: IETF RFC 9110

PATCH method: IETF RFC 5789

HTTP-05 Which HTTP statuses should an ABAP API developer explain?

Distinguish 200 success, 201 creation, 202 accepted but not completed, and 204 success without response content. Typical failures include 400 invalid request, 401 missing or invalid authentication, 403 refusal, 404 unavailable resource, 409 conflict, 412 failed precondition and 415 unsupported media type.

Example and follow-up A 429 or 503 can justify a bounded delayed retry under the service contract, sometimes using Retry-After. A 500 needs diagnosis. Record a correlation identifier and inspect the business error payload without exposing credentials. For asynchronous 202, explain how the caller learns the eventual outcome.

Avoid this mistake HTTP 200 on a batch envelope does not prove every contained operation succeeded.

Read and verify this answer

HTTP status reference: MDN

HTTP-06 How do Accept and Content-Type differ?

Content-Type describes the representation actually carried in a message. Accept states which response media types the client can handle. They are separate from authentication and from the URL of the resource.

Example and follow-up A client can send application/json and request an application/json response. A server can reject unsupported request content with 415 or an unacceptable response representation with 406. Always inspect the actual response type before deserializing an error page as business JSON.

Avoid this mistake Changing Content-Type does not convert an XML body into JSON.

Read and verify this answer

HTTP semantics: IETF RFC 9110

HTTP-07 What must you preserve when mapping JSON to ABAP?

JSON supports objects, arrays, strings, numbers, booleans and null. Preserve the API contract for property names, missing versus null values, number precision and ordered array entries. Do not depend on object member ordering or duplicate member names.

Example and follow-up Model an identifier such as "0004711" as a string. Decide deliberately whether a monetary value is represented as a decimal number or a string under the contract; retain currency and precision. Standard JSON has no native date type, comments, NaN or Infinity.

Avoid this mistake Do not coerce every missing or null property into the same initial ABAP value without considering update semantics.

Read and verify this answer

JSON format: IETF RFC 8259

SAP JSON transformations

HTTP-08 What XML concepts matter in an integration interview?

XML is a structured representation with elements, attributes and nested content. Well-formedness checks syntax; schema validation checks additional declared constraints. A namespace is identified by its namespace URI. An element or attribute's expanded name combines its namespace name, if any, with its local name; the chosen prefix spelling is only an alias.

Example and follow-up Two partners can use different prefixes for the same namespace, so parse by expanded name. A default namespace applies to unprefixed element names, not unprefixed attribute names. An empty element, an absent element and an explicit schema-defined nil value may differ. Escaping, encoding and parser security settings belong to the implementation contract.

Avoid this mistake Do not process arbitrary XML with string splitting or assume that well-formed XML is business-valid.

Read and verify this answer

XML specification: W3C

SAP sXML parsing

W3C Namespaces in XML

HTTP-09 How do you choose ABAP JSON and XML processing tools?

Choose a serializer/parser or transformation supported and released in the target environment. CALL TRANSFORMATION, the sXML interfaces and applicable JSON libraries solve different mapping and streaming needs. Verify naming rules, types and error handling with representative payloads.

Example and follow-up The identity transformation uses canonical ABAP representations such as asXML/asJSON; that is not automatically the same shape as an arbitrary partner API. For a large XML stream, compare a serial parser with a full DOM approach. Avoid manual concatenation of unescaped values.

Avoid this mistake Do not promise that a familiar on-premise helper class is released for ABAP Cloud.

Read and verify this answer

SAP JSON transformations

SAP sXML parsing

HTTP-10 How would you implement and troubleshoot an outbound HTTP call in ABAP?

Use the permitted destination and HTTP client APIs, configure credentials outside source, send the contractually correct request, handle transport exceptions and HTTP failures separately, then deserialize and validate the response. Apply timeouts and an explicit retry policy.

Example and follow-up In a suitable ABAP Cloud environment, released HTTP destination/client APIs and communication configuration establish the boundary; classic landscapes may use other client APIs. Diagnose DNS, TLS trust, destination, authentication, headers, payload and application logs in order. Redact secrets and sensitive business data.

Avoid this mistake Do not hard-code a password, suppress all exceptions, or assume a remote call shares the local rollback.

Read and verify this answer

SAP ABAP integration and connectivity

HTTP-11 How do authentication, authorization, CORS and CSRF differ?

Authentication establishes identity and authorization governs permitted actions. CORS is a browser-enforced cross-origin access mechanism. CSRF defenses address forged requests in relevant credential-bearing browser flows. These mechanisms solve different problems.

Example and follow-up An HTTP call working in an ABAP backend but failing in a browser can involve CORS or preflight handling. Conversely, allowing an origin does not authorize its users to approve orders. Never fix a CORS issue by removing business authorization checks.

Avoid this mistake CORS is not a general server-to-server firewall or a replacement for authentication.

Read and verify this answer

CORS: MDN

HTTP-12 How would you test an HTTP integration contract?

Test both structure and behavior: valid payload, malformed input, missing/null fields, precision, time zones, unexpected content type, business errors, expired credentials, timeouts, duplicate requests and concurrency. Include the real destination and authorization setup in integration tests.

Example and follow-up Use a controlled double for deterministic error cases, then a target-system check for actual TLS, identity, serialization and service behavior. A creation timeout test should verify that retry does not create a second business document. Preserve a redacted request, correlation and outcome as evidence.

Avoid this mistake A serializer round-trip test alone does not prove that the partner accepts your representation.

Read and verify this answer

HTTP semantics: IETF RFC 9110

Official references

HTTP semantics: IETF RFC 9110

PATCH method: IETF RFC 5789

HTTP status reference: MDN

JSON format: IETF RFC 8259

XML specification: W3C

REST constraints: Fielding dissertation

CORS: MDN

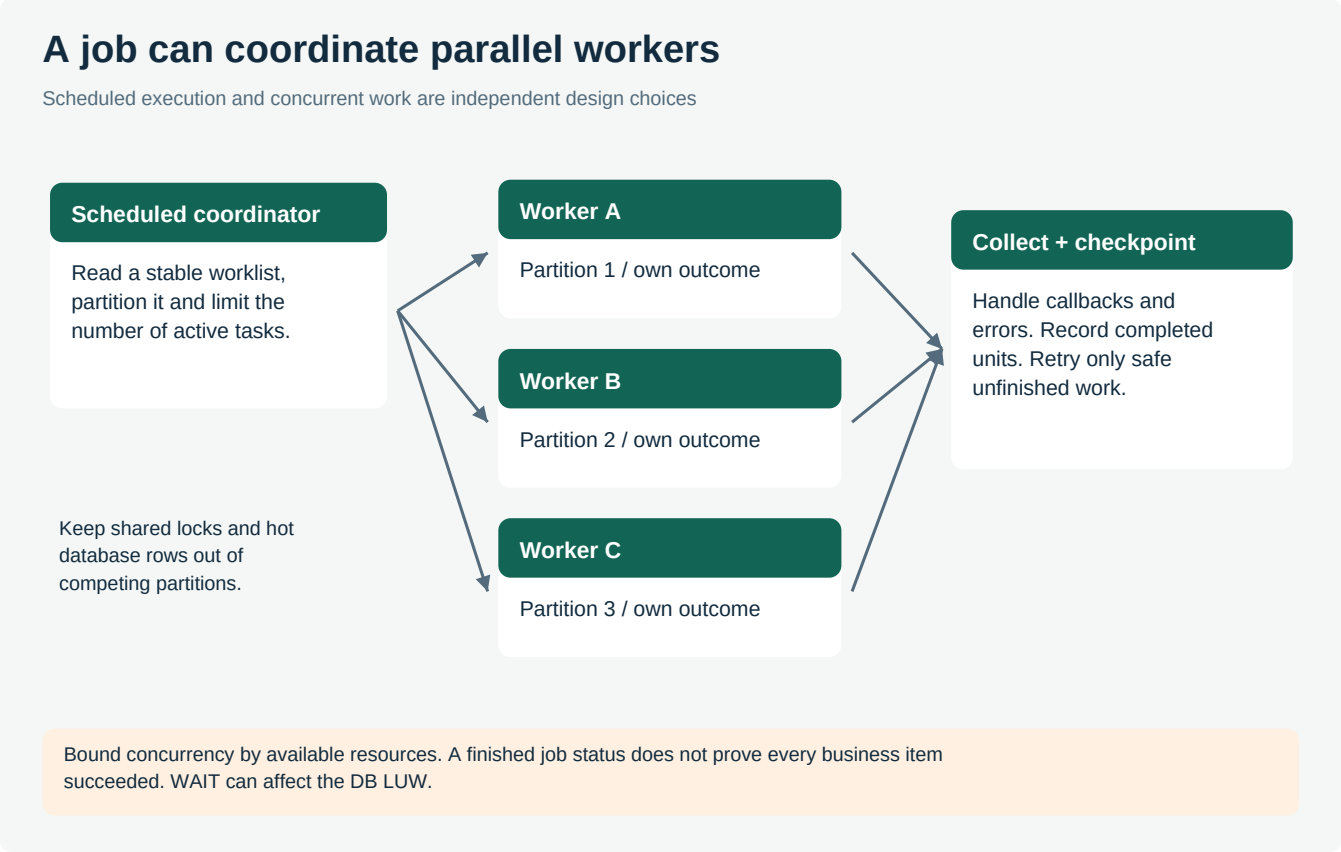
SAP JSON transformations

SAP sXML parsing

SAP ABAP integration and connectivity

Function modules, RFC, BAPI, parallel processing, and jobs

Separate the callable unit, the communication mechanism, and the business transaction. Classic RFC and job transactions remain important interview topics, but their statements and APIs are not automatically available in ABAP Cloud. Explain ownership, restart behavior, and observable outcomes before promising reliability or faster processing.



A coordinator schedules bounded independent work and collects explicit outcomes; a scheduler does not create parallelism automatically.

See the visual atlas for the recall prompt, limits and related questions.

RFC-01 How does a function module differ from an ABAP method?

A function module is a callable procedure belonging to a function group, with a defined interface and potentially shared function-group state. A method belongs to a class or interface and supports object-oriented contracts and dependency substitution. I usually organize new business logic in classes while preserving function modules required by existing APIs, RFC, or framework integration.

Example and follow-up A normal function module is not automatically remote-enabled or an update module; those are distinct attributes and invocation models. A thin, approved integration entry point can delegate calculations to a testable class without changing its external contract.

Avoid this mistake Moving logic into a method does not automatically make it released for ABAP Cloud, stateless, or safe to call remotely.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

RFC-02 How do synchronous, asynchronous, transactional, queued, and background RFC differ?

Synchronous RFC waits for the response. Asynchronous RFC lets the caller continue and optionally collect results through callbacks. Transactional RFC registers calls and parameters in the current SAP LUW; COMMIT WORK triggers processing of the registered unit rather than each call running immediately. Queued RFC adds sequencing through configured queues. Background RFC supports transactional and queued units with its own infrastructure. Their documented delivery guarantees apply to those units, not automatically to an entire distributed business process.

Example and follow-up A tRFC registration does not prove completed remote processing. ROLLBACK WORK removes the current registrations; after commit, temporary unavailability is handled through retry. sRFC and aRFC lack that durable retry model. tRFC and bgRFC type T provide documented Exactly Once unit semantics; qRFC and bgRFC type Q add ordering within the configured dependency scope. bgRFC is the successor technology; choose what the target environment permits and let the runtime own the unit's transaction. Separate business submissions can still duplicate an order.

Avoid this mistake STARTING NEW TASK is not durable background messaging, and queued processing does not imply one global order across unrelated queues.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

SAP: CALL FUNCTION IN BACKGROUND TASK

SAP: CALL FUNCTION IN BACKGROUND UNIT

RFC-03 What is the difference between a BAPI and an RFC-enabled function module?

RFC describes a mechanism for calling remote-enabled function modules. A BAPI describes a business API contract, traditionally exposed through a remote-enabled function module associated with a business object or interface. A generic RFC-enabled module is not automatically a BAPI. I check business semantics, release status, supported deployment, transaction behavior, and return messages for the particular API.

Example and follow-up A released classic BAPI may be suitable for an existing integration while not being released for direct ABAP Cloud use. Conversely, modern released business APIs can use HTTP and OData instead of a classic BAPI implementation. The name alone does not establish clean-core classification.

Avoid this mistake Do not equate remote-enabled, BAPI, released API, and clean-core Level A; they describe different properties that must be checked separately.

Read and verify this answer

SAP BAPI transaction model and documented exceptions

RFC-04 When should BAPI_TRANSACTION_COMMIT or BAPI_TRANSACTION_ROLLBACK be called, and what does WAIT mean?

The owner of a BAPI transaction decides whether to commit after evaluating the complete return contract or roll back after a failure. For remote calls, transaction control must reach the correct remote context. WAIT requests synchronous high-priority update completion; it does not wait for every downstream workflow, interface, or lower-priority activity. I verify exceptions in the specific BAPI documentation.

Example and follow-up A successful transport response is not enough: inspect all relevant business return messages and the commit result. Some documented legacy BAPIs commit internally. In RAP behavior implementations, inserting BAPI_TRANSACTION_COMMIT is not a valid way to bypass framework transaction restrictions.

Avoid this mistake An empty RETURN table is not the only possible success condition, and a local COMMIT WORK does not commit an unrelated remote BAPI context.

Read and verify this answer

SAP BAPI transaction model and documented exceptions

RFC-05 Does a remote call participate in the caller's local SAP LUW?

A remote execution has its own processing context; I do not assume one atomic rollback spans caller and receiver. I establish which side owns each transaction and whether a stateful remote session is required. If the remote change commits and the local step later fails, recovery needs an explicit reconciliation or compensating business operation.

Example and follow-up Two remote BAPI calls intended to share a transaction need the appropriate connector context, followed by transaction control in that context. Merely reusing a destination name is not a complete session-management design. An aRFC worker cannot inherit the caller's uncommitted memory or transaction buffer.

Avoid this mistake ROLLBACK WORK in the caller cannot undo an already committed remote business document or external side effect.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

SAP BAPI transaction model and documented exceptions

RFC-06 How would you collect results from CALL FUNCTION STARTING NEW TASK?

I dispatch a remote-enabled function module with a task identifier and register a callback using CALLING or PERFORMING ON END OF TASK. The callback uses RECEIVE RESULTS FROM FUNCTION to collect outputs and handle communication or system failures. I track launched, completed, failed, and timed-out work explicitly instead of assuming launch success means processing success.

Example and follow-up The callback can correlate the task identifier with its input package, then store a result or failure. The caller must remain alive and allow callback processing, using an appropriate bounded wait when needed. Callback order need not match the launch order.

Avoid this mistake An immediate successful dispatch does not mean business success, and the normal synchronous IMPORTING pattern does not collect an aRFC result.

Read and verify this answer

SAP parallel processing, callbacks, and resource groups

RFC-07 How do you prevent parallel RFC processing from exhausting resources?

In classic parallel processing I use DESTINATION IN GROUP with an appropriate RFC server group, bound the number of outstanding tasks, and size work packages deliberately. SPBT_INITIALIZE can help inspect available resources, but availability changes. I handle RESOURCE_FAILURE with bounded retry or throttling and measure whether parallelism actually helps the workload.

Example and follow-up RZ12 defines relevant RFC groups in classic systems. A batch coordinator may dispatch aRFC workers to dialog work processes; its own background status does not mean every worker is a background work process. Avoid spawning one task for every tiny row.

Avoid this mistake A resource count is a snapshot, not a reservation; hard-coded maximum concurrency can overload the system or increase lock contention.

Read and verify this answer

SAP parallel processing, callbacks, and resource groups

RFC-08 Why are WAIT and implicit database commits important in parallel processing?

Waiting and RFC boundaries can have database-transaction consequences, so I check the exact statement's documented behavior. Classic WAIT can trigger a database commit; it is not a harmless sleep around partially completed writes or an open database cursor. That database boundary does not mean all application update tasks or remote business transactions have completed successfully.

Example and follow-up Prepare work packages without relying on pending caller writes becoming visible to workers. Define explicit transaction boundaries before dispatch and wait. COMMIT WORK AND WAIT, WAIT UNTIL, and collecting aRFC results solve different problems and cannot be substituted for one another.

Avoid this mistake Neither 'RFC never commits' nor 'every RFC commits the whole business transaction' is a safe rule; statement, context, and framework restrictions matter.

Read and verify this answer

SAP WAIT UNTIL and database commit implications

RFC-09 How would you design parallel workers that can be restarted safely?

I partition independent work by stable business keys, define each worker's transaction scope, and persist enough progress to distinguish pending, successful, and uncertain outcomes. Retries must be idempotent or detect prior effects. I avoid assigning the same locked business object to concurrent workers and design reconciliation for partial completion across independently committed packages.

Example and follow-up If five packages commit and a sixth times out, restarting all six blindly can duplicate work. A package status alone may be insufficient if the posting committed before status recording failed; correlate to the actual business result using a stable identifier.

Avoid this mistake Parallel execution does not provide all-or-nothing behavior across separate worker transactions, and more workers can make a lock-bound workload slower.

Read and verify this answer

SAP parallel processing, callbacks, and resource groups

SAP classic background scheduling and monitoring

RFC-10 How do you schedule and troubleshoot a classic background job?

In a classic system I define job steps, program variants, execution identity, and start conditions, commonly through SM36. I monitor status, step details, job log, and spool through SM37, then correlate application logs, dumps, and external dependencies. Before repeating a failed job, I establish which business effects already committed and whether the input selection is stable.

Example and follow-up A periodic posting job needs an overlap rule, a defined selection window, and monitoring for missed runs. Scheduled, released, ready, active, finished, and canceled describe different execution states. A technically finished job may still have logged rejected business records.

Avoid this mistake SM37 Finished is not proof that every business record succeeded, and canceling a job does not reverse its earlier commits.

Read and verify this answer

SAP classic background scheduling and monitoring

RFC-11 How do application jobs differ from classic report scheduling in ABAP Cloud?

Application jobs use supported execution contracts and job artifacts rather than assuming classic report scheduling APIs are available. A catalog entry identifies executable logic and parameter metadata; a template supplies parameter defaults for scheduling. The Application Jobs app and released APIs support operation according to the product's capabilities. I verify the interfaces and authorization setup for the target release.

Example and follow-up One catalog can have several templates for different business selections. The executable class, catalog, template, and allowed scheduler must fit together. Application jobs also exist in supported on-premise scenarios; they are not exclusively a cloud concept.

Avoid this mistake Do not copy an unreleased JOB_OPEN or SUBMIT pattern into ABAP Cloud, or assume an interface signature from a different release.

Read and verify this answer

SAP application job implementation, catalog entries, and templates

RFC-12 Which operational and security checks make RFC or job processing production-ready?

I verify destination and execution identities, least-privilege technical and business authorization, secure connectivity, structured logs, correlation, and alert ownership. Tests include unavailable targets, exhausted resources, malformed input, duplicate requests, lock conflicts, and uncertain outcomes after timeouts. I distinguish a successful scheduling or transport acknowledgment from completed business processing and document a controlled recovery path.

Example and follow-up Classic diagnosis may use SM58 for tRFC, SMQ1 or SMQ2 for queues, and SBGRFCMON for bgRFC where available. Operators need enough authority to investigate without unrestricted payload editing or posting rights. Exercise retry scenarios with realistic volumes in an authorized test environment.

Avoid this mistake An RFC authorization check is not a replacement for application-level authorization, and a durable queue still needs operational monitoring and recovery.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

SAP classic background scheduling and monitoring

Official references

SAP RFC variants and their quality-of-service scope

SAP BAPI transaction model and documented exceptions

SAP parallel processing, callbacks, and resource groups

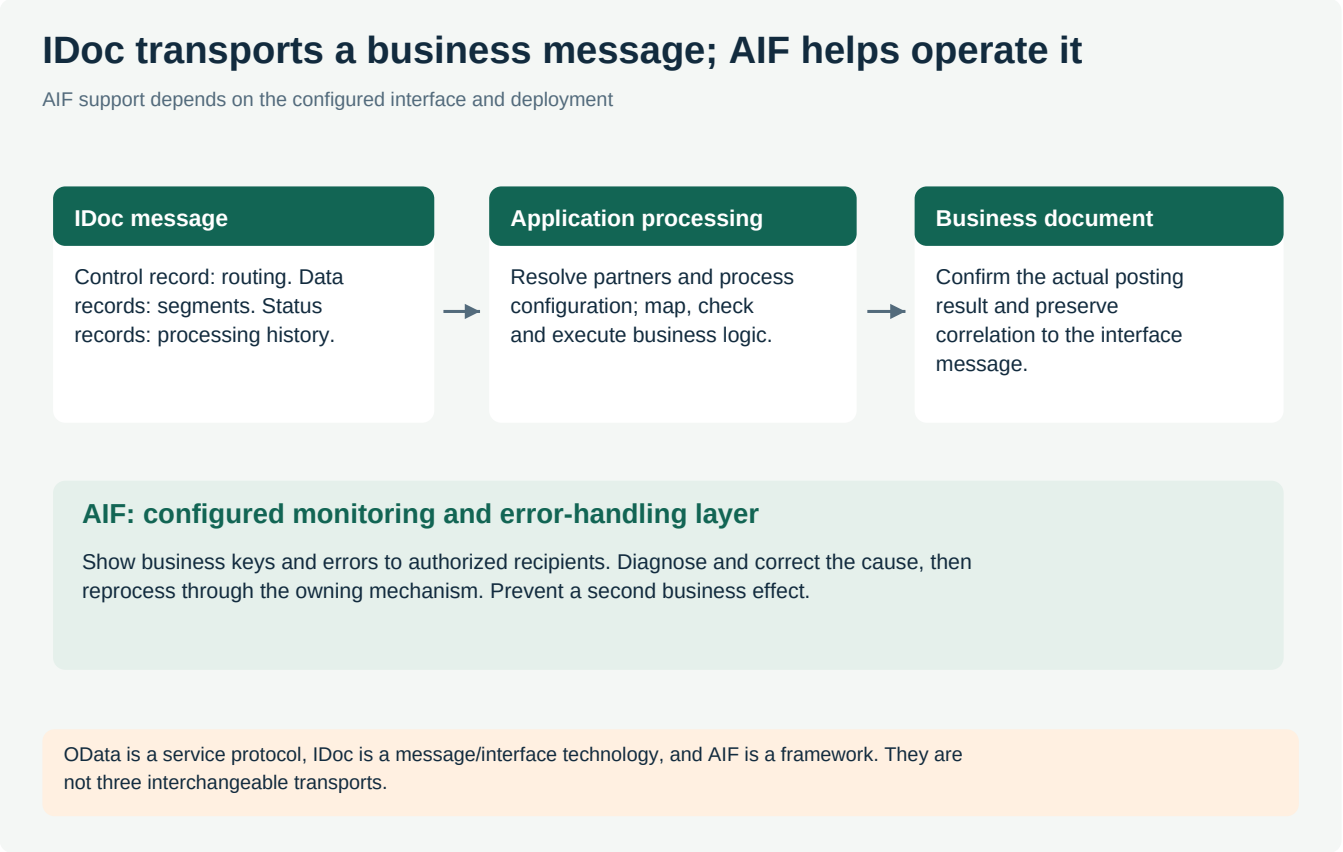
SAP WAIT UNTIL and database commit implications

SAP classic background scheduling and monitoring

SAP application job implementation, catalog entries, and templates

IDoc, ALE, AIF, and interface operations

Follow an integration message from business intent through routing, validation, posting, and recovery. IDoc is a structured message and processing technology; AIF supplies interface processing and operational capabilities for configured scenarios rather than replacing the transport. Classic transaction examples apply only where available; AIF capabilities, installed content, and usage rights depend on the product and contract.



Follow the message to the business outcome. The AIF band represents supported configured capabilities, not an obligatory stage for every IDoc.

See the visual atlas for the recall prompt, limits and related questions.

INTF-01 What does an IDoc contain, and how do message type, basic type, and extension differ?

An IDoc contains a control record identifying routing and message characteristics, data records holding segment payloads, and status records documenting processing. The message type expresses business meaning. The basic type defines the standard segment structure, while an extension adds supported customer segments. These concepts must agree with sender, receiver, and application processing capabilities.

Example and follow-up For an order-related message, the control record describes what is being sent and which structure applies. Adding a custom segment requires compatible generation, interpretation, and mapping; declaring an extension alone does not populate or process its values.

Avoid this mistake The message type is not the same as the basic type, and the IDoc document number is not automatically the target business document number.

Read and verify this answer

SAP IDoc control record, message type, basic type, and extension

INTF-02 What do ALE distribution models, partner profiles, ports, and process codes do?

ALE models describe configured distribution relationships. Partner profiles specify how particular partners and messages are processed; ports describe technical communication endpoints or mechanisms. Inbound process codes select the relevant processing implementation. I trace these settings together with logical-system identities and destination configuration rather than assuming a correctly formatted IDoc will automatically reach the right application.

Example and follow-up Classic investigation may inspect WE20 partner profiles, WE21 ports, and an applicable BD64 distribution model. Not every scenario uses the same model-driven routing. A partner is a logical or business identity; a port and an RFC destination describe different technical configuration roles.

Avoid this mistake A successful connection test proves neither correct partner-profile matching nor that the selected inbound process can post the payload.

Read and verify this answer

SAP IDoc partner and port configuration

INTF-03 Walk through outbound and inbound IDoc processing.

Outbound processing starts from an application trigger, constructs the message, determines its recipient, and dispatches it through the configured port. Inbound processing receives the IDoc, checks routing and syntax, selects the application processor, and records its outcome. Immediate versus collected or background processing changes when each stage happens, so I monitor the full chain.

Example and follow-up An outbound trigger may come from application output, change pointers, or an explicit application API, depending on the scenario. A received IDoc can exist in the receiver before business posting occurs. Sender and receiver logs must be correlated rather than assumed to share one identifier.

Avoid this mistake Creation, dispatch, receipt, and business posting are separate milestones; success at one does not establish success at all later stages.

Read and verify this answer

SAP IDoc partner and port configuration

INTF-04 How do you interpret common IDoc statuses without overstating success?

I interpret status with direction, history, and detailed application messages. Common inbound statuses include 51 for an application posting error, 53 for an application document posted, and 64 for readiness to transfer to the application. Outbound status 03 means data was passed to the port; it does not prove successful business posting in the receiver.

Example and follow-up Outbound 30 commonly means ready for dispatch. An inbound 53 is still evidence about that configured processing step, not every downstream business process. Check linked document references and scenario-specific acknowledgments when the requirement is end-to-end confirmation.

Avoid this mistake Do not treat every low-numbered status as failure or map an outbound success code directly to an inbound business success guarantee.

Read and verify this answer

SAP IDoc communication status and application outcome

INTF-05 How would you investigate an IDoc that is missing or has failed?

I start with sender, receiver, message type, business key, and time, then locate the IDoc and inspect its control data, segments, and status history. I separate generation, routing, transport, syntax, and application failures. Classic tools such as WE02 or WE05 support inspection; communication and application monitors explain failures beyond the IDoc record itself.

Example and follow-up If the sender shows dispatch but the receiver has no IDoc, investigate the technical channel and relevant RFC or middleware queue. If the receiver has status 51, inspect the application error, master data, mapping, and authorizations before changing connectivity settings.

Avoid this mistake Repeatedly testing a destination will not fix an application validation error, and an empty IDoc search may simply use the wrong partner or time range.

Read and verify this answer

SAP IDoc partner and port configuration

SAP IDoc communication status and application outcome

INTF-06 How do you reprocess an IDoc without creating duplicate business documents?

I identify and correct the root cause, confirm whether any business effect already occurred, and choose the supported reprocessing path for that status and scenario. I preserve correlation and audit history and verify the result after retry. Duplicate prevention should use a stable business identity or application contract, not only an assumption about the technical message number.

Example and follow-up BD87 supports classic status-based reprocessing where appropriate. Recreating a message or using a test copy can produce a new technical identity and bypass duplicate protection tied only to the original one. A partial or uncertain outcome needs reconciliation before another posting attempt.

Avoid this mistake Changing a status to look successful, or creating another IDoc blindly, does not repair the original error and may duplicate its business effect.

Read and verify this answer

SAP IDoc communication status and application outcome

INTF-07 What is SAP Application Interface Framework, and what is it not?

AIF provides configurable interface processing, monitoring, and error-handling capabilities so technical and business users can investigate messages in a meaningful context. Depending on the scenario, it can organize mappings, checks, actions, and recovery. It does not replace the underlying transport or automatically monitor every interface. Availability, delivered content, and custom-development usage rights must be verified.

Example and follow-up SAP distinguishes full custom-interface use from limited runtime rights supplied with particular products. Seeing an AIF monitor in a system does not establish permission to build arbitrary custom AIF interfaces. Confirm the target product's current scope and the customer's entitlement before proposing that implementation.

Avoid this mistake AIF is not a substitute for RFC, IDoc, OData, or middleware, and its presence does not establish unrestricted licensing or identical cloud capabilities.

Read and verify this answer

SAP AIF monitoring and error handling

SAP AIF licensing and limited runtime usage distinction

INTF-08 How do AIF structure mappings, value mappings, checks, and actions work together?

Structure mapping transforms the message shape, while value mapping translates business codes between representations. Checks validate applicable conditions, and actions perform configured processing such as calling application logic. I separate these responsibilities, version their contracts, and preserve useful error context. The actual execution sequence and underlying engine depend on the configured interface.

Example and follow-up An external company code may map to an internal company code, then be checked before an application posting action. A missing mapping should identify the source value and responsible owner. An action must respect the application's commit, authorization, and duplicate-handling rules.

Avoid this mistake Passing AIF checks does not replace the target application's business validation, and arbitrary commits inside processing actions can break coordinated recovery.

Read and verify this answer

SAP AIF monitoring and error handling

SAP AIF: Interface Creation

INTF-09 How would you configure useful and secure AIF monitoring?

I expose meaningful business keys for searching, assign responsibility and alerts to the correct recipients, and provide enough context to distinguish mapping, validation, and technical failures. Authorization must separate viewing, editing, restarting, and canceling messages. In classic AIF, controls can be scoped by namespace, interface, version, and activity; applicable field restrictions must also be considered.

Example and follow-up A business operator may need to inspect a failed cost-center mapping but should not automatically receive access to every payload or permission to edit amounts. Transactions such as /AIF/IFMON and /AIF/ERR are relevant where installed; cloud monitoring uses its supported apps and roles.

Avoid this mistake Recipient assignment or visibility in a monitor is not a replacement for authorization, and broad payload access can expose sensitive business information.

Read and verify this answer

SAP AIF error-handling authorization

INTF-10 What must you establish before resubmitting or editing a message in AIF?

I establish which engine owns the message, what processing already succeeded, which fields may be corrected, and what the configured restart operation actually repeats. After correcting the cause, an authorized resubmission must follow that ownership and preserve the audit trail. I do not assume every AIF interface persists or restarts data in exactly the same way.

Example and follow-up An interface using underlying IDoc processing and an interface using AIF-controlled actions can have different restart behavior. Avoid concurrent retries from both the underlying technical monitor and AIF. Verify the target document and message status after a retry, especially following a timeout.

Avoid this mistake AIF resubmission does not roll back an earlier committed document and is not automatically equivalent to creating a new business request.

Read and verify this answer

SAP AIF monitoring and error handling

INTF-11 When would you use OData, IDoc, or AIF?

OData is an HTTP-based service protocol useful for supported queries and business operations, including UI and system integration. IDoc provides structured messages and established asynchronous application integration scenarios. AIF adds processing and operational capabilities around supported interfaces. They address different concerns and can coexist; I choose the business contract and delivery model before selecting monitoring.

Example and follow-up A UI reading current request status may use OData while an established partner sends order messages as IDocs. AIF can support operational error handling where the integration is configured for it. Replacing one technology solely because another is newer ignores available contracts and recovery needs.

Avoid this mistake AIF is not a third wire protocol, OData is not limited to UI use, and IDoc is not restricted to batch-only dispatch.

Read and verify this answer

SAP AIF monitoring and error handling

INTF-12 What security and end-to-end tests would you require for an IDoc or AIF interface?

I test identity, technical and business authorization, allowed data scope, validation, and safe logging, then exercise the full path to the target business effect. Recovery tests cover duplicates, malformed segments, missing mappings, unavailable targets, ordering dependencies, and partial or uncertain outcomes. Reconciliation must compare business counts and values, not only green monitor statuses.

Example and follow-up Use controlled test data to replay a request after a simulated lost response and confirm that it does not create another document. Also test an unauthorized operator attempting payload changes or resubmission. Keep monitoring evidence linked to the business key, interface version, and execution time.

Avoid this mistake A mock transport or successful technical acknowledgment does not verify the real receiver's persistence, authorization, duplicate handling, or operational recovery.

Read and verify this answer

SAP AIF error-handling authorization

Official references

SAP IDoc control record, message type, basic type, and extension

SAP IDoc partner and port configuration

SAP IDoc communication status and application outcome

SAP AIF monitoring and error handling

SAP AIF error-handling authorization

SAP AIF licensing and limited runtime usage distinction

Supporting skills: HANA decisions and source-to-release delivery

Three focused senior code-review decisions complement the main ABAP Cloud track: choose database implementation from semantics and evidence, protect client and authorization boundaries, and distinguish source history from tested runtime delivery. The fixtures are original; target product and release remain required inputs.

Correct grain before faster execution

Fictional Order 4711: two items; each accidentally joins to two language texts

Item	Amount EUR
10	40
20	60

Correct total: 100 EUR

join →

Item	Language	Amount EUR
10	EN	40
10	DE	40
20	EN	60
20	DE	60

Wrong total: 200 EUR

Review gates before comparing SQL/CDS and AMDP

1 Restore result grain, client isolation and business authorization.
2 For a CDS table function, verify read-only and target-specific client safety.
3 Measure database work, returned rows and end-to-end time on equal results.

One million detail rows -> grouped results can reduce transfer; measure the actual plan.

A faster wrong aggregate is still wrong. General AMDP capabilities differ from the CDS table-function contract; eligible APIs and release support still apply.

A two-language text join duplicates both item amounts. Repair the relationship before accepting a performance rewrite, then verify security, client isolation and the chosen implementation contract.

See the visual atlas for the recall prompt, limits and related questions.

SQLX-01 A CDS table function joins client-dependent purchasing data in SQLScript, writes a helper table, and assumes the outer CDS DCL protects every internal read. Would you approve it?

No. For the documented CDS table-function contract, the AMDP database function is read-only, uses SQLScript on HANA and must satisfy the applicable client-safety rules. I would remove the write and review every source, join and client boundary rather than trust an outer annotation. Client isolation and business authorization are separate concerns: establish protection at the supported consumption path and do not assume SQLScript reproduces ABAP SQL or CDS access-control behavior automatically. Confirm target-release and language-version eligibility first.

Example and follow-up Use two clients containing the same business key and two users with different purchasing scopes. Verify that results, aggregates and error behavior cannot expose foreign data. Check the declared return types and result grain as well as the USING dependencies. Prefer a supported CDS or ABAP SQL expression if it already meets the requirement; a table function adds a database-specific contract.

Avoid this mistake Treating every AMDP method as read-only, or treating this read-only table-function constraint as permission to read any SAP table. General AMDP capabilities and eligible dependencies are separate.

Interviewer probes How would a cross-client join contaminate an aggregate even if the final rows are later filtered? / What evidence shows that both client isolation and the consumer's business authorization are enforced?

Strong answer criteria Distinguishes table-function restrictions from general AMDP capabilities; Reviews client safety and authorization as separate boundaries; Verifies target support, dependencies and result contract

Verification evidence Activate against the intended release, inspect dependencies and trace the real consumption path; test overlapping keys across clients and allowed/denied purchasing scopes.

Read and verify this answer

SAP CDS table functions: SQLScript, read-only and client safety

SAP ABAP data models and access-control concepts

SQLX-02 A report transfers one million item rows to ABAP to calculate totals. A rewrite uses AMDP but doubles amounts after a text join. Which implementation would you recommend?

First establish the required result grain and repair the text relationship; a faster wrong aggregate is unacceptable. Then compare a supported ABAP SQL or CDS aggregation with AMDP only if its extra database capabilities are actually needed. Push selective filters and suitable aggregation toward the database to reduce transferred data, while preserving authorization, currencies, null handling and multiplicity. Measure database work, transferred rows and end-to-end time under representative volume rather than claiming SQLScript is inherently faster.

Example and follow-up For order 4711, two items worth 40 and 60 EUR should total 100 EUR. Joining each item to two language texts produces four rows and a wrong 200 EUR total. Restrict the relationship or aggregate at the correct boundary as required by the semantics. Verify equivalent results before comparing traces; fewer returned rows alone does not prove less database work.

Avoid this mistake Using DISTINCT or dividing by a current duplication factor without fixing the grain, or selecting AMDP simply because HANA is present.

Interviewer probes Which trace observations separate excessive transfer from an expensive database plan? / How would you test multiple currencies, missing texts and duplicate text records?

Strong answer criteria Proves result equivalence before accepting optimization; Selects the simplest supported pushdown mechanism; Measures transfer, execution and full-response cost

Verification evidence Use known-total fixtures and representative datasets; compare result sets, row multiplicity, SQL trace duration/records and end-to-end timings for the alternatives.

Read and verify this answer

SAP Learning: code pushdown with ABAP SQL and CDS

SAP SQL trace analysis

DELIV-01 A pull request passes local linting and its ABAP sources are pushed with abapGit. The team calls the cloud feature deployed. What evidence and delivery steps are missing?

Git records source history; abapGit transfers supported development objects. Neither statement proves target activation or productive delivery. Establish the product-specific lifecycle route, software component and dependency order; BTP ABAP environment can use supported transport/gCTS or add-on delivery scenarios, while other ABAP Cloud environments have their own routes. Tie the source revision to target syntax/activation, ATC, meaningful ABAP Unit and integration results, then verify the imported revision, service access and configuration in the intended system.

Example and follow-up A CI result must name what actually ran and where: local lint is not SAP activation, and mocked tests do not prove runtime authorization. Include dependency versions, transport or delivery identifiers, import evidence, communication setup and recovery ownership. Keep technical identities and secrets out of Git. A source export can support review without representing a complete deployable application.

Avoid this mistake Equating an abapGit push with a transport release, treating gCTS and abapGit as interchangeable names, or applying one BTP delivery route to every ABAP Cloud product.

Interviewer probes How would you prove the tested source revision is the one imported into the target? / Which prerequisites can make an active service fail after an otherwise successful import?

Strong answer criteria Separates source control, source transfer and runtime delivery; Chooses a lifecycle route for the actual product; Requires traceable target checks and operational readiness

Verification evidence Record commit, component/dependency versions and delivery IDs; retain target activation/ATC/test/import logs and a service smoke test with an authorized and a denied identity.

Read and verify this answer

SAP: working with abapGit

SAP BTP ABAP environment: delivery via add-on or gCTS

Official references

SAP CDS table functions: SQLScript, read-only and client safety

SAP ABAP data models and access-control concepts

SAP Learning: code pushdown with ABAP SQL and CDS

SAP SQL trace analysis

SAP: working with abapGit

SAP BTP ABAP environment: delivery via add-on or gCTS